Track A: Databricks pipeline engineering

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track A: Databricks pipeline engineering 3

A0 Start here: foundations for Databricks 4

A0-L1 Spark DataFrames, the model and the core methods 4

A0-L2 The SQL that pipelines use 8

A0-L3 The lakehouse map, Databricks, Unity Catalog, Delta and medallion layers 11

A1 Code structure 16

A1-L1 Single-purpose functions and clear names 16

A1-L2 Type hints and docstrings a reviewer can trust 20

A1-L3 Config over hardcoded values, and PEP 8 with ruff 25

A2 Medallion architecture 31

A2-L1 Bronze: keep every delivery exactly as received 31

A2-L2 Silver: canonical names, checked types, one row per respondent per wave 36

A2-L3 Gold: state the grain, group by all of it, guard every ratio 40

A3 Delta Lake 46

A3-L1 Idempotent MERGE, and why you deduplicate the source first 46

A3-L2 Schema enforcement is on by default. Evolve on purpose. 52

A3-L3 Time travel, RESTORE, OPTIMIZE and VACUUM 57

A4 Unity Catalog 62

A4-L1 The three-level namespace, one catalog per environment 62

A4-L2 Grant to groups, grant the least, grant at the right level 66

A4-L3 Row filters and column masks in Unity Catalog 70

A5 PySpark performance 75

A5-L1 Lazy evaluation, and why every count() re-runs the plan 75

A5-L2 Built-in functions over Python UDFs 79

A5-L3 Joins, broadcast, partitions and reading the plan 83

A6 Data quality 88

A6-L1 Data contracts with severity levels and structured results 88

A6-L2 Statistical checks that survive empty, single-row and zero data 93
A6-L3 Reconcile counts and totals between layers 98
A7 Observability 103
A7-L1 Structured logging with the logging module, not print 103
A7-L2 A pipeline run log table with an explicit schema 107
A7-L3 Alert on failures, and on the silence that hides them 113
A8 Incremental processing 118
A8-L1 Process only the waves you have not processed, correctly 118
A8-L2 Auto Loader for file ingestion into bronze 122
A8-L3 Change data feed, so downstream layers read only what changed 127
A9 Testing 132
A9-L1 pytest basics, and a SparkSession fixture shared by every test 132
A9-L2 A fast local SparkSession and readable test data 136
A9-L3 Compare DataFrames properly, and test that MERGE is idempotent 140
A10 Deployment 146
A10-L1 Git folders, one branch per change, and no secrets in the repository 146
A10-L2 Declarative Automation Bundles and parameterised jobs 150
A10-L3 CI: lint, type-check, test and validate on every pull request 155
Track A best-practice checklist 160
Appendix: solutions and answer keys 161

Track A: Databricks pipeline engineering

Build Databricks pipelines that are correct, fast, observable and tested.

A0 Start here: foundations for Databricks

Roadmap: Prerequisite for all months. Spark DataFrames, the SQL pipelines use, and the lakehouse map: Databricks, Unity Catalog, Delta and medallion layers.

A0-L1 Spark DataFrames, the model and the core methods

Concept

Databricks runs **Apache Spark**, which splits work across many machines. You work with **DataFrames**: tables with named, typed columns, spread over the cluster. Examples use PySpark 4.0, as in Databricks Runtime 17.3 LTS. A notebook gives you a ready `spark` session.

```
1 from pyspark.sql import functions as F
2
3 df = spark.table("research_prod.research_silver.nat_respondents")
```

`spark.createDataFrame(rows, "region STRING, w DOUBLE")` builds a small DataFrame from Python tuples, which is handy in tests.

Transformations describe a new DataFrame and run nothing yet:

- `select("region", "w")` keeps columns.
- `filter(F.col("w") > 0)` keeps rows. A boolean column filters directly: `filter(F.col("in_scope"))`.
- `withColumn("w2", F.col("w") * 2)` adds or replaces a column. `F.lit(1)` is a constant, and `F.when(cond, a).otherwise(b)` picks a value per row.
- `groupBy("region").agg(F.sum("w").alias("total"), F.count("*").alias("n"))` summarises per group.
- `join(other, "region_code", "left")` and `orderBy("region")` join and sort.

Actions run the plan and return a result: `show()` prints rows, `count()` counts them, `collect()` returns a list of `Row` objects to Python, and `write` saves a table. Spark is **lazy**: nothing happens until an action, so a typo in a column name may only fail at the action.

DataFrames are **immutable**. Every transformation returns a new DataFrame and leaves the old one unchanged, so always use the result: `df = df.withColumn(...)` or `return df.withColumn(...)`.

Import the functions module as `F`. Importing `sum` or `max` from it by name hides Python's own `sum` and `max`. Collect only small results: `collect()` pulls every row onto one machine.

Why it matters

A junior engineer wrote `df.withColumn("wave", F.lit("W14"))` on its own line and saved `df`. The wave column never existed, because the result was thrown away. Every later job that grouped by wave failed, and a morning was spent looking at the wrong notebook. Assigning the result, `df = df.withColumn(...)`, is the whole fix.

Worked example: weak code

```
1 from pyspark.sql.functions import *
2
3 df = spark.table("research_prod.research_silver.nat_respondents")
2 df.filter(col("in_scope") == True)
3 rows = df.collect()
6 total = sum(r.final_weight for r in rows)
```

1. **Line 1.** Imports Spark's sum, max and others over Python's built-ins.
2. **Line 4.** The filtered DataFrame is thrown away, and == True is redundant.
3. **Line 5.** Pulls every row onto the driver, then adds them up in Python.

Worked example: strong code

```
1  from pyspark.sql import DataFrame
1  from pyspark.sql import functions as F
3
4
5  def weighted_base(df: DataFrame) -> DataFrame:
6      """Weighted base and respondent count per region, in-scope rows
       only."""
7      return (
2          df.filter(F.col("in_scope"))
9          .groupBy("region")
3          .agg(F.sum("final_weight").alias("weighted_base"),
       F.count("*").alias("n"))
11     )
```

1. **Line 2.** Spark functions stay behind F., so Python's built-ins keep their meaning.
2. **Line 8.** A boolean column filters directly, and the result is used.
3. **Line 10.** The cluster does the summing. Only the small result ever needs collecting.

Chain transformations and use their results, let Spark aggregate, and run an action only when you need the answer.

Recap card

- **Transformation against action** Transformations (select, filter, withColumn, groupBy, join) describe a new DataFrame. Actions (show, count, collect, write) run the plan.
- **Why df.withColumn(...) on its own line does nothing** DataFrames are immutable. Use the result: df = df.withColumn(...).
- **Summarise per group** df.groupBy("region").agg(F.sum("w").alias("total"), F.count("*").alias("n")).
- **How to import Spark functions** from pyspark.sql import functions as F. Never import sum, max or min from it by name.

Practice

Exercise 1. Multiple choice (level 1)

Which line makes Spark read data and compute?

- a) `kept = df.filter(F.col("w") > 0)`
- b) `totals = kept.groupBy("region").agg(F.sum("w").alias("total"))`
- c) `totals.count()`
- d) `totals = totals.orderBy("region")`

Why?

- r1) count() is an action. The other lines are transformations that only describe the plan.
- r2) count() is the last line.

Exercise 2. Predict the output (level 2)

What does this print? (PySpark 4.0)

```
from pyspark.sql import functions as F

df = spark.createDataFrame([("N", 1.0), ("N", 2.0), ("S", 4.0)], "region STRING, w DOUBLE")
out = df.filter(F.col("w") > 1).groupBy("region").agg(F.sum("w").alias("total")).orderBy("region")
print([(r.region, r.total) for r in out.collect()])
```

Exercise 3. Fill the gap (level 2)

Total the positive weights per region.

```
totals = df.filter(F.col("w") > 0).groupBy("region")._____(1)(F._____(2)("w").alias("total"))
```

Exercise 4. Write the function (level 3)

Write `region_totals(df: DataFrame) -> DataFrame`. Keep rows that are in scope (the boolean column `in_scope`) and have a positive `w`. Return one row per `region` with `n`, the number of rows, and `total_weight`, the sum of `w`. Use type hints and a docstring. Do not collect.

Spark does not run in the browser: the checker reads your code, and the CI runs the reference solution on real Spark.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Row count and weight total per region, for in-scope rows with a positive weight.
def region_totals(df: DataFrame) -> DataFrame: ...
```

Starter code:

```
def region_totals(df):
    rows = df.collect()
    out = {}
    for r in rows:
        if r.in_scope == True and r.w > 0:
            out[r.region] = out.get(r.region, 0) + r.w
    return out
```

Exercise 5. Spot the bug (level 3)

The returned DataFrame never has a `weight_band` column. Tap the line with the bug, then fix the function.

```
def add_weight_band(df: DataFrame) -> DataFrame:
    """Add weight_band: high above 3, otherwise normal."""
    df.withColumn("weight_band", F.when(F.col("w") > 3, "high").otherwise("normal"))
    return df
```

Write down the line number of the bug. Then write the fix.

Sources

- Transformations return DataFrames without running; count and collect run the plan; withColumn leaves the original DataFrame unchanged; the E2 code prints [('N', 2.0), ('S', 4.0)]. (verified: Run on PySpark 4.0.4 locally.)

A0-L2 The SQL that pipelines use

Concept

Databricks pipelines mix PySpark with Spark SQL. The examples use Spark 4.0 SQL. Run SQL from Python with `spark.sql("SELECT ...")`, which returns a DataFrame.

Read and filter. `SELECT region, final_weight FROM respondents WHERE final_weight > 0` picks columns and keeps rows.

Summarise. `GROUP BY region` makes one row per region. Aggregates work per group: `COUNT(*)` counts rows, `COUNT(final_weight)` counts non-NULL values, and `SUM` and `AVG` skip NULLs. `HAVING SUM(final_weight) > 100` filters groups after they are summarised, where `WHERE` filters rows before.

Join. `JOIN regions g ON r.region_code = g.region_code` keeps only matching rows. `LEFT JOIN` keeps every row of the left table and fills the right side with NULL when there is no match. `LEFT ANTI JOIN` keeps the left rows that have **no** match, which finds orphan keys in one line.

NULL means unknown. Any comparison with NULL is unknown, never true, so `WHERE final_weight = NULL` matches nothing. Use `IS NULL` and `IS NOT NULL`.

Pick one row per key. A delivery can repeat a respondent. A **window function** numbers the rows within each key, newest first, and you keep number 1:

```
1 SELECT respondent_id, wave_id, final_weight
2 FROM (
3   SELECT *,
4          ROW_NUMBER() OVER (PARTITION BY respondent_id, wave_id ORDER BY
    delivery_version DESC) AS rn
5   FROM bronze_deliveries
6 )
7 WHERE rn = 1
```

`PARTITION BY` sets the groups, `ORDER BY ... DESC` puts the newest first, and `ROW_NUMBER()` gives exactly one row the number 1 in each group. `DISTINCT` would keep both versions, because their values differ.

Lessons A3 and A4 add `MERGE`, `GRANT` and row filters on top of these.

Why it matters

A data-quality check counted missing weights with `WHERE final_weight = NULL`. It reported zero every night for two months while 1,800 weights were

missing, because a comparison with NULL is never true. `IS NULL` found them on the first run, and the check now catches the problem it was written for.

Worked example: weak code

```
1 SELECT DISTINCT *
2 FROM bronze_deliveries
3 WHERE final_weight != NULL;
```

1. **Line 1.** Two versions of one respondent differ in their values, so DISTINCT keeps both.
2. **Line 3.** A comparison with NULL is never true, so this returns no rows at all.

Worked example: strong code

```
1 SELECT respondent_id, wave_id, final_weight
2 FROM (
3   SELECT *,
4          ROW_NUMBER() OVER (PARTITION BY respondent_id, wave_id ORDER BY
    delivery_version DESC) AS rn
5   FROM bronze_deliveries
6   WHERE final_weight IS NOT NULL
7 )
8 WHERE rn = 1;
```

1. **Line 4.** Numbers each respondent's versions, newest first.
2. **Line 6.** IS NOT NULL is the test that works for NULL.
3. **Line 8.** Exactly one row per respondent and wave: the latest version.

Filter rows with WHERE and groups with HAVING, test NULL with IS NULL, and keep one row per key with ROW_NUMBER, not DISTINCT.

Recap card

- **WHERE against HAVING** WHERE filters rows before grouping. HAVING filters groups after aggregation.
- COUNT() against COUNT(column) COUNT() counts rows. COUNT(column) counts non-NULL values. SUM and AVG skip NULLs.
- **Find rows with no match in a lookup** LEFT ANTI JOIN lookup ON key. A LEFT JOIN keeps all rows and fills NULLs.
- **Test for a missing value in SQL** IS NULL or IS NOT NULL. = NULL is never true.
- **Keep the latest row per key** ROW_NUMBER() OVER (PARTITION BY keys ORDER BY version DESC) AS rn, then WHERE rn = 1.

Practice

Exercise 1. Multiple choice (level 1)

Which condition finds respondents whose weight is missing?

- a) `WHERE final_weight = NULL`
- b) `WHERE final_weight IS NULL`
- c) `WHERE final_weight = 0`
- d) `WHERE NOT final_weight`

Why?

- r1) A comparison with NULL is unknown, never true. IS NULL is the test made for it.
- r2) IS is faster than =.

Exercise 2. Predict the output (level 2)

`respondents` has three rows, with final_weight 1.0, NULL and 2.0. What does this return? Answer as two numbers.

```
SELECT COUNT(*), COUNT(final_weight)
FROM respondents;
```

Exercise 3. Fill the gap (level 2)

List respondents whose region code is missing from the regions lookup.

```
SELECT r.respondent_id, r.region_code
FROM respondents r
LEFT _____(1) JOIN regions g
  _____(2) r.region_code = g.region_code;
```

Exercise 4. Write the function (level 3)

`bronze_deliveries` can hold several versions of a respondent in a wave. Write a query that returns `respondent_id`, `wave_id`, `delivery_version` and `final_weight` for the latest version only: the highest `delivery_version` per respondent and wave. Use `ROW_NUMBER()`.

Shape of the answer. The reference answer follows this outline. Replace each ... with your own code.

```
SELECT ...
...
  SELECT ...
         ...
  FROM ...
...
WHERE ...
```

Starter code:

```
SELECT DISTINCT respondent_id, wave_id, delivery_version, final_weight
FROM bronze_deliveries;
```

Exercise 5. Spot the bug (level 3)

This check reports 0 missing weights every night, though weights are missing. Tap the line with the bug, then fix it.

```
SELECT COUNT(*) AS missing_weights
FROM research_prod.research_silver.nat_respondents
WHERE final_weight = NULL;
```

Write down the line number of the bug. Then write the fix.

Sources

- COUNT(*) is 3 and COUNT(col) is 2 with one NULL; WHERE col = NULL matches no rows; LEFT ANTI JOIN returns unmatched left rows; the ROW_NUMBER query keeps the highest delivery_version per respondent and wave. (verified: Run on Spark 4.0.4 SQL locally.)

A0-L3 The lakehouse map, Databricks, Unity Catalog, Delta and medallion layers

Concept

Track A uses a handful of Databricks words on every page. This lesson places each one.

Workspace, notebooks, compute, jobs. You work in a **workspace**, writing code in **notebooks**. Code runs on **compute**: **serverless** compute that Databricks manages for you, or a cluster you configure. Databricks Free Edition offers serverless compute only. A **job** runs notebooks on a schedule, with parameters (lesson A10-L2).

Unity Catalog organises data and permissions with three-level names, `catalog.schema.table`:

- a **catalog** per environment, such as `research_dev` and `research_prod`,
- a **schema** per layer, such as `research_silver`,
- a **table**, such as `nat_respondents`.

Volumes hold files, such as raw delivery CSVs, under the same names and permissions. Grants go to groups, never to individual users (lesson A4-L2).

Delta tables. A table created `USING DELTA`, the Databricks default, is a folder of Parquet data files plus a transaction log, `_delta_log`. Each write adds one numbered JSON commit to the log, so the table has versions 0, 1, 2 and so on. `DESCRIBE HISTORY t` lists them, and `SELECT * FROM t VERSION AS OF 1` reads an older one (lesson A3-L3). Readers never see a half-finished write.

Medallion layers give each table one job:

- **Bronze** keeps every delivery as received, plus audit columns: `nat_raw`.
- **Silver** has clean names and types and one row per respondent per wave: `nat_respondents`.
- **Gold** holds business-ready aggregates with a stated grain: `nat_metrics`.

Data moves forward only. A fix in silver logic means rebuilding silver from bronze, never editing bronze. Module A2 sets the contract for each layer.

Why it matters

Two analysts both wrote to a table called `respondents`, one in the development catalog and one in production, because their notebooks used two-part names and different default catalogs. Production silver received a week of test rows before anyone noticed. Three-level names, with the catalog stated explicitly or set from a job parameter, make the target impossible to mistake.

Worked example: weak code

```
❶ CREATE TABLE respondents AS SELECT * FROM csv.`/Users/anna/w14.csv`;
❷ UPDATE respondents SET final_weight = 1.0 WHERE final_weight IS NULL;
```

1. **Line 1.** A one-part name lands in whatever catalog and schema are current, read from a personal folder.
2. **Line 2.** Edits the only copy of the delivery in place, so the original values are lost.

Worked example: strong code

```
1  -- Bronze: the delivery as received, from a governed volume
❶ CREATE TABLE IF NOT EXISTS research_prod.research_bronze.nat_raw
3  AS SELECT *, current_timestamp() AS ingestion_timestamp
❷ FROM csv.`/Volumes/research_prod/landing/deliveries/w14.csv`;
5
6  -- Every write is a version you can inspect
❸ DESCRIBE HISTORY research_prod.research_bronze.nat_raw;
```

1. **Line 2.** Catalog, schema and table spelled out: production, bronze layer, the tracker's raw table.

2. **Line 4.** Files live in a Unity Catalog volume, with the same permissions as tables.
3. **Line 7.** The Delta log shows every version, who wrote it and what changed.

Name every table in full, keep files in volumes, let Delta keep versions, and move data forward from bronze to silver to gold.

Recap card

- **Unity Catalog name** catalog.schema.table, such as research_prod.research_silver.nat_respondents. Files live in volumes.
- **What makes a Delta table** Parquet data files plus _delta_log, where each write adds a numbered commit: versions 0, 1, 2.
- **Read an older version of a table** SELECT * FROM t VERSION AS OF 1. DESCRIBE HISTORY t lists the versions.
- **The three medallion layers** Bronze: as received, with audit columns. Silver: clean, one row per key. Gold: aggregates with a stated grain.

Practice

Exercise 1. Multiple choice (level 1)

In `research_prod.research_silver.nat_respondents`, what is `research_silver`?

- a) The catalog
- b) The schema
- c) The table
- d) The workspace

Why?

- r1) Unity Catalog names run catalog.schema.table, so the middle part is the schema.
- r2) Names containing silver are always schemas.

Exercise 2. Predict the output (level 2)

These statements run in order on a new Delta table. What does the last query return?

```
1 CREATE TABLE t (id INT) USING DELTA;   -- version 0
2 INSERT INTO t VALUES (1);              -- version 1
3 INSERT INTO t VALUES (2);              -- version 2
4 SELECT id FROM t VERSION AS OF 1;
```

Exercise 3. Fill the gap (level 2)

Name the NAT tracker's tables by the team convention.

```
bronze: research_prod.research_bronze.nat_____(1)
silver: research_prod.research_silver.nat_____(2)
gold:   research_prod.research_gold.nat_____(3)
```

Exercise 4. Write the function (level 3)

Write `parse_table_name(name: str) -> tuple[str, str, str]` for a job that must only ever write to fully named tables. Split the name on dots and return `(catalog, schema, table)`. Raise `ValueError` naming the input when there are not exactly three parts, or when any part is empty.

Your code must pass these checks:

- Return the catalog, schema and table as a tuple of three strings.
- Raise ValueError naming the input when the name has fewer than three parts.
- Raise ValueError for more than three parts, or for an empty part.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Split a Unity Catalog table name into catalog, schema and table.
def parse_table_name(name: str) -> tuple[str, str, str]: ...
```

Starter code:

```python
def parse_table_name(name):
    return tuple(name.split("."))
```

Exercise 5. Architecture design (level 3)

Place each item for the NAT tracker.

Options: A Unity Catalog volume, Bronze table nat_raw, Silver table nat_respondents, Gold table nat_metrics.

1. The supplier's delivery file, w14.csv, exactly as sent
2. Every row of every delivery, with ingestion_id and source_file
3. One typed row per respondent per wave, latest delivery only
4. Weighted reach by wave and region

Sources

- A Delta table's _delta_log holds one JSON commit per write (00000000000000000000.json, ...); DESCRIBE HISTORY lists versions 0, 1, 2 after a create and two inserts; VERSION AS OF 1 returns the first inserted row. (verified: Run on Spark 4.0.4 with delta-spark 4.0.1 locally.)

- Databricks Free Edition offers serverless compute only. (verified: docs.databricks.com/aws/en/getting-started/free-edition-limitations (checked 2026-09-25, VERSIONS.md).)

A1 Code structure

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Functions, type hints, docstrings, naming, config files and PEP 8.

A1-L1 Single-purpose functions and clear names

Concept

A function should do **one** of five jobs: read, transform, validate, write or log. A sixth kind, the **orchestrator**, calls the others in order and does nothing else.

Single-purpose functions pay off in three ways:

- **Tests.** A transform that takes a DataFrame and returns a DataFrame runs on a local SparkSession with three rows. A function that reads a table, transforms it and writes it needs a live catalog.
- **Reuse.** `filter_in_scope()` serves gold, QA and ad hoc analysis. A 200-line `process()` serves one notebook.
- **Change.** When the supplier renames a column, you edit one transform. Nothing else moves.

A useful test: describe the function in one sentence without "and". "Aggregate reach by wave and subgroup" passes. "Read silver and compute reach and write gold" fails three times.

Names carry the contract. Follow the standard from your reference project:

Object	Convention	Example
Function	`verb_noun`, snake_case	`check_table_health`
Class	PascalCase noun	`CheckResult`
Constant	UPPER_SNAKE_CASE	`MAX_WEIGHT_CV`
Bronze table	`[tracker]_raw`	`nat_raw`
Silver table	`[tracker]_respondents`	`nat_respondents`
Gold table	`[tracker]_metrics`	`nat_metrics`
Job	`[tracker]_[layer]_pipeline`	`nat_silver_pipeline`

A name must match what the function does. `get_` and `check_` promise no side effects. A `get_unprocessed_waves()` that also deletes rows breaks that promise, and the next reader calls it without knowing.

Why it matters

A gold notebook holds one 350-line cell: read silver, filter, weight, aggregate, write, log. A change request asks for a new subgroup. The developer edits the aggregation, and a variable named `df` later in the cell now points at the new frame. The write step saves the unfiltered data. Gold publishes reach figures that include out-of-scope respondents for two waves.

Nobody could test the cell without writing to production tables, so nobody tested it. With `filter_in_scope()` and `aggregate_reach()` as separate functions, a three-row unit test catches the mistake before the merge.

Worked example: weak code

```
❶ def process(t, w):
2     df = spark.table(t).filter(F.col("wave_id") == w)
❷     df = df.filter(F.col("in_scope") == True)
4     df = df.groupBy("wave_id",
    "region").agg(F.sum("final_weight").alias("weighted_base"))
❸     df.write.mode("overwrite").saveAsTable("research_gold.nat_metrics")
❹     print("done", df.count())
```

1. **Line 1.** The name says nothing, and `t` and `w` hide what the arguments are.
2. **Line 3.** `== True` trips ruff E712. Filter on the boolean column directly.
3. **Line 5.** The target table is hardcoded inside a function that also reads and transforms.
4. **Line 6.** Logging by `print`, and a second full scan to count rows.

Worked example: strong code

```
❶ def read_wave(spark: SparkSession, table: str, wave_id: str) -> DataFrame:
2     """Read one wave from a silver table."""
3     return spark.table(table).filter(F.col("wave_id") == wave_id)
4
5
❷ def filter_in_scope(df: DataFrame) -> DataFrame:
7     """Keep respondents flagged as in scope."""
❸     return df.filter(F.col("in_scope"))
9
10
11 def aggregate_base(df: DataFrame) -> DataFrame:
12     """Weighted base at wave x region grain."""
13     return df.groupBy("wave_id",
    "region").agg(F.sum("final_weight").alias("weighted_base"))
14
15
❹ def run_gold(spark: SparkSession, silver: str, gold: str, wave_id: str) -
    > None:
17     """Orchestrate: read, filter, aggregate, write."""
```

```
18      metrics = aggregate_base(filter_in_scope(read_wave(spark, silver,
        wave_id)))
5       metrics.write.mode("overwrite").option("replaceWhere", f"wave_id =
        '{wave_id}'").saveAsTable(gold)
```

1. **Line 1.** Read only. The session and the table name come in as parameters.
2. **Line 6.** A pure transform: DataFrame in, DataFrame out. Testable with three rows.
3. **Line 8.** Boolean column used directly.
4. **Line 16.** The orchestrator calls the steps in order and holds no logic of its own.
5. **Line 19.** Overwrite only this wave, so a rerun never wipes other waves.

Each strong function passes the one-sentence test. The weak one needs "and" four times.

Recap card

- **The five jobs a pipeline function does, one at a time** Read, transform, validate, write, log. An orchestrator calls them in order.
- **The one-sentence test** Describe the function without using "and". If you need "and", split it.
- **Function, class and constant naming** check_table_health, CheckResult, MAX_WEIGHT_CV.
- **What does a get_ or check_ prefix promise?** No side effects. The function reads and returns. It never writes or deletes.

Practice

Exercise 1. Multiple choice (level 1)

You write a function that returns a list of `CheckResult` records for one table. Which name follows the standard?

- a) `CheckTableHealth`
- b) `check_table_health`
- c) `checkTableHealth`
- d) `CHECK_TABLE_HEALTH`

Why?

- r1) Functions use snake_case and start with a verb.
- r2) Shorter names run faster.

Exercise 2. Fill the gap (level 2)

Fill in the object names for tracker `nat`, following the naming standard.

```
BRONZE_TABLE = "research_bronze._____(1)"
SILVER_TABLE = "research_silver._____(2)"
GOLD_TABLE = "research_gold._____(3)"
JOB_NAME = "_____(4)"
```

Exercise 3. Spot the bug (level 3)

This function has a name that promises no side effects. One line breaks that promise. Tap it, then fix the function so it only returns the waves.

What you need to know.

The code chains five DataFrame and session methods (PySpark 4.0):

- `df.select("wave_id")` keeps only the named columns.
- `.distinct()` drops duplicate rows.
- `a.subtract(b)` returns the distinct rows of `a` that are not in `b`.
- `.collect()` brings the rows back to Python as a list of `Row` objects. Read a field with `row.wave_id`.
- `spark.sql("...")` runs one SQL statement, such as a `SELECT` or a `DELETE`.

```
a = spark.createDataFrame([("W1",), ("W2",), ("W2",)], "wave_id STRING")
b = spark.createDataFrame([("W1",)], "wave_id STRING")
[row.wave_id for row in a.subtract(b).collect()]   # ['W2']
```

```
def get_unprocessed_waves(spark: SparkSession, bronze: str, silver: str,
    tracker_id: str) -> list[str]:
    """Return wave ids in bronze but not yet in silver for one tracker."""
    bronze_waves = spark.table(bronze).filter(F.col("tracker_id") ==
    tracker_id).select("wave_id").distinct()
    silver_waves = spark.table(silver).filter(F.col("tracker_id") ==
    tracker_id).select("wave_id").distinct()
    waves = [row.wave_id for row in
    bronze_waves.subtract(silver_waves).collect()]
    spark.sql(f"DELETE FROM {silver} WHERE wave_id IN ({','.join(repr(w)
    for w in waves)})")
    return waves
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Refactor (level 4)

Split the transform logic out of `process`. Write two pure functions:

- `filter_in_scope(df)` keeps rows where `in_scope` is true.
- `aggregate_base(df)` returns `wave_id`, `region` and `weighted_base` (sum of `final_weight`).

Neither function may read or write a table. Add type hints and docstrings.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Keep respondents flagged as in scope.
def filter_in_scope(df: DataFrame) -> DataFrame: ...

# Weighted base at wave x region grain.
def aggregate_base(df: DataFrame) -> DataFrame: ...
```

Starter code:

```python
def process(t, w):
    df = spark.table(t).filter(F.col("wave_id") == w)
    df = df.filter(F.col("in_scope") == True)
    df = df.groupBy("wave_id",
   "region").agg(F.sum("final_weight").alias("weighted_base"))
    df.write.mode("overwrite").saveAsTable("research_gold.nat_metrics")
```

Exercise 5. Architecture design (level 5)

You split a gold notebook into functions. Where does each piece of work belong?

Options: Config file, Read function, Transform function, Orchestrator.

1. The name of the gold table for tracker nat
2. `spark.table(silver).filter(wave_id == ...)`
3. Summing weights by wave and region
4. Calling read, filter, aggregate and write in turn, then logging the run

Sources

- ruff rule E712 flags comparisons to True or False. (verified: Reference README lists it; ruff 0.16.9 flags `x == True` under E712 in this repository.)
- replaceWhere overwrites only matching rows in a Delta table. (verified: Run locally on delta-spark 4.0.1 (VERSIONS.md, Track A facts).)

A1-L2 Type hints and docstrings a reviewer can trust

Concept

A type hint states a contract. `keys: list[str]` tells the caller, the reviewer and mypy that `keys` is a list of column names. A bare `list` says "some list". No hint says nothing.

Use the modern forms, valid from Python 3.10:

- Built-in generics: `list[str]`, `dict[str, int]`, `tuple[str, ...]`.
- Optional values: `str | None = None`. Never `str = None`, which claims the value is always a string.

- Fixed sets of strings: `Literal["PASS", "WARN", "FAIL"]` from `typing`. mypy then rejects `"pass"` or `"FAILED"`.
- A return type on every function, including `-> None`.

Run `mypy --strict` in CI. Strict mode refuses functions without hints, so gaps cannot creep back.

A docstring explains what hints cannot: meaning, units, grain and failure modes. Use the Google style:

```
def weighted_share(df: DataFrame, flag_col: str) -> float | None:
    """Weighted share of rows where flag_col is 1.

    Args:
        df: Respondents with final_weight.
        flag_col: 0/1 column to share.

    Returns:
        Share between 0 and 1, or None when the total weight is 0.

    Raises:
        ValueError: If flag_col is missing from df.
    """
```

The first line fits on one line and says what the function returns or does. Args, Returns and Raises follow. A reviewer reads the docstring, then checks the body against it. A mismatch between the two is a bug in one of them.

PySpark is not fully typed, and mypy cannot see column names. Hints still catch the most common mistake: passing the wrong kind of object.

Why it matters

The reference `upsert_silver` takes `merge_keys: list`. A caller passes the string `"respondent_id"` instead of `["respondent_id"]`. Python iterates a string character by character, so the join builds `t.r = s.r AND t.e = s.e AND ...`. The MERGE fails with a column-not-found error at 02:00, and the on-call engineer spends an hour reading the wrong part of the stack trace.

With `merge_keys: list[str]`, mypy flags the call in CI: a `str` is not a `list[str]`. The bug never reaches the scheduler.

Worked example: weak code

```
def log_pipeline_run(pipeline_name, layer, rows_processed, status,
    error_message: str = None):
    """Append one record per pipeline run to the governance run log."""
    record = {
        "pipeline_name": pipeline_name,
        "layer": layer,
```

```
6          "rows_processed": rows_processed,
4          "status": status,
8          "error_message": error_message,
9      }
10     return record
```

1. **Line 1.** Claims a string, defaults to None. Reference defect 7.
2. **Line 1.** No hints on four parameters and no return type. mypy --strict rejects the function.
3. **Line 2.** The docstring says it appends, but the function returns a dict. One of them is wrong.
4. **Line 7.** `status` accepts any string, so `"sucess"` slips into the run log.

Worked example: strong code

```
1  from typing import Literal
2
1  RunStatus = Literal["SUCCESS", "FAILED"]
4  Layer = Literal["bronze", "silver", "gold"]
5
6
2  def build_run_record(
8      pipeline_name: str,
9      layer: Layer,
10     rows_processed: int,
11     status: RunStatus,
3      error_message: str | None = None,
4  ) -> dict[str, str | int | None]:
14     """Build one run log record.
15
16     Args:
17         pipeline_name: Job name, for example nat_silver_pipeline.
18         layer: Medallion layer the run wrote to.
19         rows_processed: Rows written by the run.
20         status: Final status of the run.
21         error_message: Error text for a failed run, otherwise None.
22
23     Returns:
24         A record ready to append to the run log table.
25     """
26     return {
27         "pipeline_name": pipeline_name,
28         "layer": layer,
29         "rows_processed": rows_processed,
30         "status": status,
31         "error_message": error_message,
32     }
```

1. **Line 3.** A named Literal type. mypy rejects any other string at every call site.
2. **Line 7.** The name now matches the job: it builds a record. Writing it is a separate function.

3. **Line 12.** Optional, stated honestly.
4. **Line 13.** The return type matches what the body returns.

The strong version tells mypy, the reviewer and the caller the same story. The weak version tells three different ones.

Recap card

- **Optional parameter that defaults to None** param: str | None = None. Never param: str = None.
- **A fixed set of string values** Literal["PASS", "WARN", "FAIL"] from typing.
- **Google docstring sections** One-line summary, then Args, Returns, Raises.
- **Why merge_keys: list[str] and not list?** mypy then rejects a string, which Python would split into characters.

Practice

Exercise 1. Multiple choice (level 1)

A function takes an optional error message that defaults to `None`. Which signature is correct for Python 3.11 with `mypy --strict`?

- a) `def log_run(error_message: str = None) -> None:`
- b) `def log_run(error_message: str | None = None) -> None:`
- c) `def log_run(error_message=None) -> None:`
- d) `def log_run(error_message: None = str) -> None:`

Why?

- r1) The hint must include None, because None is a value the parameter takes.
- r2) Type hints change how Python runs the function.

Exercise 2. Predict the output (level 2)

The reference code builds a MERGE condition like this. A caller passes a string instead of a list. What does this print?

```
1 merge_keys = "id"
2 condition = " AND ".join(f"t.{k} = s.{k}" for k in merge_keys)
3 print(condition)
```

Exercise 3. Fill the gap (level 2)

Complete the signature. `status` accepts only `"PASS"`, `"WARN"` or `"FAIL"`, and the function returns nothing.

```python
from typing import _____(1)

def record_check(name: str, status: _____(2)["PASS", "WARN", "FAIL"]) ->
    _____(3):
    ...
```

Exercise 4. Write the function (level 3)

Write `build_merge_condition(keys, target="t", source="s")`. It returns a condition such as `t.respondent_id = s.respondent_id AND t.wave_id = s.wave_id`.

- Raise `TypeError` when `keys` is a string, not a list.
- Raise `ValueError` when `keys` is empty.
- Add type hints and a Google-style docstring with Args, Returns and Raises.

Your code must pass these checks:

- Join one `t.k = s.k` clause per key with `AND`.
- Use the `target` and `source` aliases you receive.
- A string must raise TypeError. Otherwise it is split into characters.
- An empty key list must raise ValueError. An empty condition matches nothing and inserts every row.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Build a MERGE ON condition that matches every key column.
def build_merge_condition(
    keys: list[str],
    target: str = 't',
    source: str = 's',
) -> str: ...
```

Starter code:

```python
def build_merge_condition(keys, target="t", source="s"):
    ...
```

Exercise 5. Refactor (level 4)

Type this function. Return one of `"SKIP"`, `"WARN"` or `"PASS"` as a `Literal`, accept a missing `change`, and add a Google-style docstring. Keep the behaviour.

Your code must pass these checks:

- The behaviour changed. None gives SKIP, above tolerance gives WARN, otherwise PASS.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Classify a wave-over-wave change in a share.
def classify_change(
    change: float | None,
    tolerance: float = 0.1,
) -> ChangeStatus: ...
```

Starter code:

```python
def classify_change(change, tolerance=0.10):
    if change is None:
        return "SKIP"
    return "WARN" if change > tolerance else "PASS"
```

Sources

- Iterating a string in a join builds one clause per character. (verified: Run in Python 3.11 in this repository.)
- The X | Y union syntax works in annotations from Python 3.10. (verified: Python 3.10 release notes (PEP 604); used throughout this repository on Python 3.11.)

A1-L3 Config over hardcoded values, and PEP 8 with ruff

Concept

Pipeline code should contain logic. Everything that changes between environments, trackers or contract versions belongs in configuration:

- catalog, schema and table names
- column maps and type maps
- thresholds, such as `weight_sum_min` or a stability tolerance
- contract rules and their severity

Keep configuration in one of two places. Versioned JSON or YAML files in the repository suit values that change with a code release. Registry tables in Unity Catalog suit values that analysts maintain, such as the Variable Registry column map.

Load configuration once, at the start of the run, into a typed object. A frozen dataclass or a pydantic model works. Validate on load: every required key present, `weight_sum_min` below `weight_sum_max`, names non-empty. Fail at 00:01 with "missing key: silver_table", not at 02:47 with a KeyError deep in a transform.

Then pass the object, or the values from it, into your functions. A function that reads a global config dictionary is as hard to test as one with hardcoded strings.

Style is part of the same discipline. PEP 8 is the Python style guide. **ruff** checks it and formats code in one fast tool:

- `ruff check .` finds errors and style problems, such as undefined names (F821) or `== True` (E712).
- `ruff format .` rewrites layout consistently. `ruff format --check .` fails CI when a file needs formatting.

Settle the rules once in `pyproject.toml` under `[tool.ruff]`, run both commands in CI, and stop debating layout in reviews.

Why it matters

A developer copies the production silver notebook to test a change in the dev workspace. The notebook hardcodes `research_prod.research_silver.nat_respondents`. The dev run MERGEs half-finished test data into the production table. The morning report shows impossible reach figures, and the team restores the table from Delta history.

With the catalog in a config file per environment, the dev run writes to `research_dev`. The same code runs in both places, and only the config changes.

Worked example: weak code

```
1 def check_weight_sum(spark, wave_id):
2     total = (
1         spark.table("research_prod.research_silver.nat_respondents")
4         .filter(F.col("wave_id") == wave_id)
5         .agg(F.sum("final_weight")).collect()[0][0]
6     )
2     if total < 45000000 or total > 55000000:
3         raise ValueError("weight sum out of range")
```

1. **Line 3.** The production table is typed into the function. A dev run writes, or here reads, production.
2. **Line 7.** Contract thresholds are buried in code. A new contract version needs a code change in every notebook that repeats them.
3. **Line 8.** The error hides the numbers. You cannot tell how far out of range the total is.

Worked example: strong code

```
1 import json
```

```
from dataclasses import dataclass
from pathlib import Path


@dataclass(frozen=True)
class WeightContract:
    """Weight total limits from the data contract."""

    silver_table: str
    weight_sum_min: float
    weight_sum_max: float

    def __post_init__(self) -> None:
        if self.weight_sum_min >= self.weight_sum_max:
            raise ValueError("weight_sum_min must be below weight_sum_max")


def load_contract(path: Path) -> WeightContract:
    """Load and validate the contract file. Fails fast on a missing key."""
    raw = json.loads(path.read_text())
    return WeightContract(
        silver_table=raw["silver_table"],
        weight_sum_min=float(raw["weight_sum_min"]),
        weight_sum_max=float(raw["weight_sum_max"]),
    )


def weight_sum_in_range(total: float, contract: WeightContract) -> bool:
    """True when the weight total is inside the contract limits."""
    return contract.weight_sum_min <= total <= contract.weight_sum_max
```

1. **Line 6.** Frozen: no code can change a limit halfway through a run.
2. **Line 14.** Validation on load. An impossible contract fails before any data moves.
3. **Line 23.** A missing key raises KeyError here, at load, with the key's name.
4. **Line 29.** Pure logic with the limits passed in. Test it with plain numbers, no Spark needed.

The strong version runs unchanged in dev and prod. The contract file decides the table and the limits.

Recap card

- **What belongs in configuration?** Anything that changes between environments, trackers or contract versions: names, maps, thresholds, rules.
- **When do you validate configuration?** Once, on load, before any data moves. Fail fast with the key's name.

- **Why not raw.get("catalog", "research_prod")?** A default hides a missing key and can point a run at production.
- **The two ruff commands for CI** ruff check . and ruff format --check .

Practice

Exercise 1. Multiple choice (level 1)

Which of these belongs in configuration rather than in pipeline code?

- a) The formula for weighted reach: weighted reach divided by weighted base
- b) The catalog name, `research_prod` or `research_dev`
- c) The order of steps: read, validate, transform, write
- d) The decision to MERGE rather than append in silver

Why?

- r1) It changes between environments while the logic stays the same.
- r2) Short strings belong in configuration.

Exercise 2. Predict the output (level 2)

What does this print?

What you need to know.

`@dataclass(frozen=True)` makes every instance read-only. Assigning to a field after the object exists raises `dataclasses.FrozenInstanceError`, and the field keeps its old value.

`try` runs a block. When the block raises an exception named in `except`, Python runs the `except` block instead of stopping, then carries on after it.

```python
from dataclasses import FrozenInstanceError, dataclass

@dataclass(frozen=True)
class Limit:
    value: int

limit = Limit(5)
try:
    limit.value = 9
except FrozenInstanceError:
    print("read-only")
```

```python
from dataclasses import FrozenInstanceError, dataclass


@dataclass(frozen=True)
class RunConfig:
```

```
    catalog: str


cfg = RunConfig("research_dev")
try:
    cfg.catalog = "research_prod"
except FrozenInstanceError:
    print("blocked")
print(cfg.catalog)
```

Exercise 3. Fill the gap (level 2)

Complete the two CI commands. The first finds lint errors. The second fails when a file needs reformatting, without changing it.

```
ruff _____(1) .
ruff _____(2) --check .
```

Exercise 4. Write the function (level 4)

Write `load_run_config(raw)`. It takes a dict parsed from JSON and returns a frozen `RunConfig` dataclass with `catalog` (str), `tracker` (str) and `stability_tolerance` (float).

- Raise `KeyError` naming the key when one is missing.
- Raise `ValueError` when `stability_tolerance` is not between 0 and 1.
- Add a property `silver_table` that returns `f"{catalog}.research_silver.{tracker}_respondents"`.

Your code must pass these checks:

- A valid dict must load into RunConfig with the same values.
- `silver_table` must build the three-level name from catalog and tracker.
- A missing key must raise KeyError that names the key.
- A tolerance outside 0 to 1 must raise ValueError. 10 means 1000 percentage points.
- The config must be frozen, so no code can change it mid-run.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Settings for one pipeline run.
class RunConfig: ...

# Build a validated RunConfig from parsed JSON.
def load_run_config(raw: dict[str, object]) -> RunConfig: ...
```

Starter code:

```
from dataclasses import dataclass

```

```python

@dataclass(frozen=True)
class RunConfig:
    ...


def load_run_config(raw: dict[str, object]) -> RunConfig:
    ...
```

Exercise 5. Spot the bug (level 5)

The team moved every table name into `RunConfig`. A dev run still reads production. Tap the line, then fix the function.

```python
def weighted_base_by_wave(spark: SparkSession, cfg: RunConfig) ->
    DataFrame:
    """Weighted base per wave from the configured silver table."""
    silver = spark.table(cfg.silver_table)
    scope = spark.table("research_prod.research_governance.scope_rules")
    in_scope = silver.join(scope, "region").filter(F.col("in_scope"))
    return
    in_scope.groupBy("wave_id").agg(F.sum("final_weight").alias("weighted_base"))
```

Write down the line number of the bug. Then write the fix.

Sources

- ruff check reports F821 (undefined name) and E712 (comparison to True); ruff format --check fails when a file needs formatting. (verified: ruff 0.16.9 run in this repository.)
- Assigning to a frozen dataclass field raises FrozenInstanceError. (verified: Python 3.11 standard library, run in this repository.)

A2 Medallion architecture

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Bronze, silver and gold contracts.

A2-L1 Bronze: keep every delivery exactly as received

Concept

Bronze is your evidence. Its contract has four rules.

1. **Append only.** Every delivery adds rows. Nothing in bronze is updated, deleted or overwritten. A redelivery of wave W14 appends again, with a new `ingestion_id`.
2. **As received.** No cleaning, filtering, renaming or type casting. Read CSV columns as strings. `inferSchema` guesses types, and a guess can destroy data: the ID `00123` becomes the integer `123`, and the leading zeros are gone for good.
3. **Audit columns.** Add four columns to every row: `ingestion_id` (one UUID per load), `source_file`, `ingestion_timestamp` and `delivery_version`. Later layers use them to pick the newest version and to trace any number back to a file.
4. **Grain.** One row per respondent per wave per delivery file. Duplicates across deliveries are expected here. Silver resolves them.

Why keep the mess? Because silver logic changes. When you fix a mapping bug in March, you rebuild silver from bronze for every wave since January. You can only do that if bronze still holds the original deliveries.

Two practical points for Databricks Runtime 17.3 LTS:

- Count what you wrote from the Delta history instead of calling `count()` before the write. `DeltaTable.forName(spark, table).history(1)` returns `operationMetrics["numOutputRows"]`. A separate `count()` reads the source file a second time.
- Open-source Spark defaults `saveAsTable` to Parquet. Databricks defaults to Delta. Write `.format("delta")` explicitly, so the code behaves the same in local tests.

Why it matters

A bronze job reads supplier CSVs with `inferSchema=true`. Respondent IDs such as `00123` land as the integer `123`. Months later, the supplier sends a correction file keyed on the original IDs, and 18% of rows fail to match silver. The original IDs are not recoverable from bronze, because bronze never held them. The team asks the supplier to resend six months of deliveries.

Reading every column as a string keeps `00123` intact. Silver can then cast deliberately, with checks.

Worked example: weak code

```
1 def ingest(path, table):
(1)     df = spark.read.option("header", True).option("inferSchema",
      True).csv(path)
(2)     df = df.filter(F.col("final_weight").isNotNull())
(3)     print(df.count())
(4)     df.write.mode("overwrite").saveAsTable(table)
```

1. **Line 2.** Type guessing. `00123` becomes `123`, and the original value is lost.
2. **Line 3.** Cleaning in bronze. The rows with missing weights vanish before anyone can count them.
3. **Line 4.** Reads the whole file a second time, only to print a number.
4. **Line 5.** Overwrite destroys every earlier delivery.

Worked example: strong code

```
 1 import uuid
 2
 3 from delta.tables import DeltaTable
 4 from pyspark.sql import DataFrame, SparkSession
 5 from pyspark.sql import functions as F
 6
 7
(1) def add_audit_columns(df: DataFrame, ingestion_id: str, source_file: str,
     delivery_version: int) -> DataFrame:
 9     """Add the four bronze audit columns."""
10     return df.withColumns({
11         "ingestion_id": F.lit(ingestion_id),
12         "source_file": F.lit(source_file),
(2)         "ingestion_timestamp": F.current_timestamp(),
14         "delivery_version": F.lit(delivery_version),
15     })
16
17
18 def ingest_to_bronze(spark: SparkSession, path: str, table: str,
     delivery_version: int) -> int:
19     """Append one delivery file to bronze as received. Returns rows
     written."""
```

```
❸     raw = spark.read.option("header", True).csv(path)
21    audited = add_audit_columns(raw, str(uuid.uuid4()), path,
      delivery_version)
❹     audited.write.format("delta").mode("append").saveAsTable(table)
❺     metrics = DeltaTable.forName(spark, table).history(1).collect()[0]
      ["operationMetrics"]
24    return int(metrics["numOutputRows"])
```

1. **Line 8.** A pure transform. Test it with two rows and no files.
2. **Line 13.** When the row landed, from the cluster clock.
3. **Line 20.** No inferSchema: every column arrives as a string, exactly as delivered.
4. **Line 22.** Append only, and Delta stated explicitly.
5. **Line 23.** The row count comes from the commit metrics. No second read of the file.

The strong version keeps every byte the supplier sent, labels every row with its origin, and reads the file once.

Recap card

- **The four rules of the bronze contract** Append only. As received. Audit columns. Grain: respondent per wave per delivery file.
- **The four bronze audit columns** ingestion_id, source_file, ingestion_timestamp, delivery_version.
- **Why no inferSchema in bronze?** It guesses types and can destroy values, such as the leading zeros in 00123.
- **Rows written, without a second read** DeltaTable.forName(spark, t).history(1) and operationMetrics["numOutputRows"].

Practice

Exercise 1. Multiple choice (level 1)

A delivery contains 12 rows with a null `final_weight`. What should the bronze job do with them?

- a) Drop them, so silver never sees bad rows.
- b) Append them as received, with the audit columns.
- c) Replace the null weights with 1.0.
- d) Reject the whole file.

Why?

- r1) Bronze records deliveries as received. Silver and the data contract decide what to do with bad rows.
- r2) Delta tables cannot hold null values.

Exercise 2. Predict the output (level 2)

`ids.csv` holds a header line `respondent_id` and one data line `00123`. What does this print on Spark 4.0?

```
df = spark.read.option("header", True).option("inferSchema", True).csv("ids.csv")
print(df.collect()[0]["respondent_id"])
```

Exercise 3. Fill the gap (level 2)

Complete the bronze write and the timestamp audit column.

What you need to know.

`df.withColumn(name, column)` returns a new DataFrame with one column added, or replaced when the name exists. It is the one-column form of `withColumns({...})` from the worked example.

```
dated = df.withColumn("loaded_on", F.current_date())
```

The writer's `.mode(...)` decides what happens to rows already in the table: `"append"` adds the new rows, `"overwrite"` replaces the old ones.

```
audited = raw.withColumn("ingestion_timestamp", F._____(1)())
audited.write.format("delta").mode("_____(2)").saveAsTable(table)
```

Exercise 4. Write the function (level 3)

Write `add_audit_columns(df, ingestion_id, source_file, delivery_version)`. It returns `df` with four extra columns: `ingestion_id`, `source_file`, `ingestion_timestamp` (current timestamp) and `delivery_version`. Do not change any existing column. Use type hints and a docstring.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Add the four bronze audit columns to a delivery.
def add_audit_columns(
    df: DataFrame,
    ingestion_id: str,
    source_file: str,
    delivery_version: int,
) -> DataFrame: ...
```

Starter code:

```
from pyspark.sql import DataFrame
from pyspark.sql import functions as F


def add_audit_columns(df: DataFrame, ingestion_id: str, source_file: str, delivery_version: int) -> DataFrame:
    ...
```

Exercise 5. Spot the bug (level 4)

Run this job twice for two deliveries of the same wave. Bronze ends with only the second delivery. Tap the line, then fix the function.

```
1 def ingest_to_bronze(spark: SparkSession, path: str, table: str,
    delivery_version: int) -> None:
2     """Append one delivery file to bronze as received."""
3     raw = spark.read.option("header", True).csv(path)
4     audited = add_audit_columns(raw, str(uuid.uuid4()), path,
    delivery_version)
5     audited.write.format("delta").mode("overwrite").saveAsTable(table)
```

Write down the line number of the bug. Then write the fix.

Exercise 6. Architecture design (level 5)

Assign each piece of work to the layer that owns it.

Options: Bronze, Silver, Gold.

1. Store the supplier's CSV with every column as a string
2. Rename wgt to final_weight and cast it to DOUBLE
3. Keep only the newest delivery version per respondent and wave
4. Weighted reach by wave, age group and region for Power BI

Practical: Build a bronze loader and prove it keeps every delivery

Platform: Databricks Free Edition. Run this in Databricks Free Edition. It builds two small deliveries of the same wave in memory, loads both into bronze, and shows them side by side. In production the rows come from `spark.read.option("header", True).csv(path)`.

Download the notebook `A2-L1.ipynb` from the app.

Sources

- inferSchema turns 00123 into the integer 123. (verified: Run on PySpark 4.0.4 in this repository.)
- Delta history operationMetrics holds numOutputRows for an append. (verified: Run on delta-spark 4.0.1 in this repository.)
- DataFrame.withColumns exists (PySpark 3.3 and later). (verified: Checked on PySpark 4.0.4 in this repository.)
- Open-source Spark defaults saveAsTable to Parquet; Databricks defaults to Delta. (verified: OSS behaviour run locally; Databricks default per docs (VERSIONS.md).)

A2-L2 Silver: canonical names, checked types, one row per respondent per wave

Concept

Silver turns deliveries into a dependable table. Its contract:

- **Canonical names.** Rename supplier columns to the names in the Variable Registry, loaded at run time.
- **Declared types.** Cast each column to the type in the registry.
- **Grain.** One row per respondent per wave. Keep the newest delivery version per key, then MERGE (lesson A3-L1).

Two Spark behaviours make the first two steps fail silently or loudly, in the wrong place.

Renames skip missing columns. `withColumnRenamed("wgt", "final_weight")` does nothing when `wgt` is absent, and so does `withColumnsRenamed`. No error, no warning. If the supplier renames `wgt` to `weight`, silver loses its weight column, and the failure surfaces hours later in gold. Check the source columns yourself before you rename, and raise one error that lists every missing column.

Casts under ANSI mode. Databricks Runtime 17.3 LTS runs Spark 4.0, which turns ANSI mode on by default. `cast("abc" as int)` now raises `CAST_INVALID_INPUT` and stops the job at the first bad value. Older runtimes returned NULL without a word. Neither is what you want in silver. Use `try_cast`, which returns NULL for a bad value, then count the new NULLs and let a data-quality check decide: under 0.1% is a WARN, above it is CRITICAL.

```
1 F.col("final_weight").try_cast("double")
```

The pattern is: check columns, rename, `try_cast`, count cast failures, deduplicate, MERGE. Each step is a small function you can test.

Why it matters

A supplier renames `wgt` to `weight` without notice. The silver job calls `withColumnRenamed("wgt", "final_weight")`, which silently does nothing. The type-casting loop skips `final_weight` because the column is missing. Silver writes the wave with no weight column, since schema evolution is on. At 03:00 the gold job fails with `final_weight` not found. The morning report shows nothing for the new wave.

A column check before the rename fails the silver job at 01:00 with one clear message: "missing source columns: wgt". The on-call engineer knows exactly what to ask the supplier.

Worked example: weak code

```
1  def transform_to_silver(bronze_df, column_map, type_map, wave_id):
2      for src, tgt in column_map.items():
1          bronze_df = bronze_df.withColumnRenamed(src, tgt)
4      for col_name, col_type in type_map.items():
2          bronze_df = bronze_df.withColumn(col_name,
       col(col_name).cast(col_type))
3      bronze_df = bronze_df.withColumn("wave_id", lit(wave_id))
7      return bronze_df
```

1. **Line 3.** A missing source column is skipped without an error.
2. **Line 5.** Under ANSI mode one bad value stops the job. Without ANSI, bad values become NULL with no count.
3. **Line 6.** Overwrites the wave_id from the delivery, hiding a file loaded against the wrong wave.

Worked example: strong code

```
1  def require_columns(df: DataFrame, columns: list[str]) -> None:
2      """Raise ValueError listing every column missing from df."""
3      missing = sorted(set(columns) - set(df.columns))
4      if missing:
1          raise ValueError(f"missing source columns: {missing}")
6
7
8  def rename_to_canonical(df: DataFrame, column_map: dict[str, str]) ->
     DataFrame:
9      """Rename supplier columns to registry names. Fails on any missing
       column."""
10     require_columns(df, list(column_map))
2      return df.withColumnsRenamed(column_map)
12
13
14 def cast_to_registry_types(df: DataFrame, type_map: dict[str, str]) ->
     DataFrame:
15
       """Cast with try_cast, so a bad value becomes NULL instead of stopping
       the job."""
16     require_columns(df, list(type_map))
3      return df.withColumns({c: F.col(c).try_cast(t) for c, t in
       type_map.items()})
18
19
20 def count_cast_failures(df: DataFrame, type_map: dict[str, str]) ->
     dict[str, int]:
```

```
21
    """Per column, rows whose non-null value fails try_cast. One pass over
    df."""
22      failures = [
4           F.count_if(F.col(c).isNotNull() &
    F.col(c).try_cast(t).isNull()).alias(c)
24          for c, t in type_map.items()
25      ]
26      return df.agg(*failures).collect()[0].asDict()
```

1. **Line 5.** One error that names every missing column.
2. **Line 11.** Safe to call now: every source column exists.
3. **Line 17.** Column.try_cast (PySpark 4.0) returns NULL for a bad value.
4. **Line 23.** Run on the uncast frame: a value that is present but fails the cast is a failure. All columns counted in one agg pass.

The strong version fails early and clearly on a changed delivery, and turns bad values into a number you can check instead of a crash or a silent NULL.

Recap card

- **What does withColumnRenamed do with a missing column?** Nothing, silently. Check the source columns yourself and raise one error listing every missing one.
- **cast on Spark 4.0 (Runtime 17.3 LTS) with a malformed value** Raises CAST_INVALID_INPUT, because ANSI mode is on by default.
- **Safe cast for silver** F.col(c).try_cast(t), then count the new NULLs and check them against a threshold.
- **Silver dedup key** The full grain: respondent_id and wave_id.

Practice

Exercise 1. Predict the output (level 1)

What does this print on PySpark 4.0?

What you need to know.

`spark.createDataFrame(rows, schema)` builds a small DataFrame from Python tuples, one tuple per row. The schema string names each column and gives its type. Tests and examples use it to make data without a table.

`df.columns` returns the column names as a Python list, in order.

```
1 df = spark.createDataFrame([("R1", 3)], "id STRING, n INT")
2 df.columns   # ['id', 'n']
```

```
1 df = spark.createDataFrame([(1, 2.5)], "respondent_id INT, weight DOUBLE")
2 print(df.withColumnRenamed("wgt", "final_weight").columns)
```

Exercise 2. Multiple choice (level 2)

On Databricks Runtime 17.3 LTS, silver runs `F.col("final_weight").cast("double")`. One row holds `"n/a"`. What happens?

- a) The row gets NULL in final_weight, and the job succeeds.
- b) The job fails with CAST_INVALID_INPUT.
- c) The row is dropped.
- d) The value becomes 0.0.

Why?

- r1) Spark 4.0 enables ANSI mode by default, and ANSI casts raise on malformed input.
- r2) Delta rejects NULL values in DOUBLE columns.

Exercise 3. Fill the gap (level 2)

Cast every column in `type_map` so that a bad value becomes NULL instead of stopping the job.

```
df = df.withColumns({c: F.col(c)._____(1)(t) for c, t in type_map.items()})
```

Exercise 4. Write the function (level 3)

Write `rename_to_canonical(df, column_map)`. It raises `ValueError` naming every missing source column, then renames all columns in one call. Use type hints and a docstring.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Rename supplier columns to registry names.
def rename_to_canonical(
    df: DataFrame,
    column_map: dict[str, str],
) -> DataFrame: ...
```

Starter code:

```
from pyspark.sql import DataFrame


def rename_to_canonical(df: DataFrame, column_map: dict[str, str]) -> DataFrame:
    ...
```

Exercise 5. Spot the bug (level 4)

Silver's grain is one row per respondent per wave. After loading waves W1 and W2, silver holds only one row per respondent. Tap the line, then fix the function.

```
1 def prepare_silver(df: DataFrame, column_map: dict[str, str], type_map:
    dict[str, str]) -> DataFrame:
2     """Rename, cast and keep the newest version per respondent and
    wave."""
3     renamed = rename_to_canonical(df, column_map)
4     typed = cast_to_registry_types(renamed, type_map)
5     return latest_per_key(typed, ["respondent_id"], "delivery_version")
```

Write down the line number of the bug. Then write the fix.

Exercise 6. Put in order (level 5)

Put the silver steps in order.

- Count cast failures and apply the WARN or CRITICAL threshold
- Rename to canonical names
- Check that every mapped source column exists
- MERGE into silver
- Keep the newest version per respondent and wave
- try_cast to registry types

Sources

- withColumnRenamed and withColumnsRenamed ignore missing columns without an error. (verified: Run on PySpark 4.0.4 in this repository.)
- Spark 4.0 enables ANSI mode by default; cast raises CAST_INVALID_INPUT on malformed input; try_cast returns NULL. (verified: Run on PySpark 4.0.4 in this repository. Databricks Runtime 17.3 LTS uses Spark 4.0.0 (search-only, VERSIONS.md).)
- Column.try_cast exists in PySpark 4.0. (verified: Run on PySpark 4.0.4 in this repository.)

A2-L3 Gold: state the grain, group by all of it, guard every ratio

Concept

Gold tables feed Power BI and clients. The contract has three parts.

1. State the grain. Write it in the function docstring and on the table. "One row per tracker x wave x age group x gender x region." Store it as a table comment or a custom table property:

```
COMMENT ON TABLE research_gold.nat_metrics IS 'Grain: tracker x wave x
    age_group x gender x region';
ALTER TABLE research_gold.nat_metrics SET TBLPROPERTIES ('grain' =
    'tracker x wave x age_group x gender x region');
```

Avoid the property name `owner`. It is reserved, and Spark 4.0 rejects it with `UNSUPPORTED_FEATURE.SET_TABLE_PROPERTY`. Ownership is set with `ALTER TABLE ... OWNER TO`.

2. Group by every grain column. If the grain names `tracker_id`, the `groupBy` must include `tracker_id`. Miss one and two trackers' respondents fall into the same row. The numbers look plausible and are wrong.

3. Make aggregates safe.

- A `sum` over no matching rows returns NULL, not 0. `F.sum(F.when(F.col("reach_flag") == 1, 1))` gives NULL for a subgroup with no reach. Wrap it: `F.coalesce(F.sum(...), F.lit(0))`.
- Guard every ratio. `F.when(F.col("weighted_base") > 0, reach / base)` returns NULL for an empty base instead of an error or an infinite value.
- Keep the unweighted count next to every weighted figure. Analysts need it to judge whether a subgroup is too small to report.

Gold is also where you check totals against silver. The sum of `weighted_base` across all subgroups for a wave must equal the silver weight total for in-scope respondents. Lesson A6-L3 builds that check.

Why it matters

The reference `compute_gold_reach_metrics` promises "tracker x wave x subgroup grain" but groups by wave and subgroup only. When a second tracker starts writing to the same silver table, its respondents fold into the first tracker's rows. Weighted bases roughly double, and reach percentages blend two audiences. The dashboard shows smooth, believable figures for three weeks, until a client compares a subgroup against its published sample size.

Adding `tracker_id` to `groupBy`, and a test that checks the output has one row per grain key, prevents it.

Worked example: weak code

```
from pyspark.sql.functions import col, count, sum, when
```

```
2
3  def compute_gold_reach_metrics(silver_table, wave_id,
       weight_col="final_weight"):
4      """Compute reach metrics at tracker x wave x subgroup grain."""
5      silver = spark.table(silver_table).filter(col("wave_id") == wave_id)
6      return (
7          silver.filter(col("in_scope") == True)
2          .groupBy("wave_id", "age_group", "gender", "region")
9          .agg(
10             count("*").alias("unweighted_n"),
11             sum(weight_col).alias("weighted_base"),
3              sum(when(col("reach_flag") == 1,
       col(weight_col))).alias("weighted_reach"),
13         )
4          .withColumn("weighted_reach_pct", col("weighted_reach") /
       col("weighted_base"))
15     )
```

1. **Line 1.** Imports `sum` by name, shadowing the Python built-in (reference defect 3).
2. **Line 8.** The docstring promises tracker grain, but tracker_id is missing.
3. **Line 12.** NULL, not 0, for a subgroup where nobody is reached.
4. **Line 14.** No guard on the base, and NULL reach gives a NULL percentage.

Worked example: strong code

```
1  from pyspark.sql import DataFrame
2  from pyspark.sql import functions as F
3
1  GRAIN = ["tracker_id", "wave_id", "age_group", "gender", "region"]
5
6
7  def reach_metrics(df: DataFrame, weight_col: str = "final_weight") ->
       DataFrame:
8      """Weighted reach at tracker x wave x age_group x gender x region
       grain.
9
10     Args:
11         df: In-scope silver respondents with reach_flag and weights.
12         weight_col: Weight column to aggregate.
13
14     Returns:
15         One row per GRAIN key with unweighted_n, weighted_base,
       weighted_reach
16         and weighted_reach_pct (NULL when the base is 0).
17     """
18     reached = F.when(F.col("reach_flag") == 1, F.col(weight_col))
2      metrics = df.groupBy(*GRAIN).agg(
20         F.count("*").alias("unweighted_n"),
21         F.coalesce(F.sum(weight_col), F.lit(0.0)).alias("weighted_base"),
3          F.coalesce(F.sum(reached), F.lit(0.0)).alias("weighted_reach"),
23     )
```

```
24      return metrics.withColumn(
25          "weighted_reach_pct",
4           F.when(F.col("weighted_base") > 0, F.col("weighted_reach") /
    F.col("weighted_base")),
27      )
```

1. **Line 4.** The grain is one constant, used by the groupBy and by the tests.
2. **Line 19.** Groups by every grain column, tracker included.
3. **Line 22.** Zero, not NULL, when a subgroup has no reach.
4. **Line 26.** NULL for an empty base, never an error or infinity.

The strong version cannot merge two trackers, reports a true zero for zero reach, and never divides by nothing.

Recap card

- **Where do you record a gold table's grain?** In the function docstring and on the table: COMMENT ON TABLE or a custom property such as 'grain'.
- **sum over no matching rows** Returns NULL, not 0. Wrap it: F.coalesce(F.sum(...), F.lit(0)).
- **Safe ratio in PySpark** F.when(base > 0, part / base). NULL when the base is 0.
- **Why is SET TBLPROPERTIES ('owner' = ...) wrong?** owner is reserved. Spark 4.0 raises UNSUPPORTED_FEATURE.SET_TABLE_PROPERTY. Use ALTER TABLE ... OWNER TO.

Practice

Exercise 1. Spot the bug (level 1)

The docstring states the grain. The code does not match it. Tap the line, then fix the function.

```
1  def base_by_subgroup(df: DataFrame) -> DataFrame:
2      """Weighted base at tracker x wave x region grain."""
3      return df.groupBy("wave_id",
       "region").agg(F.sum("final_weight").alias("weighted_base"))
```

Write down the line number of the bug. Then write the fix.

Exercise 2. Predict the output (level 2)

Region `S` has one respondent, and nobody in `S` is reached. What does this print?

```
1  df = spark.createDataFrame([("S", 0)], "region STRING, reach_flag INT")
```

```
out = df.groupBy("region").agg(F.sum(F.when(F.col("reach_flag") == 1,
    1)).alias("reached"))
print(out.collect()[0]["reached"])
```

Exercise 3. Fill the gap (level 3)

Make reach a true zero when nobody is reached, and the percentage NULL when the base is zero.

```
F._____(1)(F.sum(reached), F.lit(0.0)).alias("weighted_reach")
...
F._____(2)(F.col("weighted_base") > 0, F.col("weighted_reach") /
    F.col("weighted_base"))
```

Exercise 4. Write the function (level 4)

Write `reach_metrics(df, grain)`. It groups by every column in `grain` and returns `unweighted_n`, `weighted_base` (sum of `final_weight`, 0 when empty), `weighted_reach` (sum of `final_weight` where `reach_flag == 1`, 0 when none) and `weighted_reach_pct` (NULL when the base is 0).

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Weighted reach, one row per grain key.
def reach_metrics(
    df: DataFrame,
    grain: list[str],
) -> DataFrame: ...
```

Starter code:

```
from pyspark.sql import DataFrame
from pyspark.sql import functions as F


def reach_metrics(df: DataFrame, grain: list[str]) -> DataFrame:
    ...
```

Exercise 5. Multiple choice (level 5)

You want the grain of `research_gold.nat_metrics` recorded on the table itself. Which statement fails on Spark 4.0?

- a) `COMMENT ON TABLE research_gold.nat_metrics IS 'Grain: tracker x wave x region'`
- b) `ALTER TABLE research_gold.nat_metrics SET TBLPROPERTIES ('grain' = 'tracker x wave x region')`
- c) `ALTER TABLE research_gold.nat_metrics SET TBLPROPERTIES ('owner' = 'data.engineering@example.org')`

- d) `ALTER TABLE research_gold.nat_metrics SET TBLPROPERTIES ('layer' = 'gold')`

Why?

- r1) `owner` is a reserved table property, so Spark rejects setting it this way.
- r2) Table properties cannot hold email addresses.

Sources

- sum(when(cond, 1)) over rows where cond is false returns NULL. (verified: Run on PySpark 4.0.4 in this repository.)
- 'owner' is a reserved table property; Spark 4.0 raises UNSUPPORTED_FEATURE.SET_TABLE_PROPERTY. (verified: Run on PySpark 4.0.4 with delta-spark 4.0.1 in this repository.)
- COMMENT ON TABLE and custom TBLPROPERTIES work on Delta tables. (verified: Run on delta-spark 4.0.1 in this repository.)

A3 Delta Lake

Roadmap: Months 16 to 18: MLOps and deployment; Current work. MERGE, idempotent writes, schema enforcement and evolution, time travel, maintenance.

A3-L1 Idempotent MERGE, and why you deduplicate the source first

Concept

A pipeline is **idempotent** when running it twice with the same input leaves the table in the same state as running it once. Retries, reruns and redeliveries happen every week in production, so idempotence is not optional.

An append is not idempotent. Run `write.mode("append")` twice and every row lands twice. A Delta `MERGE` is idempotent when two conditions hold:

1. The `ON` condition covers every column of the table's grain. For a silver table with one row per respondent per wave, that means `respondent_id` **and** `wave_id`.
2. The source holds at most one row per key.

The second condition catches people out. When two source rows match the same target row, Delta cannot tell which one should win. It stops the `MERGE` with the error `DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE`. When neither row matches a target row, both rows are inserted, and your grain is broken without any error.

So deduplicate the source before you merge. Rank the rows for each key by a version column, such as `delivery_version` or `ingestion_timestamp`, and keep the newest. Use a window with `row_number()`. Do not use `dropDuplicates()`: it keeps an arbitrary row per key, so a rerun keeps a different version with no warning.

The pattern for every silver load:

- Keep the newest source row per key.
- `MERGE` on the full grain.
- `whenMatchedUpdateAll()` for redelivered rows, `whenNotMatchedInsertAll()` for new ones.

Bronze stays append-only, because bronze records every delivery as received. Silver is where you resolve versions.

Why it matters

A supplier redelivers a wave with corrected weights. The file repeats 37 respondents, because the supplier's export joined two tables. The silver job uses the reference `upsert_silver`, which merges without deduplicating. The `MERGE` fails at 02:00 with `DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE`. Gold does not refresh, and the morning Power BI report shows last week's numbers.

The on-call engineer "fixes" it by switching the job to append. The next run succeeds. Silver now holds the wave twice. Reach percentages look normal, because both the numerator and the denominator doubled. Weighted population estimates are twice their true size for a month, until a reconciliation check against the weight total catches it.

Deduplicating on the full grain before the `MERGE` prevents both failures.

Worked example: weak code

```
1  from delta.tables import DeltaTable
2  from pyspark.sql import DataFrame, SparkSession
3
1  spark = SparkSession.builder.getOrCreate()
5
6
2  def upsert_silver(new_df: DataFrame, target_table: str, merge_keys: list)
       -> None:
8      """Merge new data into silver table."""
9      delta_table = DeltaTable.forName(spark, target_table)
10     merge_condition = " AND ".join(
11         [f"target.{k} = source.{k}" for k in merge_keys]
12     )
13     (
14         delta_table.alias("target")
3          .merge(new_df.alias("source"), merge_condition)
16         .whenMatchedUpdateAll()
17         .whenNotMatchedInsertAll()
18         .execute()
19     )
```

1. **Line 4.** A session created at import time. Tests cannot inject a local session, and importing the module has a side effect.
2. **Line 7.** `list` with no element type. mypy cannot check that callers pass column names.

3. **Line 15.** The source goes straight into the MERGE. One repeated key in a redelivery fails the whole job.

Worked example: strong code

```python
from delta.tables import DeltaTable
from pyspark.sql import DataFrame, SparkSession, Window
from pyspark.sql import functions as F


def latest_per_key(df: DataFrame, keys: list[str], order_col: str) -> DataFrame:
    """Keep the newest row for each key.

    Args:
        df: Source rows, possibly with repeated keys.
        keys: Columns that identify one target row.
        order_col: Column that ranks versions. Highest wins.

    Returns:
        One row per key.
    """
    window = Window.partitionBy(*keys).orderBy(F.col(order_col).desc())
    return (
        df.withColumn("_rank", F.row_number().over(window))
        .filter(F.col("_rank") == 1)
        .drop("_rank")
    )


def upsert_silver(
    spark: SparkSession,
    source: DataFrame,
    target_table: str,
    keys: list[str],
    order_col: str,
) -> None:
    """MERGE the newest source row per key into a silver table.

    Running it twice with the same source leaves the table unchanged.
    """
    condition = " AND ".join(f"t.{k} = s.{k}" for k in keys)
    (
        DeltaTable.forName(spark, target_table).alias("t")
        .merge(latest_per_key(source, keys, order_col).alias("s"), condition)
        .whenMatchedUpdateAll()
        .whenNotMatchedInsertAll()
        .execute()
```

```
43      )
```

1. **Line 3.** Import the module as `F`. Importing `sum` or `max` by name shadows the Python built-ins.
2. **Line 17.** One partition per key, newest version first. The result is the same on every run.
3. **Line 19.** `row_number()` gives exactly one row rank 1 per key, even when versions tie. `rank()` would keep ties.
4. **Line 26.** The session is a parameter. A test passes a local session, a job passes the cluster session.
5. **Line 39.** Deduplicate inside the MERGE call, so no caller can skip it.

The strong version is idempotent for any source: repeated keys, a retry or a full redelivery. The weak version works only while the supplier never repeats a row.

Recap card

- **When is a pipeline idempotent?** When running it twice with the same input leaves the table as one run would.
- **Two conditions for an idempotent Delta MERGE** The ON condition covers the full grain, and the source has at most one row per key.
- **What happens when two source rows match one target row?** The MERGE fails with DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE.
- **Why not dropDuplicates() before a MERGE?** It keeps an arbitrary row per key. Use row_number() over a window ordered by version, newest first.

Practice

Exercise 1. Multiple choice (level 1)

The silver table holds one row for respondent `R1` in wave `W1`. A redelivery holds **two** rows for `R1` in `W1`. You run a `MERGE` on `respondent_id` and `wave_id` with `whenMatchedUpdateAll()` and `whenNotMatchedInsertAll()`. What happens?

- a) The target row takes the values of the last source row.
- b) The MERGE fails with an error.
- c) Both source rows are inserted as new rows.
- d) The target row takes the values of the first source row.

Why?

- r1) Two source rows match one target row, and Delta cannot tell which should win.
- r2) Delta enforces a primary key on the target table.
- r3) The source DataFrame must be cached before a MERGE.

Exercise 2. Fill the gap (level 2)

Complete the chain so that redelivered rows update, new rows insert, and the operation runs.

```
(
    DeltaTable.forName(spark, "research_silver.nat_respondents").alias("t")
    .merge(source.alias("s"), "t.respondent_id = s.respondent_id AND t.wave_id = s.wave_id")
    ._____(1)()
    ._____(2)()
    ._____(3)()
)
```

Exercise 3. Predict the output (level 2)

`silver` starts empty. `source` holds 3 rows with 3 distinct `(respondent_id, wave_id)` keys. `load()` calls the strong `upsert_silver` from the worked example. What does the last line print?

```
load(source)  # MERGE on respondent_id and wave_id
load(source)  # the scheduler retries the same run
print(spark.table("silver").count())
```

Exercise 4. Spot the bug (level 3)

Silver has one row per respondent per wave. After wave `W2` loads, every wave `W1` row has disappeared. Tap the line with the bug, then fix the code.

```
def upsert_respondents(spark: SparkSession, source: DataFrame) -> None:
    """MERGE one wave of respondents into silver. Grain: respondent per wave."""
    latest = latest_per_key(source, ["respondent_id", "wave_id"], "delivery_version")
    (
        DeltaTable.forName(spark, "research_silver.nat_respondents").alias("t")
        .merge(latest.alias("s"), "t.respondent_id = s.respondent_id")
        .whenMatchedUpdateAll()
        .whenNotMatchedInsertAll()
        .execute()
    )
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Write the function (level 4)

Write `latest_per_key(df, keys, order_col)`. It returns one row per key: the row with the highest `order_col`. Use type hints and a docstring. The checker reads your code without running Spark.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Keep the row with the highest order_col for each key.
def latest_per_key(
    df: DataFrame,
    keys: list[str],
    order_col: str,
) -> DataFrame: ...
```

Starter code:

```
from pyspark.sql import DataFrame, Window
from pyspark.sql import functions as F


def latest_per_key(df: DataFrame, keys: list[str], order_col: str) ->
   DataFrame:
    ...
```

Exercise 6. Put in order (level 5)

Put the steps of an idempotent silver load in order.

- Write one record to the pipeline run log
- Rename and cast columns from the Variable Registry
- Validate the bronze delivery against its data contract
- MERGE into silver on respondent_id and wave_id
- Run the table health check on silver
- Keep the newest row per respondent and wave

Practical: Prove a MERGE is idempotent

Platform: Databricks Free Edition. Run this in Databricks Free Edition on serverless compute. It creates a schema called `learning_a3` in your current catalog, loads the same data twice, and shows that the row count does not change. It then repeats a key and shows the error.

Download the notebook `A3-L1.ipynb` from the app.

Sources

- MERGE fails with DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE

when several source rows match one target row. (verified: practice/tests/test_a3_l1_merge.py reproduces the error on delta-spark 4.0.1. Docs: https://docs.databricks.com/aws/en/sql/language-manual/delta-merge-into)

- DeltaTable.forName, alias, merge, whenMatchedUpdateAll, whenNotMatchedInsertAll, whenNotMatchedBySourceDelete and execute exist in delta-spark 4.0. (verified: practice/tests/test_a3_l1_merge.py checks the methods on delta-spark 4.0.1.)
- Free Edition users can create schemas in the workspace catalog, whose name matches the workspace. (search-only: https://docs.databricks.com/aws/en/catalogs/default)

A3-L2 Schema enforcement is on by default. Evolve on purpose.

Concept

A Delta table has a schema, and writes must match it. This is **schema enforcement**, and it is on by default. Tested on delta-spark 4.0.1:

Write	Result
Append with an extra column	Fails: schema mismatch
Append with a column missing	Succeeds: the missing column is NULL
Append with a clashing type (INT into DOUBLE)	Fails: `DELTA_FAILED_TO_MERGE_FIELDS`
Overwrite with a different schema	Fails unless `overwriteSchema` is true
MERGE with `updateAll`/`insertAll` and an extra source column	Succeeds, and **silently drops the extra column**

Enforcement is your friend. A failing write is a cheap, early signal that a delivery changed.

Schema evolution changes the table's schema to accept the new shape. Delta offers it per write:

- Append: `.option("mergeSchema", "true")` adds new columns.
- MERGE: `.withSchemaEvolution()` on the merge builder adds new source columns.
- Planned change: `ALTER TABLE ... ADD COLUMN (region STRING)` in a reviewed migration.

Evolve on purpose, never by default. Two habits cause trouble:

- Leaving `mergeSchema` on every silver write. A supplier typo such as `final_wieght` becomes a new, permanent column, and the real weight is NULL for that wave.
- Setting `spark.databricks.delta.schema.autoMerge.enabled` for the session, which Databricks now labels legacy. It turns evolution on for every MERGE in the job, including ones you did not think about.

The safe pattern: validate the incoming columns against the registry first (lesson A2-L2). If a column is new and expected, add it with `ALTER TABLE` or evolve that one write. If it is unexpected, fail the run and ask the supplier.

Note the silent MERGE case in the table. Enforcement does not protect you there. Your column check does.

Why it matters

A silver job appends with `mergeSchema` always on, "to avoid schema errors". A supplier's export misspells `final_weight` as `final_wieght` for one wave. The append succeeds. Silver gains a permanent `final_wieght` column, and `final_weight` is NULL for 2,000 respondents. Gold sums NULL weights to a smaller base, and weighted reach for that wave is computed on the wrong population.

Without `mergeSchema`, the append fails with a schema mismatch naming the unexpected column. The fix takes minutes, and silver never holds the bad wave.

Worked example: weak code

```
1 spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "true")
2
3
4 def append_wave(df, table):
2     df.write.format("delta").mode("append").option("mergeSchema",
  "true").saveAsTable(table)
```

1. **Line 1.** Turns evolution on for every MERGE in the session, not just the one you meant.
2. **Line 5.** Every append may now add columns. A typo in a delivery becomes part of the table.

Worked example: strong code

```
1 def check_columns_match(df: DataFrame, expected: list[str]) -> None:
2     """Fail when the delivery has unexpected or missing canonical
  columns."""
```

```
    unexpected = sorted(set(df.columns) - set(expected))
    missing = sorted(set(expected) - set(df.columns))
    if unexpected or missing:
        raise ValueError(f"schema drift: unexpected={unexpected} missing={missing}")


def append_wave(df: DataFrame, table: str, expected: list[str]) -> None:
    """Append after an explicit column check. Delta enforcement stays on."""
    check_columns_match(df, expected)
    df.write.format("delta").mode("append").saveAsTable(table)


def add_planned_column(spark: SparkSession, table: str, column: str, sql_type: str) -> None:
    """Planned schema change, run from a reviewed migration."""
    spark.sql(f"ALTER TABLE {table} ADD COLUMN ({column} {sql_type})")
```

1. **Line 6.** Both directions checked. A missing column would otherwise land as NULL without an error.
2. **Line 12.** No mergeSchema. An unexpected shape fails loudly.
3. **Line 17.** New columns arrive through a reviewed change, not as a side effect of a write.

Schema changes become decisions a person reviews, not accidents a delivery causes.

Recap card

- **Delta append with an extra column, no options** Fails with a schema mismatch. Enforcement is on by default.
- **MERGE with updateAll and an extra source column, no evolution** Succeeds and silently ignores the extra column. Only your own column check catches it.
- **Evolve one write on purpose** Append: option mergeSchema. MERGE: .withSchemaEvolution(). Planned: ALTER TABLE ADD COLUMN.
- **Why not set autoMerge.enabled for the session?** It turns evolution on for every MERGE in the job, including ones you did not intend.

Practice

Exercise 1. Multiple choice (level 1)

Silver has columns `respondent_id`, `wave_id`, `final_weight`. You append a DataFrame with those three columns plus `region`, without any options. What happens on delta-spark 4.0?

- a) The append succeeds and region is added.
- b) The append succeeds and region is dropped.
- c) The append fails with a schema mismatch.
- d) Region is stored in a hidden rescue column.

Why?

- r1) Schema enforcement rejects writes with columns the table does not have.
- r2) Delta tables cannot change schema after creation.

Exercise 2. Predict the output (level 2)

Silver `s` has columns `id` and `w`. The source has `id`, `w` and `region`. What does the last line print on delta-spark 4.0?

```
(
    DeltaTable.forName(spark, "s").alias("t")
    .merge(source.alias("src"), "t.id = src.id")
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute()
)
print(spark.table("s").columns)
```

Exercise 3. Fill the gap (level 3)

You reviewed the supplier's new `region` column and want this one MERGE to add it to silver. Complete the builder.

```
(
    DeltaTable.forName(spark, table).alias("t")
    .merge(source.alias("s"), condition)
    ._____(1)()
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute()
)
```

Exercise 4. Write the function (level 4)

Write `schema_drift(actual, expected)`. It takes two lists of column names and returns a dict with sorted lists under `"unexpected"` (in actual, not expected) and `"missing"` (in expected, not actual). Then write `assert_no_drift(actual, expected)` that raises `ValueError` naming both lists when either is non-empty.

Your code must pass these checks:

- Same columns in a different order is no drift.
- Report both directions: the typo is unexpected, and the real column is missing.
- Return sorted lists, so messages and tests are stable.
- `assert_no_drift` must raise ValueError that names the drifted columns.
- No drift must return None without raising.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Compare delivered columns with the registry.
def schema_drift(
    actual: list[str],
    expected: list[str],
) -> dict[str, list[str]]: ...

# Raise ValueError when the delivery's columns differ from the registry.
def assert_no_drift(
    actual: list[str],
    expected: list[str],
) -> None: ...
```

Starter code:

```python
def schema_drift(actual: list[str], expected: list[str]) -> dict[str, list[str]]:
    ...


def assert_no_drift(actual: list[str], expected: list[str]) -> None:
    ...
```

Exercise 5. Spot the bug (level 5)

This silver job evolved the schema on every run for a month, and silver now has 14 columns nobody asked for. Tap the line that causes it, then fix the function.

```python
def upsert_silver(spark: SparkSession, source: DataFrame, table: str, keys: list[str]) -> None:
    """MERGE a deduplicated source into silver."""
```

```
    spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "true")
    condition = " AND ".join(f"t.{k} = s.{k}" for k in keys)
    (
        DeltaTable.forName(spark, table).alias("t")
        .merge(source.alias("s"), condition)
        .whenMatchedUpdateAll()
        .whenNotMatchedInsertAll()
        .execute()
    )
```

Write down the line number of the bug. Then write the fix.

Sources

- Append with an extra column fails; with a missing column fills NULL; with a type clash fails with DELTA_FAILED_TO_MERGE_FIELDS; overwrite with a new schema needs overwriteSchema. (verified: Run on delta-spark 4.0.1 in this repository.)
- MERGE with updateAll/insertAll ignores extra source columns without evolution; DeltaMergeBuilder.withSchemaEvolution() adds them. (verified: Run on delta-spark 4.0.1 in this repository.)
- spark.databricks.delta.schema.autoMerge.enabled turns on schema evolution for MERGE across the session. (verified: https://docs.databricks.com/aws/en/data-engineering/schema-evolution (checked 2026-09-25))

A3-L3 Time travel, RESTORE, OPTIMIZE and VACUUM

Concept

Every Delta write creates a new table **version**. The transaction log keeps the list, and you can read any version still on storage.

- `DESCRIBE HISTORY t` lists versions with the operation, user and metrics.
- `spark.read.option("versionAsOf", 12).table("t")` or `SELECT * FROM t VERSION AS OF 12` reads an old version. `timestampAsOf` and `TIMESTAMP AS OF` work by time.
- `RESTORE TABLE t TO VERSION AS OF 12` makes version 12 current again, as a new version. History is kept, so you can undo the restore.

Time travel is your first response to a bad write: compare the versions, restore, then fix the job.

Two maintenance commands keep tables fast and storage bounded.

OPTIMIZE compacts many small files into fewer large ones. Frequent small appends leave thousands of tiny files, and every query pays to open them. `OPTIMIZE t ZORDER BY (wave_id)` also co-locates rows by a column. On Unity Catalog managed tables, Databricks recommends liquid clustering (`CLUSTER BY`, or `CLUSTER BY AUTO`) with predictive optimization, which runs OPTIMIZE and VACUUM for you.

VACUUM deletes data files that no current version needs and that are older than the retention window. The window comes from `delta.deletedFileRetentionDuration`, which defaults to 7 days. Delta blocks `VACUUM t RETAIN 0 HOURS` with a safety check, because a running reader or writer might still need those files.

VACUUM limits time travel. After it runs, versions whose files it removed can no longer be read. The log itself keeps entries for `delta.logRetentionDuration`, 30 days by default. From Databricks Runtime 18.0, time travel beyond `deletedFileRetentionDuration` is blocked outright. If you need 30 days of recoverable history, set both properties to at least 30 days.

Why it matters

A developer runs a gold rebuild against the wrong wave filter and overwrites three weeks of metrics. The team wants to restore, but a clean-up job ran `VACUUM ... RETAIN 0 HOURS` nightly, with the retention check switched off "to save storage". The overwritten files are gone. Three weeks of gold are rebuilt from silver overnight, and two client extracts go out late.

With the default 7-day retention, `RESTORE TABLE research_gold.nat_metrics TO VERSION AS OF` the last good version takes seconds.

Worked example: weak code

```
❶ SET spark.databricks.delta.retentionDurationCheck.enabled = false;
❷ VACUUM research_gold.nat_metrics RETAIN 0 HOURS;
  3 -- gold was overwritten by mistake last night; rebuild everything
❸ DROP TABLE research_gold.nat_metrics;
```

1. **Line 1.** Switches off the safety check that exists to stop the next line.
2. **Line 2.** Deletes every unreferenced file now. Time travel and RESTORE no longer work.
3. **Line 4.** With history intact, a RESTORE takes seconds. Dropping and rebuilding takes hours.

Worked example: strong code

```
1  -- 1. Find the last good version.
1  DESCRIBE HISTORY research_gold.nat_metrics;
3
4  -- 2. Compare before you act.
5  SELECT wave_id, SUM(weighted_base) AS base
2  FROM research_gold.nat_metrics VERSION AS OF 41
7  GROUP BY wave_id;
8
9  -- 3. Restore. This writes a new version, so it can be undone.
3  RESTORE TABLE research_gold.nat_metrics TO VERSION AS OF 41;
11
12 -- 4. Keep 30 days recoverable.
13 ALTER TABLE research_gold.nat_metrics SET TBLPROPERTIES (
4    'delta.deletedFileRetentionDuration' = 'interval 30 days',
15   'delta.logRetentionDuration' = 'interval 30 days'
16 );
```

1. **Line 2.** Each version with its operation, user and metrics such as numOutputRows.
2. **Line 6.** Read the old version side by side with the current one before restoring.
3. **Line 10.** Restore adds a version. The bad write stays in history for the post-mortem.
4. **Line 14.** VACUUM keeps unreferenced files for 30 days, so 30 days of versions stay readable.

History turns a data incident into a two-minute restore. Keep it long enough to be useful.

Recap card

- **Read an old Delta version** VERSION AS OF n or TIMESTAMP AS OF ts in SQL; option versionAsOf in PySpark.
- **What does RESTORE do to history?** Adds a new version with the old content. Nothing is deleted.
- **Default retention windows** Deleted files 7 days (delta.deletedFileRetentionDuration). Log 30 days (delta.logRetentionDuration).
- **Why is VACUUM ... RETAIN 0 HOURS dangerous?** It deletes the files that time travel and RESTORE need, and it breaks readers still using them.
- **Databricks' recommendation for layout and maintenance on UC managed tables** Liquid clustering (CLUSTER BY or CLUSTER BY AUTO) with predictive optimization.

Practice

Exercise 1. Multiple choice (level 1)

A job overwrote gold with bad data one hour ago. VACUUM has not run since. What is the fastest safe recovery?

- a) Drop the table and rebuild it from silver.
- b) `RESTORE TABLE` to the version before the bad write.
- c) Run `VACUUM` to remove the bad files.
- d) Run `OPTIMIZE` to rewrite the table.

Why?

- r1) The earlier version's files are still on storage, and RESTORE makes that version current again.
- r2) RESTORE deletes the bad version from history.

Exercise 2. Fill the gap (level 2)

Read version 41 of the gold table, then make it current.

```
SELECT * FROM research_gold.nat_metrics _____(1) 41;
_____(2) TABLE research_gold.nat_metrics TO VERSION AS OF 41;
```

Exercise 3. Predict the output (level 3)

A new Delta table gets these statements in order. What version number does the RESTORE create?

What you need to know.

`CREATE TABLE t (id INT) USING DELTA` creates a table stored in the Delta format, so it has a transaction log and versions. On Databricks, Delta is the default, and `USING DELTA` states it explicitly.

Each statement that changes the table commits one new version: the `CREATE` is version 0, and every `INSERT`, `UPDATE`, `DELETE` or `MERGE` after it adds one.

```
CREATE TABLE t (id INT) USING DELTA;          -- version 0
INSERT INTO t VALUES (1);                     -- version 1
INSERT INTO t VALUES (2);                     -- version 2
DELETE FROM t WHERE id = 1;                   -- version 3
RESTORE TABLE t TO VERSION AS OF 1;           -- version ?
```

Exercise 4. Spot the bug (level 4)

A nightly maintenance script. After an incident, the team cannot time travel even one day back. Tap the line that causes it, then fix the script.

What you need to know.

`SET key = value` changes a Spark setting for the rest of the session.

`ANALYZE TABLE t COMPUTE STATISTICS` collects table statistics, such as the row count and size in bytes, so the query optimiser picks better plans. It reads data and deletes nothing.

```
OPTIMIZE research_silver.nat_respondents ZORDER BY (wave_id);
SET spark.databricks.delta.retentionDurationCheck.enabled = false;
VACUUM research_silver.nat_respondents RETAIN 0 HOURS;
ANALYZE TABLE research_silver.nat_respondents COMPUTE STATISTICS;
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Architecture design (level 5)

The data governance policy says: "Any silver or gold change must be recoverable for 30 days." Choose the setting for each item.

Options: interval 7 days (default), interval 30 days, interval 0 hours.

1. delta.deletedFileRetentionDuration
2. delta.logRetentionDuration
3. Nightly VACUUM command

Sources

- DESCRIBE HISTORY, VERSION AS OF, versionAsOf, RESTORE TABLE and OPTIMIZE ... ZORDER BY work; RESTORE creates a new version; VACUUM RETAIN 0 HOURS fails the safety check. (verified: Run on delta-spark 4.0.1 in this repository.)
- deletedFileRetentionDuration defaults to 7 days and logRetentionDuration to 30 days; from Databricks Runtime 18.0 time travel beyond deletedFileRetentionDuration is blocked. (verified: https://docs.databricks.com/aws/en/delta/vacuum and https://docs.databricks.com/aws/en/tables/history (checked 2026-09-25))
- Databricks recommends liquid clustering and predictive optimization for Unity Catalog managed tables. (verified: https://docs.databricks.com/aws/en/tables/clustering (checked 2026-09-25))

A4 Unity Catalog

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Three-level namespace, grants, row filters and column masks.

A4-L1 The three-level namespace, one catalog per environment

Concept

Unity Catalog names every table with three parts: `catalog.schema.table`.

```
research_prod.research_silver.nat_respondents
```

Use each level for one purpose:

- **Catalog = environment** (or a major business domain): `research_dev`, `research_test`, `research_prod`. Permissions and storage attach at this level, so a dev job with no rights on `research_prod` cannot damage it.
- **Schema = layer or subject**: `research_bronze`, `research_silver`, `research_gold`, `research_governance`.
- **Table = content at a stated grain**: `nat_raw`, `nat_respondents`, `nat_metrics`.

Always write the full three-part name in pipeline code, built from configuration. A two-part name such as `research_silver.nat_respondents` resolves against the session's **current catalog**. That depends on the workspace default, a `USE CATALOG` somewhere earlier in the notebook, or a job setting. The same code then reads a different table in each place it runs.

Build names in one function from config, so no code concatenates strings on its own:

```
def table_name(catalog: str, layer: str, tracker: str) -> str:
    suffix = {"bronze": "raw", "silver": "respondents", "gold": "metrics"}[layer]
    return f"{catalog}.research_{layer}.{tracker}_{suffix}"
```

In a notebook, `spark.catalog.currentCatalog()` tells you which catalog a two-part name would use. It is useful for checks, never as a substitute for naming the catalog.

Workspaces created before Unity Catalog also show `hive_metastore`, the legacy two-level store. Treat it as read-only history while you migrate.

Why it matters

A shared all-purpose cluster has `research_prod` as its default catalog. A developer tests a silver fix from a notebook that writes to `research_silver.nat_respondents`, a two-part name, believing the notebook "runs in dev". The MERGE lands in production. The fix was only half finished, and 1,400 rows get wrong region codes before anyone notices.

With a three-part name built from a dev config file, the same notebook writes to `research_dev.research_silver.nat_respondents`. The developer's lack of MODIFY on `research_prod` would also have blocked it.

Worked example: weak code

```
1  spark.sql("USE CATALOG research_prod")
2
3  def load_silver(spark):
2      return spark.table("research_silver.nat_respondents")
5
6  def write_gold(df):
3      df.write.mode("append").saveAsTable("research_gold." + "nat" +
       "_metrics")
```

1. **Line 1.** Changes the current catalog for everything after it in the session.
2. **Line 4.** Two-part name. Which catalog it reads depends on the session, not the code.
3. **Line 7.** Names glued together in several places. One typo and a new table appears.

Worked example: strong code

```
1  LAYER_SUFFIX = {"bronze": "raw", "silver": "respondents", "gold":
     "metrics"}
2
3
4  def table_name(catalog: str, layer: str, tracker: str) -> str:
5      """Three-level table name for a tracker and layer.
6
7      Raises:
8          KeyError: If layer is not bronze, silver or gold.
9      """
2      return f"{catalog}.research_{layer}.{tracker}_{LAYER_SUFFIX[layer]}"
11
12
13 def load_silver(spark: SparkSession, cfg: RunConfig) -> DataFrame:
14     """Read the silver table for the configured environment and
       tracker."""
3      return spark.table(table_name(cfg.catalog, "silver", cfg.tracker))
```

1. **Line 1.** The naming convention lives in one constant.

2. **Line 10.** Always three parts. An unknown layer raises KeyError instead of inventing a table.
3. **Line 15.** The environment comes from config. The same code reads dev in dev and prod in prod.

The strong version reads exactly the table its config names, whatever the session's current catalog is.

Recap card

- **Unity Catalog name format** catalog.schema.table, for example research_prod.research_silver.nat_respondents.
- **What does each level hold in this project?** Catalog: environment. Schema: layer or governance. Table: tracker at a grain.
- **Why never write a two-part table name in pipeline code?** It resolves against the session's current catalog, which varies by cluster, notebook and job.
- **Where do table names come from?** One function built from config, such as table_name(catalog, layer, tracker).

Practice

Exercise 1. Multiple choice (level 1)

Which level of the Unity Catalog namespace should separate dev from prod?

- a) Catalog
- b) Schema
- c) Table name prefix, such as dev_nat_metrics
- d) A column called environment

Why?

- r1) Permissions and storage attach at catalog level, so dev principals can be kept out of prod entirely.
- r2) Catalog names are shorter.

Exercise 2. Fill the gap (level 2)

Complete the fully qualified name of the gold table for tracker `nat` in production.

```
GOLD = "_____(1)._____(2).nat_metrics"
```

Exercise 3. Spot the bug (level 3)

This job runs in the dev workspace, but its writes appear in production. Tap the line, then fix the function.

```
def write_metrics(df: DataFrame, cfg: RunConfig) -> None:
    """Write gold metrics for the configured environment."""
    target = f"research_gold.{cfg.tracker}_metrics"
    df.write.format("delta").mode("append").saveAsTable(target)
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 4)

Write `table_name(catalog, layer, tracker)`. It returns `catalog.research_<layer>.<tracker>_<suffix>` where the suffix is `raw`, `respondents` or `metrics` for bronze, silver and gold. Raise `ValueError` naming the layer for any other layer, and raise `ValueError` if `catalog` contains a dot.

Your code must pass these checks:

- Build catalog.research_<layer>.<tracker>_<suffix> with raw, respondents and metrics.
- An unknown layer must raise ValueError that names it.
- A catalog containing a dot would produce a four-part name. Raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
LAYER_SUFFIX = ...

# Three-level Unity Catalog name for a tracker's table in one layer.
def table_name(
    catalog: str,
    layer: str,
    tracker: str,
) -> str: ...
```

Starter code:

```
def table_name(catalog: str, layer: str, tracker: str) -> str:
    ...
```

Exercise 5. Architecture design (level 5)

You lay out Unity Catalog for the research programme. Choose the level for each item.

Options: Catalog, Schema, Table.

1. Development versus production

2. Bronze, silver and gold
3. Run log, variable registry and contracts
4. One tracker's respondents at respondent x wave grain

Sources

- Unity Catalog uses a three-level namespace, catalog.schema.table, and two-part names resolve against the current catalog. (verified: https://docs.databricks.com/aws/en/data-governance/unity-catalog/ (checked 2026-09-25))
- spark.catalog.currentCatalog() returns the current catalog (PySpark 3.4 and later). (verified: Present on PySpark 4.0.4 in this repository.)

A4-L2 Grant to groups, grant the least, grant at the right level

Concept

Unity Catalog controls access with privileges on securable objects. Four privileges cover most pipeline work:

Privilege	On	Lets the principal
`USE CATALOG`	catalog	see into the catalog. Grants no data access by itself.
`USE SCHEMA`	schema	see into the schema. Grants no data access by itself.
`SELECT`	table, or schema	read data
`MODIFY`	table, or schema	insert, update and delete data

To read a table, a principal needs all three: `USE CATALOG` on its catalog, `USE SCHEMA` on its schema and `SELECT` on the table. `MODIFY` also needs `SELECT` and both `USE` privileges.

Grant at the container. A privilege granted on a schema is inherited by every current and future table in it. `GRANT SELECT ON SCHEMA research_prod.research_gold` covers next month's new gold table without another ticket.

Grant to groups, never to individual users. People join, move and leave. A group grant is managed once, in one place. A user grant outlives the reason it was made, and nobody finds it until an audit.

Run pipelines as a service principal. Jobs that write bronze, silver and gold run as a service principal with `MODIFY` on those schemas. People get `SELECT` on the layers they need, and no person needs `MODIFY` on production.

```
GRANT USE CATALOG ON CATALOG research_prod TO `research_analysts`;
GRANT USE SCHEMA, SELECT ON SCHEMA research_prod.research_gold TO
  `research_analysts`;
SHOW GRANTS ON SCHEMA research_prod.research_gold;
```

Avoid `ALL PRIVILEGES` outside sandboxes. It includes privileges you did not think about, and future privileges too.

Why it matters

During a deadline, an analyst gets `MODIFY` on `research_prod.research_silver` "for one day" to patch a mapping by hand. The grant is to their user account. Eight months later they run a clean-up notebook against what they believe is a dev copy and delete 12,000 silver rows. Nobody remembered the grant, and silver had to be rebuilt from bronze.

With the rule "people get SELECT, pipelines get MODIFY, grants go to groups", the patch would have gone through the pipeline, and the delete would have failed with a permission error.

Worked example: weak code

```
GRANT ALL PRIVILEGES ON CATALOG research_prod TO `alex@example.org`;
GRANT SELECT ON TABLE research_prod.research_gold.nat_metrics TO
  `research_analysts`;
GRANT MODIFY ON SCHEMA research_prod.research_silver TO
  `research_analysts`;
```

1. **Line 1.** Everything, on all of production, to one person. Outlives the reason for it.
2. **Line 2.** SELECT alone is not enough, and a table-level grant misses every new gold table.
3. **Line 3.** Analysts can now change silver directly, bypassing the pipeline and its checks.

Worked example: strong code

```
-- Analysts read gold.
GRANT USE CATALOG ON CATALOG research_prod TO `research_analysts`;
GRANT USE SCHEMA, SELECT ON SCHEMA research_prod.research_gold TO
  `research_analysts`;
```

```
4
5  -- The pipeline service principal writes silver and gold.
6  GRANT USE CATALOG ON CATALOG research_prod TO `research-pipeline-sp`;
2  GRANT USE SCHEMA, SELECT, MODIFY ON SCHEMA research_prod.research_silver
     TO `research-pipeline-sp`;
8  GRANT USE SCHEMA, SELECT, MODIFY ON SCHEMA research_prod.research_gold TO
     `research-pipeline-sp`;
9
10 -- Check what is in place.
3  SHOW GRANTS ON SCHEMA research_prod.research_gold;
```

1. **Line 3.** Schema level: inherited by every current and future gold table.
2. **Line 7.** Only the pipeline identity writes to silver.
3. **Line 11.** Review grants from SQL, and keep these statements in version control.

Three rules: groups not users, schemas not tables, SELECT for people and MODIFY for pipelines.

Recap card

- **Privileges needed to read a table** USE CATALOG on the catalog, USE SCHEMA on the schema, SELECT on the table or schema.
- **Why grant SELECT on the schema rather than the table?** Schema grants are inherited by every current and future table in it.
- **Who gets MODIFY on production silver and gold?** The pipeline service principal. People get SELECT.
- **Why never grant to a user account?** User grants outlive the reason for them. Group membership is managed in one place.

Practice

Exercise 1. Put in order (level 1)

Analysts must read every table in `research_prod.research_gold`. Put the grants in order from the outermost object inwards.

- GRANT USE SCHEMA ON SCHEMA research_prod.research_gold TO `research_analysts`
- GRANT SELECT ON SCHEMA research_prod.research_gold TO `research_analysts`
- GRANT USE CATALOG ON CATALOG research_prod TO `research_analysts`

Exercise 2. Multiple choice (level 2)

An analyst has `SELECT` on `research_prod.research_gold.nat_metrics` and nothing else. What happens when they query the table?

- a) The query succeeds.
- b) The query fails, because they lack USE CATALOG and USE SCHEMA.
- c) The query returns an empty result.
- d) The query succeeds for columns without a mask.

Why?

- r1) Reading needs USE on the parent catalog and schema as well as SELECT on the table.
- r2) SELECT only works when granted on a schema.

Exercise 3. Spot the bug (level 3)

A reviewer rejects this grant script. Tap the line that breaks the "groups, not users" rule, then rewrite the script.

```
1 GRANT USE CATALOG ON CATALOG research_prod TO `research_analysts`;
2 GRANT USE SCHEMA, SELECT ON SCHEMA research_prod.research_gold TO
    `research_analysts`;
3 GRANT USE SCHEMA, SELECT ON SCHEMA research_prod.research_silver TO
    `sam.jones@example.org`;
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 4)

Write the grants for the pipeline service principal `research-pipeline-sp`: it must read and write every table in `research_prod.research_silver`, now and in future. Give it only what it needs, on the right objects.

Shape of the answer. The reference answer follows this outline. Replace each ... with your own code.

```
1 GRANT USE CATALOG ...
2 GRANT USE ...
```

Starter code:

```
1 -- Grants for research-pipeline-sp
```

Exercise 5. Architecture design (level 5)

Assign each principal the least privilege that lets them do their job.

Options: USE CATALOG, USE SCHEMA, SELECT on the schema, USE CATALOG, USE SCHEMA, SELECT, MODIFY on the schema, ALL PRIVILEGES on the catalog, Nothing on this schema.

1. research_analysts build Power BI reports from gold
2. research-pipeline-sp writes silver
3. research_analysts and the bronze schema
4. Platform administrators managing the metastore

Sources

- Reading a table needs USE CATALOG, USE SCHEMA and SELECT; MODIFY needs SELECT and both USE privileges; privileges granted on a schema are inherited by current and future tables. (search-only: https://docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/)
- GRANT accepts a comma-separated list of privileges, and SHOW GRANTS lists grants on a securable. (verified: https://docs.databricks.com/aws/en/sql/language-manual/security-grant and https://docs.databricks.com/aws/en/sql/language-manual/security-show-grant (checked 2026-09-25))

A4-L3 Row filters and column masks in Unity Catalog

Concept

Grants decide who can read a table. **Row filters** and **column masks** decide what each reader sees inside it. Both are SQL user-defined functions that Unity Catalog applies at query time, for every query, from every tool.

Row filter. A SQL UDF that returns BOOLEAN. Unity Catalog passes it the columns you name, and drops each row where the function returns FALSE or NULL.

```
1 CREATE FUNCTION
    research_prod.research_governance.respondent_filter(tracker_id STRING)
2 RETURN is_account_group_member('research_approved_analysts') OR
    tracker_id = 'TRK-PUBLIC';
3
4 ALTER TABLE research_prod.research_silver.nat_respondents
5 SET ROW FILTER research_prod.research_governance.respondent_filter ON
    (tracker_id);
```

Column mask. A SQL UDF whose first parameter is the column value. It returns the value, or a masked version.

```
CREATE FUNCTION research_prod.research_governance.mask_postcode(postcode
  STRING)
RETURN CASE WHEN is_account_group_member('pii_readers') THEN postcode
  ELSE '***' END;

ALTER TABLE research_prod.research_silver.nat_respondents
ALTER COLUMN postcode SET MASK
  research_prod.research_governance.mask_postcode;
```

Three rules:

- Use `is_account_group_member()`. It checks account-level groups, which Unity Catalog uses. `is_member()` checks workspace-local groups.
- Test group membership, never `current_user() = 'someone@example.org'`. A named user in a function is a hidden grant nobody reviews.
- Keep filter and mask functions in a governance schema, so their owners and grants are separate from the data.

Your reference script uses `CREATE ROW ACCESS POLICY`. That is Snowflake syntax. Databricks rejects it. Where many tables share one rule, Databricks also offers attribute-based policies on governed tags.

Why it matters

The reference `unity_catalog.sql` was written with an AI assistant and never run. Its row-level security uses `CREATE ROW ACCESS POLICY`, which does not exist in Databricks. The deployment script stops at that statement, and the PII table goes live without a filter, because the grants earlier in the script had already succeeded. Every analyst with SELECT on silver can read restricted respondent rows until someone notices the failed step.

A SQL UDF with `SET ROW FILTER`, tested in dev, applies the rule where Databricks enforces it.

Worked example: weak code

```
1 CREATE ROW ACCESS POLICY restrict_respondent_data
2     ON research_silver.nat_respondents
3     USING (
4         is_member('research_approved_analysts')
5         OR current_user() = 'pipeline-svc@example.org'
6     );
```

1. **Line 1.** Snowflake syntax. Databricks raises a parse error. Reference defect 4.
2. **Line 2.** Two-part name: resolves against whatever catalog is current.
3. **Line 4.** Checks workspace-local groups. Unity Catalog groups are account-level.
4. **Line 5.** A named identity inside a function: a grant hidden from every access review.

Worked example: strong code

```
1  CREATE OR REPLACE FUNCTION
     research_prod.research_governance.respondent_filter(tracker_id STRING)
2  RETURN is_account_group_member('research_approved_analysts')
2    OR is_account_group_member('research_pipelines');
4
5  ALTER TABLE research_prod.research_silver.nat_respondents
3  SET ROW FILTER research_prod.research_governance.respondent_filter ON
     (tracker_id);
7
8  CREATE OR REPLACE FUNCTION
     research_prod.research_governance.mask_postcode(postcode STRING)
9  RETURN CASE WHEN is_account_group_member('pii_readers') THEN postcode
     ELSE '***' END;
10
11 ALTER TABLE research_prod.research_silver.nat_respondents
4  ALTER COLUMN postcode SET MASK
     research_prod.research_governance.mask_postcode;
```

1. **Line 1.** A SQL UDF in the governance schema. Its parameter receives the column named in ON (...).
2. **Line 3.** The pipeline service principal is in a group. No identity is named in the function.
3. **Line 6.** Binds the filter to the table. It applies to every query from every tool.
4. **Line 12.** Masks one column. Readers outside pii_readers see ***.

The strong script runs on Databricks, names groups instead of people, and uses three-part names throughout.

Recap card

- **Row filter in Databricks** A SQL UDF returning BOOLEAN, bound with ALTER TABLE t SET ROW FILTER f ON (col).
- **Column mask in Databricks** A SQL UDF taking the column value, bound with ALTER TABLE t ALTER COLUMN c SET MASK f.
- **is_account_group_member or is_member?** is_account_group_member: Unity Catalog groups are account-level. is_member checks workspace groups.
- **What is CREATE ROW ACCESS POLICY?** Snowflake syntax. Databricks rejects it. Reference defect 4.

Practice

Exercise 1. Multiple choice (level 1)

Which statement applies a row filter to a Unity Catalog table?

- a) `CREATE ROW ACCESS POLICY p ON t USING (...)`
- b) `ALTER TABLE t SET ROW FILTER f ON (col)`
- c) `GRANT SELECT ON t TO g WHERE col = 'x'`
- d) `CREATE VIEW v AS SELECT * FROM t WHERE ...` and nothing else

Why?

- r1) Databricks row filters are SQL UDFs bound to a table with SET ROW FILTER.
- r2) Row filters must be created before the table.

Exercise 2. Fill the gap (level 2)

Complete the mask so that only members of the account group `pii_readers` see real postcodes.

```
CREATE FUNCTION research_prod.research_governance.mask_postcode(postcode
  STRING)
RETURN CASE WHEN _____(1)('pii_readers') THEN postcode ELSE '***' END;

ALTER TABLE research_prod.research_silver.nat_respondents
ALTER COLUMN postcode _____(2)
  research_prod.research_governance.mask_postcode;
```

Exercise 3. Spot the bug (level 3)

An access review flags this filter. Tap the line that hides a grant from reviewers, then rewrite the function.

```
CREATE OR REPLACE FUNCTION
  research_prod.research_governance.respondent_filter(tracker_id STRING)
RETURN is_account_group_member('research_approved_analysts')
   OR current_user() = 'pipeline-svc@example.org';
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 4)

Write a row filter for `research_prod.research_gold.nat_metrics`. Members of `research_all_regions` see every row. Everyone else sees only rows where `region = 'NATIONAL'`. Create the function `research_prod.research_governance.region_filter` and bind it to the table on the `region` column.

Shape of the answer. The reference answer follows this outline. Replace each ... with your own code.

```
CREATE OR REPLACE ...
RETURN ...

ALTER TABLE ...
SET ROW FILTER ...
```

Starter code:

```
CREATE OR REPLACE FUNCTION
  research_prod.research_governance.region_filter(region STRING)
RETURN ...
```

Exercise 5. Architecture design (level 5)

Choose the control for each requirement on `nat_respondents`.

Options: Grants (or their absence), Row filter, Column mask.

1. The client team must not query the table at all
2. Analysts see only the trackers their group is approved for
3. Analysts see respondent rows, but postcodes appear as *** unless they are in pii_readers

Sources

- Row filters are SQL UDFs returning BOOLEAN bound with SET ROW FILTER ... ON (cols); column masks are bound with ALTER COLUMN ... SET MASK. (verified: https://docs.databricks.com/aws/en/data-governance/unity-catalog/filters-and-masks/manually-apply (checked 2026-09-25))
- is_account_group_member returns true when the session user is a direct or indirect member of an account-level group; is_member checks workspace groups. (verified: https://docs.databricks.com/aws/en/sql/language-manual/functions/is_account_group_member (checked 2026-09-25))
- Attribute-based access control (ABAC) policies apply filters and masks centrally through governed tags. (verified: https://docs.databricks.com/aws/en/data-governance/unity-catalog/abac/policies (checked 2026-09-25))

A5 PySpark performance

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Lazy evaluation, built-ins over UDFs, joins, partitioning, caching, explain plans.

A5-L1 Lazy evaluation, and why every count() re-runs the plan

Concept

Spark splits DataFrame operations into two kinds.

- **Transformations** build a plan and return a new DataFrame at once: `filter`, `select`, `withColumn`, `join`, `groupBy().agg()`. Nothing is read yet.
- **Actions** run the plan and return a result: `count()`, `collect()`, `show()`, `first()`, `toPandas()`, and every write.

This is **lazy evaluation**. It lets the optimiser see the whole plan before running it, so it can push filters into the file scan and skip columns you never use.

The cost is easy to miss: **each action runs the whole plan again**, from the source. Nothing is kept between actions unless you ask for it. A local test shows it plainly: a Python function counted its own calls inside a plan over 1,000 rows. After the first `count()`, it had run 1,000 times. After a second `count()`, 2,000.

Your reference data-contract validator calls `count()` more than ten times on the same uncached DataFrame: once for the row count, once per mandatory column, once per key and so on. Each call rereads the delivery from storage.

Three fixes, in order of preference:

1. **Combine the questions into one action.** One `agg` with many expressions answers them all in a single pass:

```
python df.agg( F.count("*").alias("rows"),
F.count_if(F.col("final_weight").isNull()).alias("null_weights"),
F.sum("final_weight").alias("weight_sum"), ).collect()[0]
```

1. **Materialise** an expensive intermediate result to a Delta table, then read that table for the follow-up actions.
2. **Cache** with `df.cache()` on classic clusters only. Serverless compute, which Free Edition uses, does not support cache APIs.

Why it matters

A contract validator checks a 40-million-row delivery with 23 separate `count()` calls, one per check. Each call rescans the source files in cloud storage. The validation step takes 38 minutes, longer than the transform it protects, and the team starts skipping it "for small redeliveries". A redelivery with duplicated keys then goes straight to silver.

Rewritten as one `agg` with 23 expressions, the same checks run in a single scan of about two minutes, and nobody has a reason to skip them.

Worked example: weak code

```
def check_delivery(df):
    results = {}
    results["rows"] = df.count()
    results["null_ids"] =
df.filter(F.col("respondent_id").isNull()).count()
    results["null_weights"] =
df.filter(F.col("final_weight").isNull()).count()
    results["distinct_ids"] =
df.select("respondent_id").distinct().count()
    return results
```

1. **Line 3.** Action 1: full scan of the source.
2. **Line 4.** Action 2: a second full scan.
3. **Line 5.** Action 3: a third.
4. **Line 6.** Action 4: a fourth scan, plus a shuffle.

Worked example: strong code

```
def check_delivery(df: DataFrame) -> dict[str, int]:
    """Row, null and distinct counts for a delivery in one pass."""
    row = df.agg(
        F.count("*").alias("rows"),
        F.count_if(F.col("respondent_id").isNull()).alias("null_ids"),
        F.count_if(F.col("final_weight").isNull()).alias("null_weights"),
        F.count_distinct("respondent_id").alias("distinct_ids"),
    ).collect()[0]
    return row.asDict()
```

1. **Line 3.** One action. Spark computes every expression during the same scan.
2. **Line 5.** count_if counts rows where a condition holds, inside the aggregation.
3. **Line 7.** Distinct count in the same pass. For very large data, approx_count_distinct is cheaper.

Four questions, one scan. The strong version reads the delivery once, however many checks you add.

Recap card

- **Transformation or action?** Transformations (filter, select, join, groupBy().agg) build a plan. Actions (count, collect, show, write) run it.
- **What happens on the second count() of an uncached DataFrame?** The whole plan runs again from the source.
- **Many checks, one scan** One df.agg(...) with count, count_if, sum, min, max expressions.
- **Reusing an expensive result on serverless** Write it to a Delta table. cache() is not supported on serverless compute.

Practice

Exercise 1. Multiple choice (level 1)

Which line starts a Spark job?

- a) `df2 = df.filter(F.col("in_scope"))`
- b) `df3 = df2.withColumn("w2", F.col("final_weight") * 2)`
- c) `n = df3.count()`
- d) `df4 = df3.groupBy("wave_id").agg(F.sum("w2"))`

Why?

- r1) count() is an action. The other lines are transformations that only extend the plan.
- r2) Only the last line of a cell runs.

Exercise 2. Predict the output (level 2)

`tag` is a Python UDF that adds 1 to an accumulator each time it runs. `src` has 1,000 rows and is not cached. What does the last line print?

```
df = src.select(tag("id").alias("id")).filter(F.col("id") % 2 == 0)
df.count()
df.count()
print(calls.value)
```

Exercise 3. Fill the gap (level 2)

Count rows and null weights in a single pass.

```
stats = df._____(1)(
    F.count("*").alias("rows"),
    F._____(2)(F.col("final_weight").isNull()).alias("null_weights"),
).collect()[0]
```

Exercise 4. Refactor (level 4)

Refactor `table_health` to answer every question in **one** action. Keep the returned keys: `rows`, `null_ids`, `null_weights`, `min_weight`, `max_weight`.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Row count, null counts and weight range in one pass over df.
def table_health(
    df: DataFrame,
) -> dict[str, float | int | None]: ...
```

Starter code:

```python
def table_health(df):
    return {
        "rows": df.count(),
        "null_ids": df.filter(F.col("respondent_id").isNull()).count(),
        "null_weights": df.filter(F.col("final_weight").isNull()).count(),
        "min_weight": df.agg({"final_weight": "min"}).collect()[0][0],
        "max_weight": df.agg({"final_weight": "max"}).collect()[0][0],
    }
```

Exercise 5. Architecture design (level 5)

You run the silver QA notebook on Databricks Free Edition (serverless). An expensive joined DataFrame feeds six different checks. Choose the approach for each situation.

Options: Combine them into one agg, Write the joined data to a Delta table and read it back, df.cache(), Leave each check as its own action.

1. Four checks are simple counts and sums over the joined data
2. Two checks need different groupings and joins over the same expensive result
3. A colleague suggests cache() to speed it all up on serverless

Sources

- Each action re-executes the plan: a counting UDF ran 1,000 times after one count and 2,000 after two, on 1,000 rows. (verified: Run on PySpark 4.0.4 in this repository.)
- F.count_if and F.count_distinct exist in PySpark 4.0. (verified: Run on PySpark 4.0.4 in this repository.)
- Serverless compute does not support DataFrame or SQL cache APIs. (verified: https://docs.databricks.com/aws/en/compute/serverless/limitations (checked 2026-09-25))

A5-L2 Built-in functions over Python UDFs

Concept

A **Python UDF** runs your Python function once per row. Spark must send each value from the JVM to a Python worker process, run the function, and send the result back. The physical plan shows this as a `BatchEvalPython` step.

A **built-in function** from `pyspark.sql.functions` runs inside the JVM, in Spark's generated code, with no round trip.

Measured on 2 million strings (PySpark 4.0.4, two local cores):

Approach	Time
`F.upper(col)`	0.30 s
pandas UDF (`ArrowEvalPython`)	1.16 s
Python UDF (`BatchEvalPython`)	1.82 s

The Python UDF was six times slower, and the gap grows with cluster size and data volume.

Speed is one reason. There are three more:

- **Nulls.** Built-ins return NULL for NULL input. A UDF receives `None` and must handle it. `lambda s: s.strip()` fails the whole job with a `PythonException` on the first NULL.
- **Optimisation.** The optimiser cannot see inside a UDF, so it cannot push a filter on the UDF's output into the file scan.
- **Readability.** `F.when(...).when(...)` states a rule in one expression a reviewer can check.

Most UDFs in pipelines replace functions that already exist: `F.upper`, `F.trim`, `F.regexp_replace`, `F.regexp_extract`, `F.split`, `F.to_date`, `F.coalesce`, `F.when`, `F.concat_ws`, `F.lpad`. Search the functions reference before you write one.

When no built-in fits, prefer a **pandas UDF**: it moves data in Arrow batches and runs vectorised pandas code. Use a row-by-row Python UDF only as the last resort, and handle `None` first.

Why it matters

A silver job cleans postcodes with a Python UDF: `lambda s: s.replace(" ", "").upper()`. It runs for months on deliveries where every postcode is present. Then a supplier sends a wave with 40 blank postcodes, which arrive as NULL. The first NULL raises `AttributeError: 'NoneType' object has no`

`attribute 'replace'` inside a Python worker, and the whole silver job fails at 02:00.

`F.upper(F.regexp_replace("postcode", r"\s+", ""))` returns NULL for NULL, runs faster, and would never have failed.

Worked example: weak code

```
1  from pyspark.sql.functions import udf
2
3
4  @udf("string")
5  def clean_postcode(s):
1      return s.replace(" ", "").upper()
7
8
9  @udf("string")
10 def age_band(age):
2      if age < 35:
12         return "16-34"
13     if age < 55:
14         return "35-54"
15     return "55+"
16
17
3  silver = bronze.withColumn("postcode",
       clean_postcode("postcode")).withColumn("age_group", age_band("age"))
```

1. **Line 6.** Crashes on NULL: None has no replace method.
2. **Line 11.** In Python, comparing None with 35 raises TypeError, so a NULL age crashes the job too.
3. **Line 18.** Two round trips to Python per row, invisible to the optimiser.

Worked example: strong code

```
1  from pyspark.sql import Column
2  from pyspark.sql import functions as F
3
4
1  def clean_postcode(col: Column) -> Column:
6      """Remove whitespace and upper-case. NULL stays NULL."""
2      return F.upper(F.regexp_replace(col, r"\s+", ""))
8
9
10 def age_band(age: Column) -> Column:
11     """16-34, 35-54 or 55+. NULL age gives NULL."""
12     return (
13         F.when(age < 35, "16-34")
14         .when(age < 55, "35-54")
3          .when(age.isNotNull(), "55+")
16     )
```

```
17
18
19 silver = bronze.withColumns({
20     "postcode": clean_postcode(F.col("postcode")),
21     "age_group": age_band(F.col("age")),
22 })
```

1. **Line 5.** A function that builds a Column expression. Reusable and testable, with no UDF.
2. **Line 7.** Built-ins pass NULL through as NULL.
3. **Line 15.** Explicit: a NULL age stays NULL instead of falling into 55+.

The strong version runs in the JVM, survives NULLs and reads as a rule. Functions that return Column expressions give you the reuse that UDFs promised.

Recap card

- **Why are Python UDFs slow?** Every value crosses from the JVM to a Python worker and back. The plan shows BatchEvalPython.
- **Measured cost on 2 million strings** Built-in 0.30 s, pandas UDF 1.16 s, Python UDF 1.82 s (PySpark 4.0.4, local).
- **What does a Python UDF receive for NULL?** None. s.strip() then raises and fails the job. Built-ins return NULL for NULL.
- **Reusable logic without a UDF** A Python function that takes and returns Column expressions, built from F.when, F.regexp_replace and friends.

Practice

Exercise 1. Multiple choice (level 1)

You need to trim and upper-case a string column on 50 million rows. Which approach is fastest?

- a) A Python UDF: `udf(lambda s: s.strip().upper() if s else None)`
- b) A pandas UDF using `s.str.strip().str.upper()`
- c) `F.upper(F.trim(F.col("s")))`
- d) `collect()` the column and process it in Python

Why?

- r1) Built-ins run inside the JVM with no data transfer to Python.
- r2) Built-ins run on the driver, which is faster than executors.

Exercise 2. Predict the output (level 2)

What does this print?

```
df = spark.createDataFrame([(" ab 1 ",), (None,)], "s STRING")
out = df.select(F.upper(F.regexp_replace(F.col("s"), r"\s+",
    "")).alias("u"))
print([r.u for r in out.collect()])
```

Exercise 3. Fill the gap (level 3)

Replace a UDF that pads respondent IDs to 8 characters with leading zeros and extracts the 2-letter region prefix from a code such as `NW-1234`.

```
df = df.withColumns({
    "respondent_id": F._____(1)(F.col("respondent_id"), 8, "0"),
    "region": F._____(2)(F.col("area_code"), r"^([A-Z]{2})-", 1),
})
```

Exercise 4. Refactor (level 4)

Replace the UDF with a function `age_band(age: Column) -> Column` built from `F.when`. Keep the bands: under 35 is `16-34`, under 55 is `35-54`, anything else `55+`. A NULL age must give NULL.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# 16-34, 35-54 or 55+. NULL age gives NULL.
def age_band(age: Column) -> Column: ...
```

Starter code:

```
@udf("string")
def age_band(age):
    if age < 35:
        return "16-34"
    if age < 55:
        return "35-54"
    return "55+"
```

Exercise 5. Spot the bug (level 5)

This job ran for months, then failed on a wave where some postcodes are blank. Tap the line that fails, then rewrite `clean_postcodes` without a UDF.

```
from pyspark.sql.functions import udf


@udf("string")
def _clean(s):
    return s.replace(" ", "").upper()


def clean_postcodes(df: DataFrame) -> DataFrame:
    """Remove spaces from postcodes and upper-case them."""
    return df.withColumn("postcode", _clean("postcode"))
```

Write down the line number of the bug. Then write the fix.

Sources

- Timings: built-in 0.30 s, pandas UDF 1.16 s, Python UDF 1.82 s on 2 million strings; plans show BatchEvalPython and ArrowEvalPython. (verified: Measured on PySpark 4.0.4, local[2], in this repository.)
- Built-ins return NULL for NULL input; a Python UDF calling a string method on None raises PythonException. (verified: Run on PySpark 4.0.4 in this repository.)

A5-L3 Joins, broadcast, partitions and reading the plan

Concept

A join between two large tables needs a **shuffle**: Spark moves rows across the network so equal keys meet on the same executor, then sorts and merges them. The plan shows this as `SortMergeJoin` with `Exchange` steps. Shuffles are the most expensive thing Spark does.

When one side is small, such as a lookup of 200 region codes, Spark can **broadcast** it: send a full copy to every executor and join locally, with no shuffle of the large side. The plan shows `BroadcastHashJoin`.

- Spark broadcasts automatically when it estimates a side is under `spark.sql.autoBroadcastJoinThreshold`, 10 MB by default in Spark 4.0.
- Adaptive query execution (AQE), on by default, can switch a join to broadcast at run time once it sees real sizes.
- You can force it with `F.broadcast(small_df)` or `small_df.hint("broadcast")`. Do this for lookups you know are small. Never broadcast something that grows with your data.

Read the plan before you tune. `df.explain()` prints the physical plan. `df.explain(mode="formatted")` adds details per step, including `PushedFilters`, which confirm that a filter reached the file scan.

Partitions decide parallelism.

- `repartition(n)` shuffles to exactly `n` partitions. `coalesce(n)` merges partitions without a shuffle, and only reduces the count.
- `repartition(1)` or `coalesce(1)` before a large write puts all the work on one task. Avoid it outside tiny outputs.

- For table layout on Unity Catalog managed tables, prefer liquid clustering over `partitionBy`. If you do partition a table, choose a low-cardinality column such as `wave_id`, never an ID.

Caching: `df.cache()` helps on classic clusters when several actions reuse a result. Serverless compute does not support it. There, write the intermediate result to a Delta table.

Why it matters

A gold job joins 40 million respondent rows to a 200-row region lookup. Auto-broadcast does not kick in, because the lookup is read through a view whose size Spark overestimates. Every run shuffles all 40 million rows across the cluster, and the join takes 22 minutes.

`explain()` shows `SortMergeJoin` with two `Exchange` steps. Wrapping the lookup in `F.broadcast()` turns it into `BroadcastHashJoin`, and the join drops to under two minutes. Five minutes reading the plan saved twenty minutes on every run.

Worked example: weak code

```
1  def enrich(fact, regions):
(1)    regions_local = regions.collect()
3      lookup = {r.region_code: r.region for r in regions_local}
(2)    label = F.udf(lambda code: lookup.get(code), "string")
5      out = fact.withColumn("region", label("region_code"))
(3)    return out.repartition(1)
```

1. **Line 2.** Pulls the lookup to the driver to build a Python dict. A join does this job.
2. **Line 4.** A Python UDF per row, where a broadcast join would stay in the JVM.
3. **Line 6.** All 40 million rows squeezed into one partition, so one task does all the writing.

Worked example: strong code

```
1  def enrich_with_regions(fact: DataFrame, regions: DataFrame) -> DataFrame:
2      """Add region names to fact rows. regions is a small lookup, so
       broadcast it.
3
4      A left join keeps every fact row, even with an unknown region code.
5      """
(2)    return fact.join(F.broadcast(regions), on="region_code", how="left")
7
8
9  def show_plan(df: DataFrame) -> None:
10     """Print the formatted physical plan: check for BroadcastHashJoin."""
```

```
❸     df.explain(mode="formatted")
```

1. **Line 6.** A left join keeps fact rows with unknown codes. An inner join would silently drop them.
2. **Line 6.** Broadcast the small side. The plan shows BroadcastHashJoin and no shuffle of the fact table.
3. **Line 11.** Confirm the strategy and the pushed filters before and after tuning.

One broadcast join replaces a collect, a dict and a UDF, keeps the work distributed, and shows up clearly in the plan.

Recap card

- **Join strategy for a large table and a small lookup** Broadcast the lookup: F.broadcast(small). The plan shows BroadcastHashJoin.
- **Default auto-broadcast threshold in Spark 4.0** 10 MB (spark.sql.autoBroadcastJoinThreshold). AQE can also switch to broadcast at run time.
- **repartition versus coalesce** repartition(n) shuffles to n partitions. coalesce(n) merges without a shuffle and only reduces.
- **How do you confirm a filter reached the file scan?** df.explain(mode="formatted") and look for PushedFilters on the scan.

Practice

Exercise 1. Multiple choice (level 1)

You join a 40-million-row respondent table with a 200-row region lookup. Which strategy should the plan show?

- a) SortMergeJoin, shuffling both sides
- b) BroadcastHashJoin, with the lookup sent to every executor
- c) Collect the lookup and use a Python UDF
- d) A cross join, then a filter

Why?

- r1) Broadcasting the small side avoids shuffling the large side.
- r2) Broadcast joins always use less memory.

Exercise 2. Predict the output (level 2)

Which join node name appears in the physical plan printed by this code on Spark 4.0?

```
1 joined = fact.join(F.broadcast(regions), "region_code", "left")
2 joined.explain()
```

Exercise 3. Fill the gap (level 3)

Broadcast the lookup, keep every fact row, then print the detailed plan.

```
joined = fact.join(F._____(1)(regions), on="region_code", how="_____(2)")
joined.explain(mode="_____(3)")
```

Exercise 4. Spot the bug (level 4)

Writing gold takes 45 minutes on a 16-core cluster, and the Spark UI shows one busy task. Tap the line, then fix the function.

```
def write_gold(metrics: DataFrame, table: str) -> None:
    """Write gold metrics for the wave."""
    (
        metrics.repartition(1)
        .write.format("delta")
        .mode("append")
        .saveAsTable(table)
    )
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Write the function (level 5)

Write `enrich_with_regions(fact, regions)`. It left-joins `fact` to the small `regions` lookup on `region_code`, broadcasting the lookup. Then write `unmatched_codes(enriched)` that returns the number of rows where `region` is NULL, using a single aggregation.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Left-join fact rows to the small regions lookup, broadcasting the
    lookup.
def enrich_with_regions(
    fact: DataFrame,
    regions: DataFrame,
) -> DataFrame: ...

# Rows whose region_code had no match in the lookup.
def unmatched_codes(enriched: DataFrame) -> int: ...
```

Starter code:

```
from pyspark.sql import DataFrame
from pyspark.sql import functions as F


def enrich_with_regions(fact: DataFrame, regions: DataFrame) -> DataFrame:
    ...


def unmatched_codes(enriched: DataFrame) -> int:
```

```
10     ...
```

Practical: Read the plan: sort-merge join versus broadcast join

Platform: Databricks Free Edition. Run this in Databricks Free Edition. It builds a fact table and a small lookup, then prints the physical plan with and without a broadcast hint, so you can find `SortMergeJoin`, `BroadcastHashJoin` and `Exchange` yourself. On serverless, Spark settings you can change are limited, so the notebook compares plans rather than timings.

Download the notebook `A5-L3.ipynb` from the app.

Sources

- F.broadcast and hint('broadcast') give BroadcastHashJoin; with broadcasting off, SortMergeJoin; autoBroadcastJoinThreshold defaults to 10 MB; AQE is on by default; explain supports mode='formatted' with PushedFilters. (verified: Run on PySpark 4.0.4 in this repository.)
- Databricks recommends liquid clustering over partitioning for UC managed tables; serverless does not support cache APIs. (verified: https://docs.databricks.com/aws/en/tables/clustering and https://docs.databricks.com/aws/en/compute/serverless/limitations (checked 2026-09-25))

A6 Data quality

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Expectations, health checks and reconciliation between layers.

A6-L1 Data contracts with severity levels and structured results

Concept

A **data contract** states what a delivery must look like before it may enter silver: row count range, mandatory columns, weight total, unique keys, allowed categories, expected distributions. Your reference project keeps each contract as versioned JSON, such as `nat_contract_v2.json`.

Every check produces a **structured result**, never a sentence:

```python
CheckName = str
Status = Literal["PASS", "WARN", "FAIL", "SKIP"]
Severity = Literal["CRITICAL", "HIGH", "MEDIUM", "LOW"]

@dataclass(frozen=True)
class CheckResult:
    check_name: CheckName
    status: Status
    severity: Severity
    expected: str
    actual: str
```

Records can be counted, stored in a Delta table, charted and compared across waves. A narrative such as "weights look a bit high" cannot.

Severity decides what happens next.

- A failed **CRITICAL** check stops the pipeline: raise an exception, so the job fails and silver stays untouched. Row count out of range, null keys, duplicate keys and a weight total outside limits are CRITICAL.
- A **WARN** logs the result and the run continues. Unexpected category values and distribution shifts are warnings: worth a look, not worth a missed report.

Evaluate every check first, store all the results, then decide. If you raise on the first failure, you learn about one problem per run.

Keep the contract in data and the logic in code. A new contract version changes JSON, not Python.

On Databricks, Lakeflow Spark Declarative Pipelines offer the same idea declaratively: `@dp.expect` records violations, `@dp.expect_or_drop` removes violating rows, and `@dp.expect_or_fail` stops the update. Use them when your pipeline runs in that framework. The contract pattern here works in any job.

Why it matters

A contract validator raises on the first failed check. A redelivery has a row count 2% below the minimum, so the run stops at check 1. The supplier fixes the count and resends. The next run stops at check 4, duplicate keys. The third run stops at check 6, a weight total outside the range. Three round trips with the supplier cost four days.

Evaluating every check, storing all results, and then raising once for all CRITICAL failures gives the supplier one complete list on the first day.

Worked example: weak code

```
1  def validate(df, contract):
2      if df.count() < contract["row_count_min"]:
(1)        raise Exception("Too few rows")
4      if df.filter("respondent_id is null").count() > 0:
(2)        raise Exception("Null ids found")
6      if df.select("gender").distinct().count() > 4:
(3)        print("warning: odd genders")
(4)    return "ok"
```

1. **Line 3.** Stops at the first failure, with no numbers. Later checks never run.
2. **Line 5.** A bare Exception with a sentence: nothing to store, count or compare.
3. **Line 7.** A warning that goes to stdout and is lost.
4. **Line 8.** The caller learns nothing about which checks ran.

Worked example: strong code

```
1  from dataclasses import dataclass
2  from typing import Literal
3
(1) Status = Literal["PASS", "WARN", "FAIL", "SKIP"]
5  Severity = Literal["CRITICAL", "HIGH", "MEDIUM", "LOW"]
6
7
8  @dataclass(frozen=True)
(2) class CheckResult:
10     """One contract check outcome, ready to store in the QA results
       table."""
11
12     check_name: str
13     status: Status
```

```
    severity: Severity
    expected: str
    actual: str


class ContractViolation(Exception):
    """Raised when one or more CRITICAL checks fail."""


def critical_failures(results: list[CheckResult]) -> list[CheckResult]:
    """Failed checks with CRITICAL severity."""
    return [r for r in results if r.status == "FAIL" and r.severity == "CRITICAL"]


def enforce(results: list[CheckResult]) -> None:
    """Raise once, listing every CRITICAL failure. WARN results pass through."""
    failures = critical_failures(results)
    if failures:
        names = ", ".join(f"{r.check_name} (expected {r.expected}, got {r.actual})" for r in failures)
        raise ContractViolation(f"{len(failures)} critical check(s) failed: {names}")
```

1. **Line 4.** Typed statuses: a typo such as "FAILED" is a mypy error.
2. **Line 9.** A record with expected and actual values, stored for every check, pass or fail.
3. **Line 19.** A specific exception type the orchestrator can catch, log and re-raise.
4. **Line 28.** Decides after every check has run, and reports all CRITICAL failures at once.

Results are data, severity decides the action, and one run reports every problem.

Recap card

- **What does a contract check produce?** A structured record: check_name, status, severity, expected, actual. Never a sentence.
- **CRITICAL versus WARN** CRITICAL failures stop the pipeline by raising. WARN results are logged and the run continues.
- **Why evaluate every check before raising?** One run reports every problem, so the supplier fixes everything in one round trip.
- **Lakeflow Declarative Pipelines expectations** @dp.expect keeps and records, @dp.expect_or_drop drops, @dp.expect_or_fail stops the update.

Practice

Exercise 1. Architecture design (level 1)

Assign a severity to each contract check.

Options: CRITICAL: stop the pipeline, WARN: log and continue.

1. Duplicate respondent_id values within a wave
2. Weight total outside the contracted range
3. 12 rows with an unexpected gender label
4. Share of 16-34s at 19%, just below the expected 20-35%

Exercise 2. Multiple choice (level 2)

A validator finds three problems in a delivery: a low row count (CRITICAL), null keys (CRITICAL) and odd labels (WARN). What should one run report?

- a) Only the first CRITICAL failure, then stop.
- b) All three results stored, then one exception naming both CRITICAL failures.
- c) Nothing: warnings and failures are only printed.
- d) All three results stored, and the run continues to silver.

Why?

- r1) Evaluate everything, record everything, then act on severity once.
- r2) Exceptions should always be raised as early as possible.

Exercise 3. Fill the gap (level 2)

Type the result fields so mypy rejects any status or severity outside the allowed values.

```
Status = _____(1)["PASS", "WARN", "FAIL", "SKIP"]


@dataclass(_____(2)=True)
class CheckResult:
    check_name: str
    status: Status
```

Exercise 4. Write the function (level 4)

Results arrive as dicts with keys `check_name`, `status`, `severity`, `expected`, `actual`. Write `enforce(results)`:

- Return the list of WARN results.

- Raise `ContractViolation` (define it, subclassing `Exception`) when any result has status `FAIL` and severity `CRITICAL`. The message must name every such check.
- A FAIL with a non-CRITICAL severity does not raise.

Your code must pass these checks:

- With no CRITICAL failure, return the WARN results.
- Raise ContractViolation naming every CRITICAL failure, not only the first.
- A FAIL with MEDIUM severity must not raise.
- ContractViolation must be its own subclass of Exception.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Raised when one or more CRITICAL checks fail.
class ContractViolation(Exception): ...

# Act on contract results.
def enforce(
    results: list[dict[str, str]],
) -> list[dict[str, str]]: ...
```

Starter code:

```python
class ContractViolation(Exception):
    ...


def enforce(results: list[dict[str, str]]) -> list[dict[str, str]]:
    ...
```

Exercise 5. Spot the bug (level 5)

This is the reference `ContractValidationResult.pass_rate`. It fails with a type error the first time it runs. Tap the line that causes it, then fix the imports.

```python
from dataclasses import dataclass, field
from pyspark.sql.functions import col, sum


@dataclass
class ContractValidationResult:
    checks: list = field(default_factory=list)

    @property
    def pass_rate(self) -> float:
        passed = sum(1 for c in self.checks if c.status == "PASS")
        return passed / len(self.checks) if self.checks else 0
```

Write down the line number of the bug. Then write the fix.

Exercise 6. Multiple choice (level 5)

Your silver table is built with Lakeflow Spark Declarative Pipelines. Rows with a NULL `respondent_id` must never reach the table, but the update should keep going for valid rows. Which decorator fits?

- a) `@dp.expect("valid_id", "respondent_id IS NOT NULL")`
- b) `@dp.expect_or_drop("valid_id", "respondent_id IS NOT NULL")`
- c) `@dp.expect_or_fail("valid_id", "respondent_id IS NOT NULL")`
- d) No decorator: filter in the next notebook

Why?

- r1) expect_or_drop removes violating rows and records how many were dropped.
- r2) expect_or_drop fails the update when any row violates.

Sources

- Importing sum from pyspark.sql.functions shadows the built-in, so sum over a generator fails. (verified: Run on PySpark 4.0.4: PySparkTypeError [NOT_COLUMN_OR_STR] Argument `col` should be a Column or str, got generator.)
- Lakeflow Spark Declarative Pipelines provide @dp.expect, @dp.expect_or_drop and @dp.expect_or_fail in pyspark.pipelines. (verified: https://docs.databricks.com/aws/en/ldp/developer/ldp-python-ref-expectations (checked 2026-09-25))

A6-L2 Statistical checks that survive empty, single-row and zero data

Concept

Statistical QA runs on every wave, including the strange ones: an empty delivery, a subgroup with one respondent, a metric that drops to exactly zero. A check that crashes on those cases fails exactly when you need it.

Know what Spark 4.0 returns at the edges (all tested on PySpark 4.0.4):

Case	Result
`F.mean`, `F.stddev`, `percentile_approx` on an empty DataFrame	NULL
`F.stddev` on one row	NULL (it is the sample standard deviation)

Case	Result
`F.col("a") / F.col("b")` where b is 0, ANSI mode on	Error: `DIVIDE_BY_ZERO`, even for DOUBLE
`F.try_divide(a, b)` where b is 0	NULL
Python `round(None, 4)`	`TypeError`

So a weight-spread check built as `round(stats["stddev"] / stats["mean"], 4)` fails three ways: NULL mean on an empty wave, NULL stddev on one row, and `round(None)` in Python.

Four habits make checks robust:

1. **Compute the count with the statistics**, in the same `agg`, and decide early. Fewer than two rows: return a `SKIP` result that says why.
2. **Divide with** `F.try_divide` or guard with `F.when(denominator > 0, ...)`. Never a bare `/` on data you do not control.
3. **Test for None with** `is None` in Python. `if cv:` treats a real CV of 0.0 as missing, and `if current_pct and prior_pct:` skips a share that fell to 0.0 (reference defect 8).
4. **Return a result for every check**, including SKIP. A check that silently returns nothing looks exactly like a check that passed.

Why it matters

A new tracker's first wave has one respondent in a small region, because of a routing test. The weight-spread check computes the sample standard deviation for that region: NULL. Python then runs `round(None, 4)` and raises `TypeError`. The QA job fails, and because QA gates publication, the whole wave's reporting is delayed by a day while someone reads the stack trace.

With the count checked first, that region gets a SKIP result, "fewer than 2 respondents", and every other region is checked as normal.

Worked example: weak code

```
def check_weight_distribution(df, weight_col):
    stats = df.agg(
        F.mean(weight_col).alias("mean"),
        F.stddev(weight_col).alias("stddev"),
    ).collect()[0]
    cv = stats["stddev"] / stats["mean"] if stats["mean"] else None
    return {
        "mean": round(stats["mean"], 4),
        "cv": round(cv, 4) if cv else None,
        "cv_warning": cv > 1.5 if cv else False,
```

```
11      }
```

1. **Line 6.** stddev is NULL for one row, so None / mean raises TypeError.
2. **Line 8.** An empty frame gives mean None, and round(None) raises TypeError.
3. **Line 10.** Truthiness: a missing CV reports no warning, as if the check had passed.

Worked example: strong code

```
1  MIN_ROWS = 2
2  MAX_WEIGHT_CV = 1.5
3
4
5  def weight_spread(df: DataFrame, weight_col: str) -> dict[str, float |
     int | None]:
6      """Count, mean, stddev and CV of weights in one pass. CV is NULL when
     undefined."""
7      row = df.agg(
(1)        F.count(weight_col).alias("n"),
9          F.mean(weight_col).alias("mean"),
10         F.stddev(weight_col).alias("stddev"),
(2)        F.try_divide(F.stddev(weight_col),
     F.mean(weight_col)).alias("cv"),
12     ).collect()[0]
13     return row.asDict()
14
15
16 def check_weight_spread(stats: dict[str, float | int | None]) ->
     CheckResult:
17     """PASS, WARN, or SKIP with a reason. Never raises on edge cases."""
(3)    if stats["n"] < MIN_ROWS or stats["cv"] is None:
19         return CheckResult("weight_cv", "SKIP", "MEDIUM", f">= {MIN_ROWS}
     rows", f"n={stats['n']}")
(4)    status = "WARN" if stats["cv"] > MAX_WEIGHT_CV else "PASS"
21     return CheckResult("weight_cv", status, "MEDIUM", f"<=
     {MAX_WEIGHT_CV}", f"{stats['cv']:.4f}")
```

1. **Line 8.** The count travels with the statistics, in the same pass.
2. **Line 11.** CV computed in Spark. try_divide returns NULL for a zero or NULL mean instead of raising.
3. **Line 18.** Edge cases become an explicit SKIP with the reason, checked with `is None`.
4. **Line 20.** Only reached with a real number, so the comparison is safe.

The strong version returns a result for every wave and every subgroup, and says why when a statistic is undefined.

Recap card

- **F.stddev on one row, and mean on an empty frame** Both NULL. stddev is the sample standard deviation.
- **Division by zero on Spark 4.0 with ANSI on** Raises DIVIDE_BY_ZERO, even for DOUBLE. Use F.try_divide or F.when(b > 0, a / b).
- **What should a check return when a statistic is undefined?** An explicit SKIP result with the reason. Never nothing, and never a crash.
- **Why is `if cv:` wrong?** A real CV of 0.0 is falsy. Test missing values with `is None`.

Practice

Exercise 1. Predict the output (level 1)

A region has exactly one respondent. What does this print?

```
one = spark.createDataFrame([(1.2,)], "final_weight DOUBLE")
print(one.agg(F.stddev("final_weight")).collect()[0][0])
```

Exercise 2. Multiple choice (level 2)

On Spark 4.0 (Runtime 17.3 LTS), `F.col("weighted_reach") / F.col("weighted_base")` meets a row where `weighted_base` is `0.0`. What happens?

- a) The result is Infinity.
- b) The result is NULL.
- c) The query fails with DIVIDE_BY_ZERO.
- d) The result is 0.0.

Why?

- r1) ANSI mode is on by default in Spark 4.0, and it raises on division by zero, DOUBLE included.
- r2) Delta rejects infinite values.

Exercise 3. Fill the gap (level 3)

Compute the coefficient of variation in Spark so that a zero or NULL mean gives NULL instead of an error.

```
cv = F._____(1)(F.stddev("final_weight"), F._____(2)("final_weight"))
```

Exercise 4. Write the function (level 4)

Write `check_weight_spread(stats, max_cv=1.5)`. `stats` is a dict with `n` (int), `cv` (float or None). Return a tuple `(status, reason)`:

- `("SKIP", "fewer than 2 rows")` when `n < 2`.
- `("SKIP", "cv undefined")` when `cv` is None.
- `("WARN", ...)` when `cv > max_cv`, otherwise `("PASS", ...)`. A CV of `0.0` is a real value and passes.

Your code must pass these checks:

- n below 2 must return ("SKIP", "fewer than 2 rows").
- A None CV with enough rows must return ("SKIP", "cv undefined").
- A CV of 0.0 means all weights are equal. It is a real value and passes.
- Above max_cv must WARN. Use the max_cv argument, not a fixed number.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Classify the weight coefficient of variation.
def check_weight_spread(
    stats: dict[str, float | int | None],
    max_cv: float = 1.5,
) -> tuple[str, str]: ...
```

Starter code:

```
def check_weight_spread(stats: dict[str, float | int | None], max_cv:
    float = 1.5) -> tuple[str, str]:
    ...
```

Exercise 5. Spot the bug (level 5)

This is the reference wave-stability comparison. A share that dropped from 12% to 0% produced no result at all. Tap the line, then fix the function.

```
def compare_share(metric: str, current_pct: float | None, prior_pct:
    float | None, tolerance: float = 0.10) -> dict[str, object] | None:
    """Return a stability result, or None when a share is missing."""
    if current_pct and prior_pct:
        change = abs(current_pct - prior_pct)
        return {"metric": metric, "change_pp": round(change, 4),
    "status": "WARN" if change > tolerance else "PASS"}
    return None
```

Write down the line number of the bug. Then write the fix.

Sources

- On PySpark 4.0.4: stddev of one row and mean/stddev/percentile_approx of an empty frame are NULL; division by zero raises DIVIDE_BY_ZERO for DOUBLE under ANSI; try_divide returns NULL; Python round(None) raises TypeError. (verified: Run in this repository.)

A6-L3 Reconcile counts and totals between layers

Concept

Each layer transforms data, and each transformation can lose or duplicate rows. **Reconciliation** checks that what came out of one layer matches what went into the next, for every wave you load.

Compare quantities that must agree by design:

From	To	Must match
Bronze, wave W	Silver, wave W	Distinct `(respondent_id, wave_id)` keys in bronze = silver rows
Silver, wave W, in scope	Gold, wave W	Sum of `final_weight` = sum of `weighted_base` over all subgroups
Silver, wave W, in scope	Gold, wave W	Row count = sum of `unweighted_n`

Pick the right bronze figure. Bronze holds every delivery, redeliveries included, so its raw row count is larger than silver's by design. Compare **distinct keys**.

Compare totals with a **tolerance**. Floating-point sums depend on the order Spark adds values in, which varies with partitioning, so two correct totals can differ in the last digits. In Python, `0.1 + 0.2 == 0.3` is `False`. Use `math.isclose(a, b, rel_tol=1e-9, abs_tol=1e-6)`. The `abs_tol` handles totals near zero, where a relative tolerance means nothing.

Compute each side with one aggregation, record the result as a structured check (expected, actual, status), and treat a failed reconciliation as **CRITICAL**. If gold does not add up to silver, every published figure for that wave is suspect.

Run reconciliation after each layer loads and before anything downstream reads it. Store the results with the run log, so you can show that every published wave reconciled.

Why it matters

A gold job joins silver to a region lookup with an inner join. A supplier introduces a new region code, `NI`, that the lookup does not have. 312 respondents drop out of gold without an error. Weighted reach for the wave looks plausible, because the base shrank along with the reach.

A reconciliation of silver's in-scope weight total against the sum of gold's `weighted_base` catches the 1.8% gap on the same run. The alert names the wave and the size of the difference, and the fix is a lookup update, not a client complaint.

Worked example: weak code

```
def reconcile(bronze, silver, wave):
    b = bronze.filter(F.col("wave_id") == wave).count()
    s = silver.filter(F.col("wave_id") == wave).count()
    if b != s:
        print(f"mismatch {b} vs {s}")
```

1. **Line 2.** Counts every delivery, redeliveries included. A redelivered wave always mismatches.
2. **Line 4.** Exact equality: fine for counts, wrong for weight totals.
3. **Line 5.** A print that nobody reads, and the pipeline carries on.

Worked example: strong code

```
import math


def silver_totals(silver: DataFrame, wave_id: str) -> tuple[int, float]:
    """In-scope row count and weight total for one wave, in one pass."""
    row = (
        silver.filter((F.col("wave_id") == wave_id) & F.col("in_scope"))
        .agg(F.count("*").alias("n"), F.coalesce(F.sum("final_weight"),
    F.lit(0.0)).alias("w"))
        .collect()[0]
    )
    return int(row["n"]), float(row["w"])


def gold_totals(gold: DataFrame, wave_id: str) -> tuple[int, float]:
    """Sum of unweighted_n and weighted_base over every subgroup of one
    wave."""
    row = (
        gold.filter(F.col("wave_id") == wave_id)
        .agg(F.coalesce(F.sum("unweighted_n"), F.lit(0)).alias("n"),
    F.coalesce(F.sum("weighted_base"), F.lit(0.0)).alias("w"))
        .collect()[0]
    )
```

```
    return int(row["n"]), float(row["w"])


def totals_match(expected: float, actual: float) -> bool:
    """Equal within floating-point tolerance, including totals near zero."""
    return math.isclose(expected, actual, rel_tol=1e-9, abs_tol=1e-6)
```

1. **Line 7.** Same population as gold: one wave, in scope only.
2. **Line 8.** Count and total in one pass. Coalesce so an empty wave gives 0, not NULL.
3. **Line 26.** Tolerant comparison for floating-point totals.

Silver and gold totals are computed the same way and compared with tolerance. A mismatch becomes a CRITICAL result with numbers, not a print.

Recap card

- **Bronze figure that must match silver's row count for a wave** Distinct (respondent_id, wave_id) keys, not raw bronze rows, because bronze keeps every delivery.
- **Silver-to-gold reconciliation** In-scope silver weight total = sum of gold weighted_base; in-scope silver rows = sum of unweighted_n.
- **How to compare floating-point totals** math.isclose(a, b, rel_tol=1e-9, abs_tol=1e-6). Never ==.
- **Severity of a failed reconciliation** CRITICAL. Every figure for the wave is suspect until it passes.

Practice

Exercise 1. Multiple choice (level 1)

Wave W14 was delivered twice to bronze. What should the silver row count for W14 equal?

- a) The bronze row count for W14
- b) The number of distinct (respondent_id, wave_id) keys in bronze for W14
- c) Half the bronze row count for W14
- d) The row count of the latest delivery only

Why?

- r1) Silver keeps one row per key, the newest version, so it matches the number of distinct keys across all deliveries.
- r2) Bronze deletes old deliveries after silver loads.

Exercise 2. Predict the output (level 2)

What does this print?

```
import math

print(0.1 + 0.2 == 0.3, math.isclose(0.1 + 0.2, 0.3))
```

Exercise 3. Write the function (level 3)

Write `reconciliation_result(name, expected, actual)` that returns a dict with `check_name`, `status` (`"PASS"` or `"FAIL"`), `severity` (always `"CRITICAL"`), `expected` and `actual` (both formatted to 2 decimal places as strings), and `difference` (actual minus expected, a float). Compare with `math.isclose(expected, actual, rel_tol=1e-9, abs_tol=1e-6)`.

Your code must pass these checks:

- 0.3 and 0.1 + 0.2 must PASS. Use math.isclose.
- A 900,000 gap must FAIL, with difference = actual - expected.
- Totals near zero need abs_tol, or 0.0 against 1e-9 fails.
- Format expected and actual to two decimal places, and keep the check name.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Compare two layer totals and return a CRITICAL check result.
def reconciliation_result(
    name: str,
    expected: float,
    actual: float,
) -> dict[str, object]: ...
```

Starter code:

```
import math


def reconciliation_result(name: str, expected: float, actual: float) -> dict[str, object]:
    ...
```

Exercise 4. Spot the bug (level 4)

This bronze-to-silver reconciliation fails every time a wave is redelivered, even when silver is correct. Tap the line, then fix the function.

```
def bronze_silver_keys(bronze: DataFrame, silver: DataFrame, wave_id: str) -> tuple[int, int]:
    """Keys in bronze and rows in silver for one wave."""
    b = bronze.filter(F.col("wave_id") == wave_id).count()
```

```
    s = silver.filter(F.col("wave_id") == wave_id).count()
    return b, s
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Put in order (level 5)

Order the steps for one wave, from load to publication.

- MERGE the wave into silver
- Store reconciliation results with the run log
- Reconcile silver in-scope totals with gold totals
- Refresh the Power BI semantic model
- Build gold metrics for the wave
- Reconcile bronze distinct keys with silver rows

Practical: Reconcile silver with gold, and catch a dropped region

Platform: Databricks Free Edition. Run this in Databricks Free Edition. It builds a small silver table and two gold versions: one correct, one built with an inner join that drops an unknown region. The reconciliation passes for the first and fails for the second.

Download the notebook `A6-L3.ipynb` from the app.

Sources

- 0.1 + 0.2 == 0.3 is False and math.isclose(0.1 + 0.2, 0.3) is True in Python. (verified: Run in Python 3.11 in this repository.)
- F.count_distinct exists in PySpark 4.0. (verified: Run on PySpark 4.0.4 in this repository.)

A7 Observability

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Structured logging and pipeline run log tables.

A7-L1 Structured logging with the logging module, not print

Concept

`print` writes text to standard output, with no level, no source and no context. In a scheduled job, that text lands in a driver log that nobody searches until something breaks.

Python's standard `logging` module fixes all three problems.

One logger per module. `log = logging.getLogger(__name__)` names the logger after the module, such as `pipelines.silver`. Every line then says where it came from.

Levels. `DEBUG` for detail while developing, `INFO` for normal milestones, `WARNING` for something odd that did not stop the run, `ERROR` for a failure, `CRITICAL` for a failure that stops everything. Loggers pass `WARNING` and above by default, so `log.info(...)` prints nothing until you configure a level.

Context on every line. Include the identifiers you will search for later: `run_id`, `tracker_id`, `wave_id`, `layer`. A `LoggerAdapter` attaches them once, and a format string prints them:

```
1 base = logging.getLogger(__name__)
2 log = logging.LoggerAdapter(base, {"run_id": run_id, "wave_id": wave_id})
3 log.info("merged rows=%d", n)
4 # INFO pipelines.silver run_id=7f3c wave_id=W14 merged rows=1832
```

Three more rules:

- Pass values as arguments, `log.info("rows=%d", n)`, not pre-built f-strings. The logger formats them only when the line is actually emitted.
- Inside `except`, call `log.exception("...")`. It logs at ERROR and appends the full traceback.
- Configure handlers and formats once, in the entry point, never inside library functions.

Logs are for people reading one run. Metrics you want to compare across runs, such as row counts and statuses, also go to the run log table (lesson A7-L2).

Why it matters

A silver job prints "done" at the end of each step. One night it finishes "successfully" with zero rows merged, because an upstream filter used the wrong wave ID. The driver log for that run holds 400 lines of "done" and Spark noise, with no wave ID and no row counts.

With `log.info("merged rows=%d", n)` and the wave ID on every line, the zero is visible in one search, and a WARNING for an empty merge would have flagged it at 02:00.

Worked example: weak code

```
1  def merge_wave(df, wave_id):
2      print("starting")
3      try:
4          n = do_merge(df)
5          print(f"done {n}")
6      except Exception as e:
7          print("error:", e)
```

1. **Line 2.** No level, no module, no wave ID: unsearchable.
2. **Line 5.** A count with no label. Zero rows looks the same as a normal run.
3. **Line 7.** Swallows the exception and loses the traceback. The job reports success.

Worked example: strong code

```
1  import logging
2
3  logger = logging.getLogger(__name__)
4
5
6  def merge_wave(df: DataFrame, run_id: str, wave_id: str) -> int:
7      """MERGE one wave, logging with run and wave context. Re-raises on failure."""
8      log = logging.LoggerAdapter(logger, {"run_id": run_id, "wave_id": wave_id})
9      log.info("merge started")
10     try:
11         rows = do_merge(df)
12     except Exception:
13         log.exception("merge failed")
14         raise
15     if rows == 0:
16         log.warning("merge wrote 0 rows")
```

```
    else:
        log.info("merge finished rows=%d", rows)
    return rows
```

1. **Line 3.** One logger per module, named after it.
2. **Line 8.** Context attached once, printed on every line by the entry point's format.
3. **Line 13.** ERROR level plus the full traceback.
4. **Line 14.** Re-raise, so the job fails and alerts fire.
5. **Line 16.** A suspicious result gets its own level, not a buried number.

Every line says what, where, which run and which wave. Failures keep their traceback and still fail the job.

Recap card

- **Create a module logger** logger = logging.getLogger(**name**)
- **Default level threshold with no configuration** WARNING. log.info(...) prints nothing until the entry point sets a level.
- **Attach run_id and wave_id to every line** logging.LoggerAdapter(logger, {"run_id": ..., "wave_id": ...})
- **Inside except** log.exception("...") for ERROR plus traceback, then a bare raise.

Practice

Exercise 1. Predict the output (level 1)

No logging configuration has run. What appears in the output?

```
import logging

log = logging.getLogger("silver")
log.info("loaded")
log.warning("3 unexpected labels")
```

Exercise 2. Fill the gap (level 2)

Create a module logger, then log the failure with its traceback and re-raise.

```
logger = logging.getLogger(_____(1))

try:
    rows = do_merge(df)
except Exception:
    logger._____(2)("merge failed")
    _____(3)
```

Exercise 3. Spot the bug (level 3)

This job failed at the MERGE last night, yet the scheduler marked it as succeeded and nobody was alerted. Tap the line, then fix the function.

```
1 def merge_wave(df, run_id: str, wave_id: str) -> int:
2     """MERGE one wave. Logs failures."""
3     log = logging.LoggerAdapter(logger, {"run_id": run_id, "wave_id":
    wave_id})
4     try:
5         return do_merge(df)
6     except Exception:
7         log.exception("merge failed")
8         return 0
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 4)

Write `wave_logger(run_id, wave_id)` that returns a `logging.LoggerAdapter` around the module logger `logging.getLogger("pipelines.silver")`, carrying `run_id` and `wave_id`. Then write `log_merge_result(log, rows)`: log at WARNING `"merge wrote 0 rows"` when rows is 0, otherwise at INFO `"merge finished rows=%d"` with the count passed as an argument.

Your code must pass these checks:

- The INFO line must carry run_id and wave_id and read `merge finished rows=5`.
- Zero rows must log at WARNING: `merge wrote 0 rows`.
- Wrap the logger named `pipelines.silver`.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
1  # Module logger with run and wave context on every line.
2  def wave_logger(
3      run_id: str,
4      wave_id: str,
5  ) -> logging.LoggerAdapter: ...
6
7  # INFO for a normal merge, WARNING for an empty one.
8  def log_merge_result(
9      log: logging.LoggerAdapter,
10     rows: int,
11 ) -> None: ...
```

Starter code:

```
1 import logging
2
3
```

```python
def wave_logger(run_id: str, wave_id: str) -> logging.LoggerAdapter:
    ...


def log_merge_result(log: logging.LoggerAdapter, rows: int) -> None:
    ...
```

Exercise 5. Architecture design (level 5)

Choose the level for each event in a silver job.

Options: DEBUG, INFO, WARNING, ERROR.

1. Run started for wave W14
2. MERGE wrote 0 rows for a wave that should have data
3. MERGE raised DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE
4. The full query plan of the MERGE source, for troubleshooting

Sources

- Without configuration, loggers pass WARNING and above, and the last-resort handler prints only the message; LoggerAdapter adds context; logger.exception includes the traceback. (verified: Run in Python 3.11 in this repository.)

A7-L2 A pipeline run log table with an explicit schema

Concept

Logs describe one run. A **run log table** lets you compare every run: which waves loaded, how many rows, how long, and which failed. It is a Delta table, append-only, with one row per pipeline run per layer.

A useful schema:

Column	Type	Notes
`run_id`	STRING, not null	`str(uuid.uuid4())`, shared by every log line of the run
`pipeline_name`	STRING, not null	`nat_silver_pipeline`
`tracker_id`, `wave_id`, `layer`	STRING	what the run touched
`rows_processed`	BIGINT, not null	from Delta commit metrics
`status`	STRING, not null	`SUCCESS` or `FAILED`

Column	Type	Notes
`error_message`	STRING, nullable	NULL on success
`started_at`, `finished_at`	TIMESTAMP, not null	UTC

Build the record against an **explicit** `StructType`. Your reference `log_pipeline_run` builds a DataFrame from a dict and lets Spark infer types. That fails on every successful run: `error_message` is `None`, and Spark raises `CANNOT_DETERMINE_TYPE` because it cannot infer a type from a column of only nulls. The explicit schema also rejects a wrong type, such as `"12"` for `rows_processed`, before anything is written.

Take timestamps with `datetime.now(timezone.utc)`. `datetime.utcnow()` is deprecated from Python 3.12 and returns a naive datetime that carries no timezone.

Import everything you use. The reference function calls `uuid` and `datetime` without importing them, a `NameError` on its first run. `ruff check` (rule F821) catches it in CI.

Write the log record in a `finally` block, or in both the success and failure paths, so failed runs are logged too. A run log that only records successes cannot tell you how often the pipeline fails.

Why it matters

The reference `log_pipeline_run` has never succeeded. It fails first with `NameError: name 'uuid' is not defined`. Once someone adds the imports, it fails on every successful run with `CANNOT_DETERMINE_TYPE`, because `error_message=None` is the only value in its column. The orchestrator catches that error "so logging never breaks the pipeline". Months later the team wants to show a client every load of 2025. The run log table is empty.

An explicit schema, imported names and a test that writes one record on a local SparkSession prevent all three problems.

Worked example: weak code

```
1 def log_pipeline_run(pipeline_name, tracker_id, wave_id, layer,
      rows_processed, status, error_message: str = None):
2     run_record = spark.createDataFrame([{
3         "run_id": str(uuid.uuid4()),
4         "pipeline_name": pipeline_name,
5         "rows_processed": rows_processed,
6         "status": status,
```

```
7           "error_message": error_message,
4           "run_timestamp": datetime.utcnow().isoformat(),
9       }])
10
    run_record.write.format("delta").mode("append").saveAsTable("research_governance.
```

1. **Line 1.** Claims a string, defaults to None (defect 7).
2. **Line 2.** Types inferred from one dict. An all-None column raises CANNOT_DETERMINE_TYPE.
3. **Line 3.** uuid is never imported: NameError (defect 1).
4. **Line 8.** datetime not imported, utcnow deprecated (defect 6), and the timestamp stored as a string.

Worked example: strong code

```
1  import uuid
2  from datetime import datetime, timezone
3
4  from pyspark.sql import DataFrame, SparkSession
5  from pyspark.sql import types as T
6
1  RUN_LOG_SCHEMA = T.StructType([
8      T.StructField("run_id", T.StringType(), False),
9      T.StructField("pipeline_name", T.StringType(), False),
10     T.StructField("wave_id", T.StringType(), True),
11     T.StructField("rows_processed", T.LongType(), False),
12     T.StructField("status", T.StringType(), False),
2      T.StructField("error_message", T.StringType(), True),
14     T.StructField("started_at", T.TimestampType(), False),
15     T.StructField("finished_at", T.TimestampType(), False),
16 ])
17
18
19 def new_run_id() -> str:
20     """A unique id shared by the log lines and the run log row of one
       run."""
21     return str(uuid.uuid4())
22
23
24 def run_record(
25     spark: SparkSession,
26     run_id: str,
27     pipeline_name: str,
28     wave_id: str | None,
29     rows_processed: int,
30     status: str,
31     started_at: datetime,
32     error_message: str | None = None,
33 ) -> DataFrame:
34     """One run log row, typed by RUN_LOG_SCHEMA."""
3      finished_at = datetime.now(timezone.utc)
```

```
36      row = (run_id, pipeline_name, wave_id, rows_processed, status,
        error_message, started_at, finished_at)
4       return spark.createDataFrame([row], RUN_LOG_SCHEMA)
```

1. **Line 7.** The schema is declared once. Nothing is inferred.
2. **Line 13.** Nullable string: None is fine, because the type is declared.
3. **Line 35.** Timezone-aware UTC, stored as a real TIMESTAMP.
4. **Line 37.** A wrong type, such as "12" for rows_processed, fails here with a clear message.

The strong record builds on every run, success or failure, and its types never depend on the values in one row.

Recap card

- **Why does the reference log_pipeline_run fail on success?** error_message=None is inferred with no type: CANNOT_DETERMINE_TYPE. Declare a StructType.
- **UTC timestamp in modern Python** datetime.now(timezone.utc). utcnow() is deprecated from 3.12 and naive.
- **Which ruff rule catches the missing uuid import?** F821, undefined name.
- **Where do you write the run record so failures are logged too?** In a finally block, after setting status in try and except.

Practice

Exercise 1. Multiple choice (level 1)

`spark.createDataFrame([{"run_id": "r1", "error_message": None}])` runs on PySpark 4.0. What happens?

- a) error_message becomes a STRING column holding NULL.
- b) It fails with CANNOT_DETERMINE_TYPE.
- c) error_message is dropped from the DataFrame.
- d) error_message becomes a NULL-typed column that Delta accepts.

Why?

- r1) Spark infers each column's type from the data, and a column holding only None gives it nothing to infer from.
- r2) Spark never allows None in a DataFrame.

Exercise 2. Fill the gap (level 2)

Complete the imports and the timestamp so the record code runs on Python 3.12 without deprecation warnings.

```
import _____(1)
from datetime import datetime, _____(2)

run_id = str(uuid.uuid4())
started_at = datetime._____(3)(timezone.utc)
```

Exercise 3. Spot the bug (level 3)

This builds a run log row. It fails on every successful run. Tap the line, then fix the function so it declares its schema.

```
def run_record(spark: SparkSession, run_id: str, rows: int, status: str,
    error_message: str | None) -> DataFrame:
    """One run log row."""
    return spark.createDataFrame([{
        "run_id": run_id,
        "rows_processed": rows,
        "status": status,
        "error_message": error_message,
    }])
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 4)

Write `run_record(spark, run_id, pipeline_name, rows_processed, status, started_at, error_message=None)` returning a one-row DataFrame typed by a module constant `RUN_LOG_SCHEMA` with columns `run_id`, `pipeline_name`, `rows_processed` (LongType), `status`, `error_message` (nullable), `started_at` and `finished_at` (TimestampType). Set `finished_at` with `datetime.now(timezone.utc)`.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
RUN_LOG_SCHEMA = ...

# One run log row typed by RUN_LOG_SCHEMA.
def run_record(
    spark: SparkSession,
    run_id: str,
    pipeline_name: str,
    rows_processed: int,
    status: str,
    started_at: datetime,
    error_message: str | None = None,
```

```
) -> DataFrame: ...
```

Starter code:

```
from datetime import datetime, timezone

from pyspark.sql import DataFrame, SparkSession
from pyspark.sql import types as T

RUN_LOG_SCHEMA = ...


def run_record(spark: SparkSession, run_id: str, pipeline_name: str,
    rows_processed: int, status: str, started_at: datetime, error_message:
    str | None = None) -> DataFrame:
      ...
```

Exercise 5. Put in order (level 5)

Order the orchestrator's steps so both successful and failed runs are logged, and a failure still fails the job.

- except: set status FAILED and error_message, log.exception, then raise
- Create run_id and started_at
- finally: append the run record to the run log table
- Set status SUCCESS
- try: run the layer and read rows_processed from the commit metrics

Practical: Write run log records for a successful and a failed run

Platform: Databricks Free Edition. Run this in Databricks Free Edition. It creates a run log table with an explicit schema, runs one step that succeeds and one that fails, and records both. It also shows the inference error the reference code hits.

Download the notebook `A7-L2.ipynb` from the app.

Sources

- createDataFrame from a dict with an all-None column raises CANNOT_DETERMINE_TYPE; an explicit StructType accepts None and rejects '12' for LongType with FIELD_DATA_TYPE_UNACCEPTABLE_WITH_NAME. (verified: Run on PySpark 4.0.4 in this repository.)
- datetime.utcnow() is deprecated from Python 3.12. (verified: Python 3.12 release notes (What's New in Python 3.12, deprecated APIs); CLAUDE.md standard.)

A7-L3 Alert on failures, and on the silence that hides them

Concept

An alert is useful when it fires for real problems, reaches the person who can act, and says enough to start. Pipelines need two kinds.

1. Loud failures: the job fails. Databricks job notifications send an email or a webhook message when a run fails. In a job definition or bundle:

```
email_notifications:
  on_failure:
    - data-oncall@example.org
```

These fire only if the job actually fails. Every `except` block that logs and carries on silences them. Log, record the run as FAILED, then re-raise (lesson A7-L2).

Job-level notifications are not sent when a failed task is retried. If you want a message for every failed attempt, add task-level notifications.

2. Silent failures: the job succeeds, but the data is wrong or missing. A job that never starts, a MERGE that writes 0 rows, a wave with a WARN on every contract check. No job fails, so no notification fires. Detect these with queries on your run log and QA results tables, run on a schedule, for example as Databricks SQL alerts:

- No SUCCESS run for a pipeline in the last 26 hours (a daily job plus slack).
- A run with `rows_processed = 0`.
- More WARN results for a wave than the last five waves' average.

Write each alert query against the exact values your tables hold. A query for `status = 'FAIL'` never fires on a table that stores `FAILED`.

Send alerts to a group address or channel, never to one person's inbox. Include the pipeline, the wave, the run ID and a link, so the reader can start without searching.

Why it matters

A nightly bronze job's schedule is paused during a workspace migration and never resumed. The job does not fail, because it never runs, so no failure email arrives. Silver and gold keep succeeding on the old data. Eleven days later a client asks why the dashboard has not moved.

A scheduled query, "no SUCCESS for nat_bronze_pipeline in 26 hours", would have alerted the on-call group on the second morning.

Worked example: weak code

```
1  -- Alert: failed runs today
2  SELECT COUNT(*) AS failures
3  FROM research_governance.pipeline_run_log
4  WHERE status = 'FAIL'
5    AND run_timestamp >= current_date();
```

1. **Line 2.** Counts failures only. A job that stopped running produces no rows and no alert.
2. **Line 4.** The table stores FAILED. This alert has never fired.

Worked example: strong code

```
1  -- Alert 1: pipelines with no successful run in the last 26 hours.
2  SELECT pipeline_name,
3         max(finished_at) AS last_success
4  FROM research_prod.research_governance.pipeline_run_log
5  WHERE status = 'SUCCESS'
6  GROUP BY pipeline_name
7  HAVING max(finished_at) < current_timestamp() - INTERVAL 26 HOURS;
8
9  -- Alert 2: successful runs that wrote nothing in the last day.
10 SELECT pipeline_name, wave_id, run_id
11 FROM research_prod.research_governance.pipeline_run_log
12 WHERE status = 'SUCCESS'
13   AND rows_processed = 0
14   AND finished_at >= current_timestamp() - INTERVAL 1 DAY;
```

1. **Line 7.** Detects silence: a pipeline whose last success is too old, whether it failed or never ran.
2. **Line 10.** The alert names what to look at: pipeline, wave and run.
3. **Line 13.** A success that did nothing is a silent failure worth an alert.

Job notifications catch runs that fail. Scheduled queries catch runs that do not happen, or that succeed at doing nothing.

Recap card

- **Why must an except block re-raise?** Otherwise the run succeeds and on_failure notifications never fire.
- **Job-level versus task-level failure notifications** Job-level notifications are not sent for retried task failures. Use task-level ones to hear about every failed attempt.
- **How do you detect a pipeline that stopped running?** A scheduled query: no SUCCESS in the run log within the expected window.
- **Where should alerts go?** A group address or channel, with pipeline, wave, run ID and a link.

Practice

Exercise 1. Multiple choice (level 1)

A job has `email_notifications.on_failure` configured. Its silver task catches every exception, logs it, and returns normally. The MERGE fails. What arrives in the on-call inbox?

- a) A failure email.
- b) Nothing.
- c) A warning email.
- d) A success email.

Why?

- r1) The task returns normally, so the run succeeds and on_failure never fires.
- r2) Notifications only work for SQL tasks.

Exercise 2. Fill the gap (level 2)

Complete the job settings so a failed run emails the on-call group.

```
email_notifications:
  _____(1):
    - _____(2)
```

Exercise 3. Spot the bug (level 3)

The run log stores status values `SUCCESS` and `FAILED`. This alert has never fired, even on nights with failures. Tap the line, then fix the query.

```
SELECT pipeline_name, run_id, error_message
FROM research_prod.research_governance.pipeline_run_log
WHERE status = 'FAIL'
  AND finished_at >= current_timestamp() - INTERVAL 1 DAY;
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 4)

Write `stale_pipelines(runs, now, max_age_hours=26)`. `runs` is a list of dicts with `pipeline_name`, `status` and `finished_at` (timezone-aware datetime). Return the sorted names of pipelines whose most recent SUCCESS is older than `max_age_hours` before `now`. A pipeline with runs but no SUCCESS at all is stale too.

What you need to know.

A `timedelta` is a length of time. Subtract one from a `datetime` to get an earlier moment, then compare datetimes with `<` and `>`:

```
from datetime import UTC, datetime, timedelta

now = datetime(2026, 3, 10, 9, 0, tzinfo=UTC)
week_ago = now - timedelta(days=7)        # 2026-03-03 09:00 UTC
datetime(2026, 3, 1, tzinfo=UTC) < week_ago   # True: older than a week
```

`d.setdefault(key, value)` adds `key` with `value` only when the key is missing, and returns what is stored.

Your code must pass these checks:

- A success 5 hours ago is fresh.
- A last success 30 hours ago is stale with the 26-hour limit.
- A pipeline with only FAILED runs has no success at all, so it is stale.
- Use the most recent SUCCESS. A later FAILED does not make a recent success stale.
- Use the max_age_hours argument and return names sorted.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Pipelines whose latest SUCCESS is older than max_age_hours, or that never succeeded.
def stale_pipelines(
    runs: list[dict[str, object]],
    now: datetime,
    max_age_hours: int = 26,
) -> list[str]: ...
```

Starter code:

```
from datetime import datetime, timedelta


def stale_pipelines(runs: list[dict[str, object]], now: datetime, max_age_hours: int = 26) -> list[str]:
    ...
```

Exercise 5. Architecture design (level 5)

Choose how each problem should be detected.

Options: Job failure notification, Scheduled query on the run log or QA tables, No alert needed.

1. The silver MERGE raises an exception
2. The bronze job's schedule was paused and nothing has run for two days
3. Gold succeeds but writes 0 rows for a new wave

4. A run took 9 minutes instead of the usual 8

Sources

- Jobs support email_notifications and webhook_notifications with on_start, on_success and on_failure; job-level notifications are not sent when failed tasks are retried; at most 3 destinations per on_failure. (verified: https://docs.databricks.com/aws/en/jobs/notifications (checked 2026-09-25))
- Databricks SQL alerts run a query on a schedule and notify when a condition is met. (verified: https://docs.databricks.com/aws/en/sql/user/alerts/ (checked 2026-09-25))

A8 Incremental processing

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Auto Loader, change data feed and processing only new waves.

A8-L1 Process only the waves you have not processed, correctly

Concept

Incremental loading means each run handles only new work: waves in bronze that silver does not have yet. It saves compute, and it keeps reruns cheap. It is also easy to get subtly wrong.

The pattern:

```
1 def unprocessed_waves(spark, bronze, silver, tracker_id) -> list[str]:
2     source = spark.table(bronze).filter(F.col("tracker_id") ==
    tracker_id).select("wave_id").distinct()
3     done = spark.table(silver).filter(F.col("tracker_id") ==
    tracker_id).select("wave_id").distinct()
4     return sorted(r.wave_id for r in source.join(done, "wave_id",
    "left_anti").collect())
```

Four rules:

1. **Filter both sides the same way.** Your reference `get_unprocessed_waves` filters bronze by tracker but reads silver for every tracker. If tracker B already has wave `W14` in silver, tracker A's `W14` is treated as done and never loads. Scope both sides with the same keys.
2. **Use a left anti join** (`"left_anti"`): rows on the left with no match on the right. It says exactly what you mean, and it works for multi-column keys.
3. **Handle the first run.** When silver does not exist yet, every bronze wave is new. Check with `spark.catalog.tableExists(silver)`.
4. **Keep each wave's load idempotent.** Detection can be wrong after a partial failure, for example if silver has half a wave. Because the silver MERGE is idempotent (A3-L1), reprocessing a wave is always safe. Design for "process again", not "process exactly once".

Deciding by "waves not in silver" misses redeliveries of a wave silver already has. Either compare the latest `ingestion_timestamp` per wave in bronze with the one recorded when silver loaded it, or read bronze incrementally with change data feed (A8-L3) or Auto Loader (A8-L2).

Why it matters

Two trackers share one silver table. Tracker B loads wave `W14` on Monday. Tracker A's `W14` arrives on Tuesday. The reference `get_unprocessed_waves` compares tracker A's bronze waves against silver waves for all trackers, finds `W14` already present, and skips it. Tracker A's dashboard shows no `W14`, and no job fails. The gap is found nine days later, by a client.

Filtering silver by `tracker_id` too makes the comparison correct, and a reconciliation of bronze keys against silver rows (A6-L3) would have flagged it on Tuesday.

Worked example: weak code

```
1  def get_unprocessed_waves(bronze_table, silver_table, tracker_id):
2      bronze_waves = (
3          spark.table(bronze_table)
4          .filter(col("tracker_id") == tracker_id)
5          .select("wave_id").distinct()
6      )
2      silver_waves = spark.table(silver_table).select("wave_id").distinct()
3      return [row.wave_id for row in
       bronze_waves.subtract(silver_waves).collect()]
```

1. **Line 1.** Uses a global spark and has no type hints.
2. **Line 7.** No tracker filter: another tracker's wave counts as done (reference defect 10).
3. **Line 8.** Unsorted output, and it fails if silver does not exist yet.

Worked example: strong code

```
1  def unprocessed_waves(spark: SparkSession, bronze: str, silver: str,
       tracker_id: str) -> list[str]:
2      """Wave ids in bronze for this tracker that silver does not hold for
       this tracker."""
3      source = (
4          spark.table(bronze)
5          .filter(F.col("tracker_id") == tracker_id)
6          .select("wave_id")
7          .distinct()
8      )
1      if not spark.catalog.tableExists(silver):
10         return sorted(r.wave_id for r in source.collect())
11     done = (
2          spark.table(silver)
13         .filter(F.col("tracker_id") == tracker_id)
14         .select("wave_id")
15         .distinct()
16     )
```

```
❸     return sorted(r.wave_id for r in source.join(done, "wave_id",
    "left_anti").collect())
```

1. **Line 9.** First run: silver does not exist, so every wave is new.
2. **Line 12.** Silver is read with the same tracker filter as bronze.
3. **Line 17.** Left anti join: waves with no match in silver. Sorted, so waves load in order and logs are predictable.

Same filters on both sides, an explicit anti join, a first-run branch, and an idempotent load behind it.

Recap card

- **Rule for incremental wave detection** Filter bronze and silver by the same keys, such as tracker_id, then left-anti-join.
- **Join type for 'in A but not in B'** left_anti.
- **First run, silver missing** spark.catalog.tableExists(silver) is False: every bronze wave is new.
- **What does wave-ID detection miss?** Redeliveries of waves silver already holds. Use ingestion timestamps, change data feed or Auto Loader.

Practice

Exercise 1. Multiple choice (level 1)

Bronze for tracker A holds waves W13 and W14. Silver holds W13 for tracker A and W14 for tracker B. The reference function compares A's bronze waves with all of silver's waves. Which waves does it return?

- a) W14
- b) None
- c) W13 and W14
- d) W13

Why?

- r1) Tracker B's W14 in silver makes tracker A's W14 look done.
- r2) subtract removes all waves when silver is not empty.

Exercise 2. Fill the gap (level 2)

Keep only bronze waves that silver does not have, and handle a missing silver table.

```
1 if not spark.catalog._____(1)(silver):
2     return sorted(r.wave_id for r in source.collect())
```

```
3 new = source.join(done, "wave_id", "_____(2)")
```

Exercise 3. Spot the bug (level 3)

Tracker A's wave W14 never loads when tracker B already has W14. Tap the line, then fix the function.

```
1 def unprocessed_waves(spark: SparkSession, bronze: str, silver: str,
    tracker_id: str) -> list[str]:
2
    """Waves in bronze for this tracker that silver does not hold for this
    tracker."""
3     source = spark.table(bronze).filter(F.col("tracker_id") ==
    tracker_id).select("wave_id").distinct()
4     done = spark.table(silver).select("wave_id").distinct()
5     return sorted(r.wave_id for r in source.join(done, "wave_id",
    "left_anti").collect())
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 4)

Write `unprocessed_waves(spark, bronze, silver, tracker_id)`. Return the sorted wave IDs present in bronze for the tracker and absent from silver for the same tracker. When silver does not exist, return every bronze wave for the tracker. Use a left anti join.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
1 # Wave ids in bronze for this tracker that silver does not hold for this
    tracker.
2 def unprocessed_waves(
3     spark: SparkSession,
4     bronze: str,
5     silver: str,
6     tracker_id: str,
7 ) -> list[str]: ...
```

Starter code:

```
1 from pyspark.sql import SparkSession
2 from pyspark.sql import functions as F
3
4
5 def unprocessed_waves(spark: SparkSession, bronze: str, silver: str,
    tracker_id: str) -> list[str]:
6     ...
```

Exercise 5. Multiple choice (level 5)

Silver already holds wave W14. The supplier redelivers W14 with corrected weights. Your job processes "waves in bronze that are not in silver". What happens to the redelivery?

- a) It is processed, because bronze has a new ingestion_id.
- b) It is skipped, because W14 is already in silver.
- c) The job fails with a duplicate key error.
- d) It is processed twice.

Why?

- r1) The detection compares wave IDs only, and W14 exists on both sides.
- r2) Bronze rejects redeliveries.

Sources

- left_anti joins and spark.catalog.tableExists work as described on PySpark 4.0. (verified: spark_tests for A8-L1-E3 and E4 run on PySpark 4.0.4 with Delta.)

A8-L2 Auto Loader for file ingestion into bronze

Concept

Suppliers drop files into a landing folder. You want each file loaded into bronze exactly once, whenever it arrives, without keeping your own list of processed files. **Auto Loader** does this on Databricks. It is a Structured Streaming source called `cloudFiles`.

```
(
    spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("cloudFiles.schemaLocation", schema_path)
    .option("header", "true")
    .load(landing_path)
    .select("*", F.col("_metadata.file_path").alias("source_file"))
    .writeStream
    .option("checkpointLocation", checkpoint_path)
    .trigger(availableNow=True)
    .toTable("research_prod.research_bronze.nat_raw")
)
```

What each piece does:

- **Checkpoint location.** Auto Loader records every discovered file in a key-value store (RocksDB) under the checkpoint. After a failure it resumes

where it stopped, and each file path is processed once. One checkpoint per stream, never shared, never deleted casually: deleting it reloads every file.

- **Schema location.** Auto Loader infers the schema once, stores it there, and tracks changes. Without a schema you provide, the default mode is `addNewColumns`: when a new column appears, the stream stops with `UnknownFieldException` after recording the new schema, and the next start includes the column. Run it from a job that retries, so the restart is automatic.
- **Rescued data.** When Auto Loader infers the schema, it adds a `_rescued_data` column that captures values that do not fit, such as a type mismatch. Count non-null `_rescued_data` in your contract checks.
- `trigger(availableNow=True)` processes every file that has arrived, then stops. Schedule it as a batch job: incremental logic, batch cost.
- `_metadata.file_path` gives each row its source file, for the bronze audit columns.

Keep checkpoints and schemas in a Unity Catalog volume, such as `/Volumes/research_prod/research_bronze/checkpoints/nat_raw`.

Why it matters

A bronze notebook lists the landing folder, compares file names with a hand-maintained "processed.txt", and loads the rest. A timeout kills the notebook after it has loaded a file but before it has updated the list. The next run loads the same file again. Bronze now holds the delivery twice with two ingestion IDs, and silver's newest-version logic cannot tell them apart, because both carry the same `delivery_version`.

Auto Loader's checkpoint records files and commits the Delta write together, so a restart after a failure neither skips nor repeats a file.

Worked example: weak code

```
1  import os
2
1  done = set(open("/dbfs/tmp/processed.txt").read().split())
4  for name in os.listdir("/dbfs/landing/nat"):
5      if name not in done:
6          df = spark.read.option("header", True).csv(f"/landing/nat/{name}")
2          df.write.mode("append").saveAsTable("research_bronze.nat_raw")
3          open("/dbfs/tmp/processed.txt", "a").write(name + "\n")
```

1. **Line 3.** Hand-built state in a temp file: easy to lose, and not linked to the table.

2. **Line 7.** The write and the list update are separate steps. A crash between them duplicates a file.
3. **Line 8.** State recorded after the write, with no transaction.

Worked example: strong code

```
1  def start_bronze_ingest(spark: SparkSession, landing: str, table: str,
       checkpoint: str, schema_path: str) -> StreamingQuery:
2      """Load every new file in landing into bronze once, then stop."""
3      raw = (
1          spark.readStream.format("cloudFiles")
5          .option("cloudFiles.format", "csv")
2          .option("cloudFiles.schemaLocation", schema_path)
7          .option("header", "true")
8          .load(landing)
9      )
10     audited = raw.select(
11         "*",
3          F.col("_metadata.file_path").alias("source_file"),
13         F.current_timestamp().alias("ingestion_timestamp"),
14     )
15     return (
16         audited.writeStream.option("checkpointLocation", checkpoint)
4          .trigger(availableNow=True)
18         .toTable(table)
19     )
```

1. **Line 4.** Auto Loader discovers new files and tracks them in the checkpoint.
2. **Line 6.** Where the inferred schema and its changes are stored.
3. **Line 12.** Source file for every row, for the bronze audit columns.
4. **Line 17.** Process everything available, then stop: scheduled like a batch job.

Auto Loader replaces a fragile processed-files list with a checkpoint that commits together with the Delta write.

Recap card

- **What does Auto Loader's checkpoint store?** Discovered files (in RocksDB) and stream progress, so each file is processed exactly once across restarts.
- **Default schema evolution mode without a supplied schema** addNewColumns: stop with UnknownFieldException, record the new schema, restart.
- **Incremental at batch cost** .trigger(availableNow=True) on a scheduled job.
- **Checkpoint rule** One per stream, in durable storage such as a UC volume. Never shared, never deleted casually.

Practice

Exercise 1. Multiple choice (level 1)

What lets Auto Loader process each landing file exactly once across restarts?

- a) The schema location
- b) The checkpoint location
- c) The `_rescued_data` column
- d) The file names in the target table

Why?

- r1) Auto Loader records discovered files in a key-value store in the checkpoint and resumes from it.
- r2) The checkpoint deletes files after loading them.

Exercise 2. Fill the gap (level 2)

Complete the Auto Loader read and a write that processes all available files, then stops.

```
raw = (
    spark.readStream.format("_____(1)")
    .option("cloudFiles.format", "csv")
    .option("cloudFiles._____(2)", schema_path)
    .load(landing)
)
raw.writeStream.option("checkpointLocation",
   checkpoint).trigger(_____(3)=True).toTable(table)
```

Exercise 3. Multiple choice (level 3)

An Auto Loader stream runs without a user-supplied schema. A new delivery adds a column `region`. What happens?

- a) The column is silently dropped.
- b) The stream stops with UnknownFieldException after recording the new schema. The next start includes region.
- c) The stream continues and region lands in _rescued_data, forever.
- d) The whole bronze table is rebuilt.

Why?

- r1) The default mode without a schema is addNewColumns, which evolves by failing once and restarting.
- r2) Auto Loader never changes a schema automatically.

Exercise 4. Spot the bug (level 4)

Two Auto Loader streams load different landing folders into two bronze tables. After a week, each table is missing files the other received. Tap the line, then fix the function.

```
1 CHECKPOINT = "/Volumes/research_prod/research_bronze/checkpoints/bronze"
2
3
4 def start_ingest(spark: SparkSession, landing: str, table: str,
    schema_path: str) -> StreamingQuery:
5     """Load new files from landing into table."""
6     raw =
    spark.readStream.format("cloudFiles").option("cloudFiles.format",
    "csv").option("cloudFiles.schemaLocation", schema_path).load(landing)
7     return raw.writeStream.option("checkpointLocation",
    CHECKPOINT).trigger(availableNow=True).toTable(table)
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Architecture design (level 5)

Design the Auto Loader job for tracker nat. Choose each setting.

Options: trigger(availableNow=True), scheduled, A continuously running stream.

1. Files arrive a few times a day, and cost matters more than latency
2. Checkpoint location
3. What to do with _rescued_data

Sources

- Auto Loader tracks files in RocksDB in the checkpoint for exactly-once processing; _metadata.file_path is available; checkpoints can live in Volumes paths. (verified: https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/ (checked 2026-09-25))
- addNewColumns is the default schema evolution mode when no schema is provided; the stream stops with UnknownFieldException and picks up new columns on restart; _rescued_data is added when the schema is inferred. (verified: https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/schema (checked 2026-09-25))
- DataStreamWriter.trigger accepts availableNow=True and toTable exists in PySpark 4.0. (verified: Checked on PySpark 4.0.4 in this repository.)

A8-L3 Change data feed, so downstream layers read only what changed

Concept

When silver changes, gold should reprocess only the rows that changed, not the whole table. **Change data feed** (CDF) makes a Delta table record row-level changes for every write.

Turn it on per table:

```sql
ALTER TABLE research_prod.research_silver.nat_respondents
SET TBLPROPERTIES (delta.enableChangeDataFeed = true);
```

Read the changes since a version:

```python
changes = (
    spark.read.option("readChangeFeed", "true")
    .option("startingVersion", last_version + 1)
    .table("research_prod.research_silver.nat_respondents")
)
```

Each row carries three extra columns:

- `_change_type`: `insert`, `update_preimage` (the row before an update), `update_postimage` (the row after), or `delete`.
- `_commit_version` and `_commit_timestamp`: which commit made the change.

Two rules make CDF consumers correct:

1. **Drop `update_preimage` rows** before you use the changes. An update produces two rows, old and new. Keep the old one and every updated respondent is counted twice.
2. **Keep one row per key**: the latest by `_commit_version`. A respondent updated in three commits since your last run appears three times.

Track progress. Either store the last processed version in a state table, or read the feed as a stream (`readStream` with `readChangeFeed`) and let the checkpoint remember it. Change data only exists from the version where you enabled it. Asking for an earlier version fails with `DELTA_MISSING_CHANGE_DATA`. VACUUM also removes old change files after the retention window.

For gold, collect the waves and subgroups the changes touch, then recompute just those aggregates. Aggregates cannot be patched row by row.

Why it matters

A gold job rebuilds weighted metrics for every wave every night: 180 waves, 40 minutes. Most nights, only one wave changed. When a late correction

arrives mid-afternoon, the team cannot refresh gold before the client call, because a full rebuild takes too long.

With CDF on silver, the job reads the day's changes, finds the one wave they touch, and recomputes it in under two minutes. The same job also stops double counting, because it drops `update_preimage` rows before aggregating.

Worked example: weak code

```
1 def changed_rows(spark, table, since):
2     changes = (
3         spark.read.option("readChangeFeed", "true")
2         .option("startingVersion", since)
5         .table(table)
6     )
3     return changes.filter(F.col("_change_type") != "delete")
```

1. **Line 1.** A respondent updated in several commits still appears once per commit.
2. **Line 4.** If since is the last processed version, that version is read again.
3. **Line 7.** Keeps update_preimage rows: every updated respondent appears twice, once with old values.

Worked example: strong code

```
1  CURRENT_STATE = ["insert", "update_postimage"]
2
3
4  def read_changes(spark: SparkSession, table: str, last_processed_version:
   int) -> DataFrame:
5      """Row changes committed after last_processed_version."""
6      return (
7          spark.read.option("readChangeFeed", "true")
1          .option("startingVersion", last_processed_version + 1)
9          .table(table)
10     )
11
12
13 def latest_state(changes: DataFrame, keys: list[str]) -> DataFrame:
14     """One row per key: its newest change, excluding pre-images."""
2      window =
   Window.partitionBy(*keys).orderBy(F.col("_commit_version").desc())
16     return (
3          changes.filter(F.col("_change_type") != "update_preimage")
18         .withColumn("_rank", F.row_number().over(window))
19         .filter(F.col("_rank") == 1)
20         .drop("_rank")
21     )
22
23
4  def touched_waves(changes: DataFrame) -> list[str]:
25     """Waves with any change: the only gold waves to recompute."""
```

```
26      return sorted(r.wave_id for r in
        changes.select("wave_id").distinct().collect())
```

1. **Line 8.** Start after the last processed version, so nothing is read twice.
2. **Line 15.** Several changes to one key since the last run: keep the newest. Deletes stay, so the consumer can remove the key.
3. **Line 17.** Pre-images describe old state. Drop them.
4. **Line 24.** Gold recomputes whole aggregates, but only for the waves that changed.

Read after the last version, drop pre-images, keep the newest change per key, and recompute only what the changes touch.

Recap card

- **Turn on change data feed** ALTER TABLE t SET TBLPROPERTIES (delta.enableChangeDataFeed = true)
- **The four _change_type values** insert, update_preimage (old), update_postimage (new), delete.
- **Two rules for consuming CDF** Drop update_preimage. Keep the newest change per key by _commit_version.
- **Where does the next run start?** startingVersion = last processed version + 1 (it is inclusive), or a streaming checkpoint.

Practice

Exercise 1. Multiple choice (level 1)

An UPDATE changes one respondent's weight. Which change feed row holds the new weight?

- a) insert
- b) update_preimage
- c) update_postimage
- d) delete

Why?

- r1) An update records the row before (pre-image) and after (post-image). The post-image holds the new values.
- r2) Updates are recorded as a delete followed by an insert.

Exercise 2. Predict the output (level 2)

CDF is enabled when the table is created. How many rows does the change read return?

What you need to know.

In SQL, `table_changes('table_name', start_version)` returns the same rows as reading the change feed with `readChangeFeed`: one row per change from `start_version` onwards, with `_change_type`, `_commit_version` and `_commit_timestamp`.

`TBLPROPERTIES (...)` inside `CREATE TABLE` sets table properties at creation, as `ALTER TABLE ... SET TBLPROPERTIES` does later.

```sql
CREATE TABLE c1 (id INT, v STRING) USING DELTA
  TBLPROPERTIES (delta.enableChangeDataFeed = true);   -- version 0
INSERT INTO c1 VALUES (1, 'a'), (2, 'b');              -- version 1
UPDATE c1 SET v = 'z' WHERE id = 1;                    -- version 2
DELETE FROM c1 WHERE id = 2;                           -- version 3

SELECT COUNT(*) FROM table_changes('c1', 2);
```

Exercise 3. Fill the gap (level 3)

Read silver's changes committed after the last processed version.

```python
changes = (
    spark.read.option("_____(1)", "true")
    .option("startingVersion", _____(2))
    .table(silver)
)
```

Exercise 4. Write the function (level 4)

Write `latest_state(changes, keys)`. From a change feed DataFrame, drop `update_preimage` rows, then keep one row per key: the one with the highest `_commit_version`. Use a window with `row_number`.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# One row per key: its newest change, excluding update pre-images.
def latest_state(
    changes: DataFrame,
    keys: list[str],
) -> DataFrame: ...
```

Starter code:

```python
from pyspark.sql import DataFrame, Window
from pyspark.sql import functions as F


def latest_state(changes: DataFrame, keys: list[str]) -> DataFrame:
    ...
```

Exercise 5. Spot the bug (level 5)

After enabling CDF, updated respondents appear twice in the gold input, once with old values. Tap the line, then fix the function.

What you need to know.

`F.col(name).isin([...])` is true when the column's value equals any item in the list. It keeps several allowed values in one condition:

```
1 df.filter(F.col("region").isin(["North", "South"]))
```

```
1 def current_rows(spark: SparkSession, table: str, last_processed_version:
    int) -> DataFrame:
2     """Inserted and updated rows since the last run, with their new
    values."""
3     changes = spark.read.option("readChangeFeed",
    "true").option("startingVersion", last_processed_version +
    1).table(table)
4     return changes.filter(F.col("_change_type") != "delete")
```

Write down the line number of the bug. Then write the fix.

Practical: Read a change data feed and keep the latest state

Platform: Databricks Free Edition. Run this in Databricks Free Edition. It enables change data feed on a small silver table, applies an insert, an update and a delete, then reads the changes and reduces them to the latest state per respondent.

Download the notebook `A8-L3.ipynb` from the app.

Sources

- CDF read semantics, the four change types, table_changes, and DELTA_MISSING_CHANGE_DATA for versions before CDF was enabled. (verified: Run on delta-spark 4.0.1 in this repository.)
- CDF on Databricks: delta.enableChangeDataFeed, readChangeFeed, startingVersion, table_changes. (verified: https://docs.databricks.com/aws/en/tables/features/change-data-feed (checked 2026-09-25))

A9 Testing

Roadmap: Months 16 to 18: MLOps and deployment; Current work. pytest, fixtures, local SparkSession and DataFrame equality checks.

A9-L1 pytest basics, and a SparkSession fixture shared by every test

Concept

pytest finds files named `test_*.py`, runs every function named `test_*`, and reports each failing `assert` with the values on both sides. No classes or special assert methods are needed.

```
def test_weighted_share_uses_weights() -> None:
    assert weighted_share([1, 0], [3.0, 1.0]) == 0.75
```

Fixtures provide what tests need. A test names a fixture as a parameter, and pytest builds it and passes it in. For Spark, one fixture creates the session:

```
@pytest.fixture(scope="session")
def spark() -> Iterator[SparkSession]:
    session = SparkSession.builder.master("local[2]").getOrCreate()
    yield session
    session.stop()
```

- `scope="session"` builds it once for the whole test run. Starting Spark takes several seconds. Once per test, and a 40-test suite spends minutes on start-up alone.
- `yield` hands the value to the tests. The code after `yield` runs once at the end, which is where you stop the session.
- Put shared fixtures in `conftest.py`. pytest loads it automatically for every test in that folder and below.

`@pytest.mark.parametrize` runs one test with several inputs, and each input is reported as its own test:

```
@pytest.mark.parametrize(("flags", "weights", "expected"), [([1], [2.0], 1.0), ([0, 0], [1.0, 1.0], 0.0)])
def test_share(flags, weights, expected): ...
```

A good test fails for the bug you fear. Write it against the known answer on small data, including the edge cases: empty input, zero totals, missing values. Check it by breaking the code on purpose and watching the test fail.

Why it matters

A team's test for `weighted_share` checks that the result is between 0 and 1. The function is later rewritten and accidentally returns the unweighted share. The result is still between 0 and 1, so the test passes, and weighted reach in the dashboard is wrong for a month.

A test with a known answer, `weighted_share([1, 0], [3.0, 1.0]) == 0.75`, fails the moment the weights are ignored, because the unweighted answer is 0.5.

Worked example: weak code

```
 1 import pytest
 2 from pyspark.sql import SparkSession
 3
 4
 5 @pytest.fixture
 6 def spark():
 7     session = SparkSession.builder.master("local[2]").getOrCreate()
 8     yield session
 9     session.stop()
10
11
12 def test_share():
13     share = weighted_share([1, 0, 1], [2.0, 1.0, 1.0])
14     assert 0 <= share <= 1
```

1. **Line 5.** Default function scope: Spark starts and stops for every test that uses it.
2. **Line 14.** Passes for almost any wrong implementation. It does not test the rule.

Worked example: strong code

```
 1 from collections.abc import Iterator
 2
 3 import pytest
 4 from pyspark.sql import SparkSession
 5
 6
 7 @pytest.fixture(scope="session")
 8 def spark() -> Iterator[SparkSession]:
 9     """One local session for the whole test run."""
10     session =
   SparkSession.builder.master("local[2]").config("spark.ui.enabled",
   "false").getOrCreate()
12     yield session
13     session.stop()
14
15 @pytest.mark.parametrize(
16     ("flags", "weights", "expected"),
```

```
    [
        ([1, 0, 1], [2.0, 1.0, 1.0], 0.75),
        ([0, 0], [1.0, 1.0], 0.0),
        ([1, 0], [0.0, 0.0], None),
    ],
)
def test_weighted_share(flags: list[int], weights: list[float], expected:
    float | None) -> None:
    assert weighted_share(flags, weights) == expected
```

1. **Line 7.** One Spark start-up for the whole run.
2. **Line 11.** Tests run here. The line after yield is the teardown.
3. **Line 18.** A known answer that differs from the unweighted one (0.67), so ignoring weights fails.
4. **Line 20.** The edge case: zero total weight must give None.

One session-scoped fixture, and tests with known answers that fail for the bugs you care about.

Recap card

- **What does pytest run?** Functions named test_* in files named test_*.py. Plain assert statements.
- **SparkSession fixture scope** scope="session": start Spark once per test run.
- **yield in a fixture** Hands the value to tests. Code after yield is the teardown.
- **What makes a test worth having?** It fails for the bug you fear: known answers that separate right from wrong, plus edge cases.

Practice

Exercise 1. Multiple choice (level 1)

Which fixture scope suits a SparkSession used by 40 tests?

- a) function
- b) session
- c) No fixture: create a session inside each test
- d) class

Why?

- r1) Spark takes seconds to start, and tests can safely share one local session.
- r2) Only session-scoped fixtures can use yield.

Exercise 2. Predict the output (level 2)

How many tests does pytest report for this file?

```python
import pytest


@pytest.mark.parametrize("wave", ["W13", "W14", "W15"])
def test_wave_loads(wave: str) -> None:
    assert wave.startswith("W")


def test_empty_wave_list() -> None:
    assert unprocessed([]) == []
```

Exercise 3. Fill the gap (level 2)

Complete the fixture so Spark starts once per test run and stops at the end.

```python
@pytest._____(1)(scope="_____(2)")
def spark() -> Iterator[SparkSession]:
    session = SparkSession.builder.master("local[2]").getOrCreate()
    _____(3) session
    session.stop()
```

Exercise 4. Write the function (level 4)

`weighted_share(flags, weights)` returns the share of weight where the flag is 1, or `None` when the total weight is 0. Write **the test**, `test_weighted_share()`, not the function.

The checker runs your test three times: against a correct implementation (it must pass), against one that ignores the weights, and against one that returns 0.0 instead of None for a zero total (both must fail).

Your code must pass these checks:

- Your test fails against a correct implementation. Check the expected values.
- Your test passes when the weights are ignored. Use weights where the weighted and unweighted answers differ, such as [1, 0] with [3.0, 1.0].
- Your test passes when a zero total returns 0.0. Add a case with all weights 0 and assert the result `is None`.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
def test_weighted_share() -> None: ...
```

Starter code:

```python
def test_weighted_share() -> None:
    ...
```

Exercise 5. Spot the bug (level 5)

The test suite takes six minutes for 45 small tests. Tap the line that causes most of it, then fix the fixture.

```
1 import pytest
2 from pyspark.sql import SparkSession
3
4
5 @pytest.fixture
6 def spark():
7     session = SparkSession.builder.master("local[2]").getOrCreate()
8     yield session
9     session.stop()
```

Write down the line number of the bug. Then write the fix.

Sources

- pytest collects one test per parametrize value; fixtures with yield run teardown after; scope='session' builds once per run. (verified: pytest 9.1.1 behaviour, used by this repository's own test suite.)

A9-L2 A fast local SparkSession and readable test data

Concept

Unit tests for PySpark run on your laptop and in CI, not on a cluster. A local session that starts in seconds and runs small jobs fast needs a few settings:

```
1 builder = (
2     SparkSession.builder.master("local[2]")
3     .config("spark.sql.shuffle.partitions", "2")
4     .config("spark.ui.enabled", "false")
5     .config("spark.sql.extensions",
    "io.delta.sql.DeltaSparkSessionExtension")
6     .config("spark.sql.catalog.spark_catalog",
    "org.apache.spark.sql.delta.catalog.DeltaCatalog")
7     .config("spark.sql.sources.default", "delta")
8 )
9 spark = configure_spark_with_delta_pip(builder).getOrCreate()
```

- `local[2]` runs Spark in one process with two worker threads. Two threads, not one, so bugs that depend on partitioning still show up.
- `spark.sql.shuffle.partitions` defaults to 200. On ten test rows, every shuffle creates 200 tiny tasks. Two is plenty.
- `configure_spark_with_delta_pip` from `delta-spark` adds the matching Delta JAR. The two `config` lines enable Delta SQL and the Delta catalog.

- Open-source Spark writes `saveAsTable` as Parquet by default. Databricks uses Delta. Set the default so tests behave like production.

Test data. Build it with an explicit schema, as a DDL string such as `"respondent_id STRING, final_weight DOUBLE"`. Inference guesses types from values, fails on a column of only `None` (`CANNOT_DETERMINE_TYPE`), and turns `2` into a LONG where production has DOUBLE.

A **test data builder** keeps tests short: one function returns a valid row with sensible defaults, and each test overrides only the field it is about.

```
1 def respondent(**overrides: object) -> dict[str, object]:
2     row = {"respondent_id": "R1", "wave_id": "W1", "final_weight": 1.0,
    "in_scope": True}
3     row.update(overrides)
4     return row
```

A test that says `respondent(final_weight=None)` tells the reader exactly what it is testing.

Why it matters

A project's Spark tests take 14 minutes in CI, so developers stop running them locally. The session uses the default 200 shuffle partitions, and each of the 60 tests runs two or three shuffles on a handful of rows. Most of the time goes into scheduling empty tasks. A MERGE bug that a test would have caught ships to production, because nobody waited for the suite.

With `local[2]` and two shuffle partitions, the same suite runs in under two minutes, and people run it before every push.

Worked example: weak code

```
1 @pytest.fixture(scope="session")
2 def spark():
(1)     return SparkSession.builder.getOrCreate()
4
5
6 def test_null_weights_are_reported(spark):
(2)     df = spark.createDataFrame([{"respondent_id": "R1", "final_weight":
    None, "wave_id": "W1", "in_scope": True}])
8     assert null_weight_count(df) == 1
```

1. **Line 3.** No master, 200 shuffle partitions, UI on, no Delta: slow and unlike production.
2. **Line 7.** Inferred schema: a column of only None raises CANNOT_DETERMINE_TYPE, and every field is spelt out in every test.

Worked example: strong code

```
1  RESPONDENT_SCHEMA = "respondent_id STRING, wave_id STRING, final_weight DOUBLE, in_scope BOOLEAN"
2
3
4  def respondent(**overrides: object) -> dict[str, object]:
5      """A valid silver respondent. Override only the fields a test is about."""
6      row: dict[str, object] = {"respondent_id": "R1", "wave_id": "W1", "final_weight": 1.0, "in_scope": True}
7      unknown = set(overrides) - set(row)
8      if unknown:
2          raise KeyError(f"unknown fields: {sorted(unknown)}")
10     row.update(overrides)
11     return row
12
13
14 def respondents_df(spark: SparkSession, rows: list[dict[str, object]]) -> DataFrame:
15     """DataFrame with the silver schema, whatever the values."""
16     fields = [f.split()[0] for f in RESPONDENT_SCHEMA.split(", ")]
17     return spark.createDataFrame([tuple(r[f] for f in fields) for r in rows], RESPONDENT_SCHEMA)
18
19
20 def test_null_weights_are_reported(spark: SparkSession) -> None:
3      df = respondents_df(spark, [respondent(), respondent(respondent_id="R2", final_weight=None)])
22     assert null_weight_count(df) == 1
```

1. **Line 1.** One DDL schema: types match production, and None is always allowed.
2. **Line 9.** A typo such as final_wieght fails loudly instead of silently adding nothing.
3. **Line 21.** The test reads as its intent: a second respondent with a missing weight.

A tuned local session makes tests fast enough to run on every change. Builders with explicit schemas make them short and exact.

Recap card

- **Local test session settings** master("local[2]"), shuffle partitions 2, UI off, Delta configured with configure_spark_with_delta_pip.
- **Default spark.sql.shuffle.partitions** 200: far too many for test data.
- **Why explicit schemas in test data?** Inference fails on all-None columns and can pick different types from production.
- **Test data builder** A function with valid defaults. Tests override only the fields they are about, and typos raise.

Practice

Exercise 1. Predict the output (level 1)

A fresh local session on PySpark 4.0 with no extra configuration. What does this print?

```
print(spark.conf.get("spark.sql.shuffle.partitions"))
```

Exercise 2. Fill the gap (level 2)

Complete the local test session: two threads, two shuffle partitions, with the Delta JAR added.

```
builder = (
    SparkSession.builder.master("_____(1)")
    .config("spark.sql.shuffle.partitions", "_____(2)")
    .config("spark.sql.extensions",
    "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog",
    "org.apache.spark.sql.delta.catalog.DeltaCatalog")
)
spark = _____(3)(builder).getOrCreate()
```

Exercise 3. Write the function (level 3)

Write `respondent(**overrides)`. It returns a dict with defaults `respondent_id="R1"`, `wave_id="W1"`, `final_weight=1.0`, `in_scope=True`, updated with the overrides. Raise `KeyError` naming any override that is not one of those four fields. Each call must return a new dict.

Your code must pass these checks:

- With no overrides, return the four defaults.
- Overrides replace only the named fields.
- An unknown field must raise KeyError naming it, or typos slip through.
- Each call must return a fresh dict. Changing one must not change the defaults.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# A valid silver respondent row. Override only the fields a test is about.
def respondent(**overrides: object) -> dict[str, object]: ...
```

Starter code:

```
def respondent(**overrides: object) -> dict[str, object]:
    ...
```

Exercise 4. Spot the bug (level 4)

This helper builds test data. It works for most tests, but fails with `CANNOT_DETERMINE_TYPE` in the test for missing weights. Tap the line, then fix the function with an explicit schema.

```
1 def weights_df(spark: SparkSession, weights: list[float | None]) ->
    DataFrame:
2     """One respondent per weight."""
3     rows = [(f"R{i}", w) for i, w in enumerate(weights)]
4     return spark.createDataFrame(rows, ["respondent_id", "final_weight"])
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Put in order (level 5)

Order the set-up of a local PySpark test harness.

- Write a session-scoped spark fixture in conftest.py
- Create a virtual environment and install pyspark, delta-spark and pytest
- Write tests with known answers against the builders
- Write test data builders with explicit schemas
- Install Java 17 or 21

Sources

- spark.sql.shuffle.partitions defaults to 200; createDataFrame with only column names infers types and fails on all-None columns. (verified: Run on PySpark 4.0.4 in this repository.)
- configure_spark_with_delta_pip adds the Delta package matching the installed delta-spark version. (verified: Used by practice/conftest.py with delta-spark 4.0.1.)

A9-L3 Compare DataFrames properly, and test that MERGE is idempotent

Concept

Comparing DataFrames by hand is harder than it looks. `actual.collect() == expected.collect()` fails when the rows come back in a different order, which Spark never guarantees. It also fails when a float is `0.30000000000000004` instead of `0.3`. Sorting both sides fixes the order but not the floats, and neither checks the schema.

`pyspark.testing` (PySpark 3.5 and later) handles all of this:

```
1 from pyspark.testing import assertDataFrameEqual, assertSchemaEqual
```

```
2
3 assertDataFrameEqual(actual, expected)
```

Tested on PySpark 4.0.4:

- Row order is ignored by default. Pass `checkRowOrder=True` when order is the point, as in a top-10 list.
- Floats compare with a tolerance (`rtol=1e-5`, `atol=1e-8` by default).
- Schemas must match: INT against BIGINT fails with `DIFFERENT_SCHEMA`.
- Duplicates count: two identical rows do not equal one.
- `expected` can be a list of `Row` objects, which is handy for snapshots.
- Failures raise `PySparkAssertionError`, a subclass of `AssertionError`, so pytest reports them as ordinary test failures, with a diff.

`assertSchemaEqual(actual.schema, expected.schema)` checks only the structure.

Testing MERGE. The property that matters is idempotence. The test:

1. Create an empty Delta table with the target schema.
2. Run the load once, and take a snapshot: `first = spark.table(t).collect()`.
3. Run the same load again.
4. `assertDataFrameEqual(spark.table(t), first)`.

Add cases for the failures you fear: a repeated key in the source, a redelivery with a newer version, and the same respondent in two waves. Each one takes a few rows and a second or two.

In the practice harness, PySpark 4.0.4 needs pandas 2.x. With pandas 3.x, `assertDataFrameEqual` fails to import.

Why it matters

A silver test compares `actual.collect() == expected.collect()`. It passes on a developer's laptop and fails one CI run in five, because the order of rows varies with partitioning. The team marks it flaky and skips it. A month later a real regression in the MERGE, duplicate rows after a retry, ships untested.

`assertDataFrameEqual` ignores order, so the test is stable, and an idempotence test that runs the load twice catches the regression.

Worked example: weak code

```
1 def test_silver_load(spark):
1     upsert_silver(spark, source, "silver", ["respondent_id", "wave_id"],
  "delivery_version")
```

```
❷     assert spark.table("silver").collect() == expected.collect()
```

1. **Line 2.** One run only. It never checks what a retry does.
2. **Line 3.** Order-dependent and exact on floats: flaky, and a failure shows no useful diff.

Worked example: strong code

```
 1 from pyspark.testing import assertDataFrameEqual
 2
 3 KEYS = ["respondent_id", "wave_id"]
 4 SCHEMA = "respondent_id STRING, wave_id STRING, final_weight DOUBLE,
     delivery_version INT"
 5
 6
 7 def test_silver_load_is_idempotent(spark: SparkSession, empty_silver:
     str) -> None:
 8     source = spark.createDataFrame(
❶          [("R1", "W1", 1.2, 1), ("R1", "W1", 1.5, 2), ("R2", "W1", 0.8,
     1)], SCHEMA
10     )
11     upsert_silver(spark, source, empty_silver, KEYS, "delivery_version")
❷     first = spark.table(empty_silver).collect()
13
14     upsert_silver(spark, source, empty_silver, KEYS, "delivery_version")
15
❸     assertDataFrameEqual(spark.table(empty_silver), first)
17     expected = spark.createDataFrame([("R1", "W1", 1.5, 2), ("R2", "W1",
     0.8, 1)], SCHEMA)
❹     assertDataFrameEqual(spark.table(empty_silver), expected)
```

1. **Line 9.** The source includes a repeated key, the case that breaks a naive MERGE.
2. **Line 12.** A snapshot of the table after one run, as a list of Rows.
3. **Line 16.** Second run leaves the table unchanged: idempotence, order-independent.
4. **Line 18.** And the content is right: the newest version of R1 won.

One helper compares DataFrames the right way, and one small test proves a MERGE survives a retry and a repeated key.

Recap card

- **Compare two DataFrames in a test**
  pyspark.testing.assertDataFrameEqual(actual, expected). Ignores order, tolerates float rounding, checks schema.
- **When should row order matter?** Only for ranked output. Pass checkRowOrder=True.

- **Idempotence test for a MERGE** Load, snapshot with collect(), load again, assertDataFrameEqual(table, snapshot).
- **Why collect() for the snapshot?** A DataFrame is lazy and would re-read the table after the second run.

Practice

Exercise 1. Multiple choice (level 1)

`actual` holds rows (1, 0.3) and (2, 1.0). `expected` holds (2, 1.0) and (1, 0.1 + 0.2). Both have the schema `id INT, w DOUBLE`. What does `assertDataFrameEqual(actual, expected)` do on PySpark 4.0?

- a) Passes
- b) Fails because the rows are in a different order
- c) Fails because 0.1 + 0.2 is not exactly 0.3
- d) Fails because it needs checkRowOrder=True

Why?

- r1) It ignores row order by default and compares floats with a relative tolerance.
- r2) It only compares schemas.

Exercise 2. Predict the output (level 2)

A hand-written comparison sorts both sides first. What does it print?

```
actual = [(1, 0.1 + 0.2), (2, 1.0)]
expected = [(2, 1.0), (1, 0.3)]
print(sorted(actual) == sorted(expected))
```

Exercise 3. Fill the gap (level 2)

Import the helpers, then check that the top-10 regions come back in the right order.

```
from pyspark._____(1) import assertDataFrameEqual, assertSchemaEqual

assertDataFrameEqual(top_regions, expected, _____(2)=True)
```

Exercise 4. Write the function (level 4)

Write `check_idempotent(spark, load, table)`. `load` is a function that takes no arguments and loads a fixed source into `table`. Run it once, snapshot the table with `collect()`, run it again, and assert the table still equals the snapshot with `assertDataFrameEqual`.

What you need to know.

`Callable` from `collections.abc` types a parameter that holds a function. `Callable[[], None]` is a function that takes no arguments and returns nothing. You run it by calling the parameter, like any function:

```
from collections.abc import Callable

def twice(step: Callable[[], None]) -> None:
    step()
    step()
```

`.collect()` returns a list of rows, a snapshot that does not change when the table changes later.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Assert that running load twice leaves table as one run did.
def check_idempotent(
    spark: SparkSession,
    load: Callable[[], None],
    table: str,
) -> None: ...
```

Starter code:

```
from collections.abc import Callable

from pyspark.sql import SparkSession
from pyspark.testing import assertDataFrameEqual


def check_idempotent(spark: SparkSession, load: Callable[[], None],
   table: str) -> None:
     ...
```

Exercise 5. Spot the bug (level 5)

This test fails about one CI run in five, though the code under test is correct. Tap the line, then fix the test.

```
def test_gold_metrics(spark: SparkSession) -> None:
    silver = spark.createDataFrame([("R1", "W1", "N", 1.0), ("R2", "W1",
   "S", 1.0)], "respondent_id STRING, wave_id STRING, region STRING,
   final_weight DOUBLE")
    expected = spark.createDataFrame([("N", 1.0), ("S", 1.0)], "region
   STRING, weighted_base DOUBLE")
    actual = aggregate_base(silver).select("region", "weighted_base")
    assert actual.collect() == expected.collect()
```

Write down the line number of the bug. Then write the fix.

Sources

- assertDataFrameEqual ignores row order by default, uses rtol=1e-5 and atol=1e-8, fails on schema differences and duplicates, accepts a list of Rows as expected, and raises PySparkAssertionError (a subclass of AssertionError). (verified: Run on PySpark 4.0.4 with pandas 2.3.3 in this repository.)
- PySpark 4.0.4 fails to import assertDataFrameEqual with pandas 3.x. (verified: ImportError: cannot import name '_builtin_table' from 'pandas.core.common', reproduced with pandas 3.0.6.)

A10 Deployment

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Git folders, Databricks Asset Bundles, parameterised jobs and CI.

A10-L1 Git folders, one branch per change, and no secrets in the repository

Concept

Production pipeline code lives in Git, not in someone's workspace folder. Databricks **Git folders** connect a workspace folder to a remote repository, so you can edit, commit, pull and switch branches from the workspace.

The workflow Databricks documents:

- Each developer clones the repository into their own Git folder under `/Workspace/Users/<email>/` and works on their own branch.
- Use a **trunk-based** strategy: short-lived feature branches off `main`, merged through a pull request with review and CI checks.
- **Production Git folders** are different. An admin creates them outside user folders. They hold a deployment branch, and only automation updates them, after a pull request merges. Nobody edits code there by hand.
- For deploying jobs and their configuration together, Databricks recommends Declarative Automation Bundles (lesson A10-L2).

Store notebooks as source files (`.py` with `# Databricks notebook source` headers, or `.ipynb`). Put reusable functions in plain Python modules that notebooks import, so tests can import them too.

Secrets never go into the repository. Not in code, not in config files, not in notebook outputs. Git keeps every version forever, and removing a token in a later commit does not remove it from history. Rotate any secret that was ever committed.

In Databricks, read secrets at run time:

```
api_key = dbutils.secrets.get(scope="research", key="supplier_api_key")
```

Databricks redacts secret values read this way, showing `[REDACTED]` if you print them. Redaction does not stop deliberate transformations, so never try to display a secret. In CI, secrets come from the CI system's secret store as environment variables.

Add a `.gitignore` for local files (`.venv/`, `__pycache__/`, `.env`), so credentials and environments never get staged by accident.

Why it matters

A developer tests a supplier API from a notebook with a hardcoded token, then commits the notebook to share the code. A week later they replace the token with a secret lookup in a new commit, believing the problem is fixed. The repository is later mirrored to a partner, and the token is still in the history. The supplier revokes the key after detecting unusual use, and every ingestion job that depends on it fails for two days.

With `dbutils.secrets.get` from the first line, there is nothing to leak.

Worked example: weak code

```
1 # silver_nat_transform (edited directly in the production folder)
2 SUPPLIER_TOKEN = "example-token-not-real-1234"
3 CATALOG = "research_prod"
4
5 def fetch_deliveries():
6     return requests.get(URL, headers={"Authorization": f"Bearer
    {SUPPLIER_TOKEN}"})
```

1. **Line 1.** Hand edits in production skip review and CI, and drift from Git.
2. **Line 2.** A token in code. Once committed, it stays in Git history for good.
3. **Line 3.** Environment hardcoded, so the same code cannot run in dev.

Worked example: strong code

```
1 # Databricks notebook source
1 # silver_nat_transform: edited on a feature branch, deployed by CI after
    review.
3 from pipelines.config import load_run_config
2 from pipelines.supplier import fetch_deliveries
5
3 cfg = load_run_config(dbutils.widgets.get("config_path"))
4 token = dbutils.secrets.get(scope="research", key="supplier_api_key")
8 deliveries = fetch_deliveries(cfg.supplier_url, token)
```

1. **Line 2.** Changes reach production only through a reviewed, tested merge.
2. **Line 4.** Logic lives in importable modules that unit tests can reach.
3. **Line 6.** The environment comes from a job parameter, not from the code.
4. **Line 7.** Read at run time from a secret scope. Nothing sensitive in the repository.

Code flows from a branch, through review and CI, to production. Secrets flow from a secret store at run time, never through Git.

Recap card

- **Where do developers work in Databricks Git folders?** Their own clone under /Workspace/Users/<email>/, on their own branch.
- **Who updates production Git folders?** Only automation, after a pull request merges into the deployment branch.
- **Read a secret in a notebook** dbutils.secrets.get(scope="...", key="..."). Output shows [REDACTED].
- **A token was committed, then removed. Is it safe?** No. Git history keeps it. Rotate the token.

Practice

Exercise 1. Multiple choice (level 1)

Who should update the production Git folder?

- a) Any developer, after testing their change
- b) Automation, after a pull request merges into the deployment branch
- c) The team lead, by editing notebooks there directly
- d) Nobody: production reads from developers' user folders

Why?

- r1) Production changes must pass review and CI, and automation makes that the only path.
- r2) Git folders cannot be edited by people.

Exercise 2. Fill the gap (level 2)

Read the supplier API key from the `research` secret scope at run time.

```
token = dbutils._____(1).get(scope="research", key="_____(2)")
```

Exercise 3. Spot the bug (level 3)

A reviewer blocks this pull request. Tap the line that must never be committed, then rewrite the module.

What you need to know.

`requests.get(url, headers={...}, timeout=30)` sends an HTTP GET request and returns a `requests.Response`, which holds the status code and the body. An `Authorization: Bearer <token>` header sends the caller's access token. `timeout` stops the call from waiting forever.

```
import requests

SUPPLIER_URL = "https://supplier.example.org/api/deliveries"
```

```
SUPPLIER_TOKEN = "example-token-not-real-5678"


def fetch_deliveries() -> requests.Response:
    """Fetch the delivery list from the supplier API."""
    return requests.get(SUPPLIER_URL, headers={"Authorization": f"Bearer {SUPPLIER_TOKEN}"}, timeout=30)
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Put in order (level 4)

Order the steps for one change to the silver pipeline.

- Push and open a pull request
- Create a feature branch from main in your Git folder
- CI runs lint, type checks and tests
- Merge to main; automation deploys to production
- A colleague reviews and approves
- Change the code and add or update tests

Exercise 5. Architecture design (level 5)

Decide where each item lives.

Options: Secret scope, read at run time, Git repository, Job parameter.

1. The supplier API token
2. The functions that clean and merge silver
3. Which catalog a run writes to (research_dev or research_prod)
4. Your local .venv folder

Sources

- Git folders: per-user branches under /Workspace/Users/, trunk-based strategy, production Git folders updated only by automation, bundles recommended for deployment. (verified: https://docs.databricks.com/aws/en/repos/ci-cd (checked 2026-09-25))
- dbutils.secrets.get(scope, key) reads a secret; values are shown as [REDACTED]; redaction does not prevent deliberate transformations. (verified: https://docs.databricks.com/aws/en/security/secrets/ (checked 2026-09-25))

A10-L2 Declarative Automation Bundles and parameterised jobs

Concept

A job built by clicking in the UI exists only in one workspace, with no history and no review. **Declarative Automation Bundles** describe jobs, their tasks, parameters and notifications as YAML files in your repository. Databricks renamed them from Databricks Asset Bundles on 16 March 2026. The `databricks bundle` commands and `databricks.yml` did not change.

A bundle has one `databricks.yml` at its root:

```yaml
bundle:
  name: research_pipelines

variables:
  catalog:
    description: Catalog the pipelines write to.

resources:
  jobs:
    nat_silver_pipeline:
      name: nat_silver_pipeline
      parameters:
        - name: catalog
          default: ${var.catalog}
      tasks:
        - task_key: silver
          notebook_task:
            notebook_path: ./notebooks/silver_nat_transform.py

targets:
  dev:
    mode: development
    default: true
    variables:
      catalog: research_dev
  prod:
    mode: production
    variables:
      catalog: research_prod
```

- **Targets** are environments. One code base deploys to each with different values.
- **Variables** hold what differs per target. They resolve from `--var`, then `BUNDLE_VAR_` environment variables, then target mappings, then defaults.

- **Job parameters** reach notebook tasks, which read them with `dbutils.widgets.get("catalog")`. Dynamic references such as `{{job.run_id}}` fill in values at run time.
- `mode: development` prefixes job names with `[dev <your name>]` and pauses schedules, so your test deployment never runs on the production timetable.

The commands:

```
databricks bundle validate -t dev
databricks bundle deploy -t dev
databricks bundle run -t dev nat_silver_pipeline
```

Deploy to `prod` only from CI, as a service principal (lesson A10-L3).

Why it matters

A team manages its jobs in the UI. Someone changes the production silver job's notebook path to test a branch, meaning to change it back. They forget. For a week, production runs a half-finished branch. Nobody can say what changed or when, because the UI keeps no diff and nothing was reviewed.

With the job in a bundle, the path is a line in `databricks.yml`. Changing it needs a pull request, CI validates it, and Git shows exactly what changed.

Worked example: weak code

```
resources:
  jobs:
    nat_silver_pipeline:
      name: nat_silver_pipeline
      tasks:
        - task_key: silver
          notebook_task:
            notebook_path: /Workspace/Users/sam.jones@example.org/
    silver_nat_transform
            base_parameters:
              catalog: research_prod
```

1. **Line 1.** No targets: there is no way to deploy the same job to dev.
2. **Line 8.** Points at one person's folder: unreviewed code, and it breaks when they leave.
3. **Line 10.** Production hardcoded: every deployment of this bundle writes to prod.

Worked example: strong code

```
bundle:
  name: research_pipelines

variables:
```

```yaml
  catalog:
    description: Catalog the pipelines write to.

resources:
  jobs:
    nat_silver_pipeline:
      name: nat_silver_pipeline
      parameters:
        - name: catalog
          default: ${var.catalog}
      tasks:
        - task_key: silver
          notebook_task:
            notebook_path: ./notebooks/silver_nat_transform.py
      email_notifications:
        on_failure:
          - data-oncall@example.org

targets:
  dev:
    mode: development
    default: true
    variables:
      catalog: research_dev
  prod:
    mode: production
    variables:
      catalog: research_prod
```

1. **Line 14.** The job parameter takes its value from the target's variable.
2. **Line 18.** A path inside the bundle, deployed from the reviewed commit.
3. **Line 25.** Dev deployments get a [dev name] prefix and paused schedules.
4. **Line 32.** Production values live only in the prod target.

One reviewed file describes the job for every environment. The target decides where it writes.

Recap card

- **What are Declarative Automation Bundles?** Jobs and their settings as YAML in Git (formerly Databricks Asset Bundles). Same CLI: databricks bundle.
- **Targets and variables** Targets are environments. Variables hold what differs, such as the catalog, set per target.
- **What does mode: development do?** Prefixes resources with [dev <user>] and pauses schedules and triggers.
- **Read a job parameter in a notebook** dbutils.widgets.get("name"). Validate it before using it.

Practice

Exercise 1. Multiple choice (level 1)

What changed when Databricks Asset Bundles became Declarative Automation Bundles on 16 March 2026?

- a) The CLI command changed from `databricks bundle` to `databricks dab`.
- b) The name. The `databricks bundle` CLI and existing configuration keep working unchanged.
- c) `databricks.yml` was renamed `automation.yml`.
- d) Targets were removed in favour of workspaces.

Why?

- r1) Databricks describes the rename as non-breaking.
- r2) Bundles were replaced by a new product.

Exercise 2. Fill the gap (level 2)

Complete the targets so dev deployments are prefixed and paused, and each target sets its own catalog.

```yaml
targets:
  dev:
    mode: _____(1)
    default: true
    variables:
      catalog: research_dev
  prod:
    mode: _____(2)
    variables:
      _____(3): research_prod
```

Exercise 3. Spot the bug (level 3)

A developer runs `databricks bundle deploy -t dev`, and the next morning production data has changed. Tap the line, then fix the job so the catalog comes from the target.

```yaml
resources:
  jobs:
    nat_silver_pipeline:
      name: nat_silver_pipeline
      parameters:
        - name: catalog
          default: research_prod
      tasks:
        - task_key: silver
          notebook_task:
            notebook_path: ./notebooks/silver_nat_transform.py
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 4)

The notebook for this job must read its `catalog` and `wave_id` job parameters. Write `read_job_parameters(widgets)`: `widgets` behaves like `dbutils.widgets` (it has `.get(name)`). Return a dict with both values. Raise `ValueError` when `catalog` is not `research_dev` or `research_prod`, or when `wave_id` is empty.

Your code must pass these checks:

- Return both parameters as read from widgets.get.
- An unexpected catalog must raise ValueError before any data moves.
- An empty wave_id must raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
ALLOWED_CATALOGS = ...

# Read and validate the job parameters passed to this notebook.
def read_job_parameters(widgets: object) -> dict[str, str]: ...
```

Starter code:

```python
def read_job_parameters(widgets: object) -> dict[str, str]:
    ...
```

Exercise 5. Put in order (level 5)

Order the commands to try a bundle change in dev.

- databricks bundle deploy -t dev
- databricks bundle run -t dev nat_silver_pipeline
- Check the run log and QA results for the dev run
- databricks bundle validate -t dev

Sources

- Databricks Asset Bundles were renamed Declarative Automation Bundles on 16 March 2026; the change is non-breaking. (verified: https://docs.databricks.com/aws/en/release-notes/dev-tools/bundles (checked 2026-09-25))
- databricks.yml structure: bundle, variables, resources.jobs with parameters and tasks, targets with mode; variable resolution order. (search-only: https://docs.databricks.com/aws/en/dev-tools/bundles/

settings and https://docs.databricks.com/aws/en/dev-tools/bundles/variables)

- Development mode prefixes resources with [dev ${workspace.current_user.short_name}] and pauses schedules and triggers. (verified: https://docs.databricks.com/aws/en/dev-tools/bundles/deployment-modes (checked 2026-09-25))
- Notebook tasks read job parameters with dbutils.widgets.get. (verified: https://docs.databricks.com/aws/en/jobs/parameter-use (checked 2026-09-25))

A10-L3 CI: lint, type-check, test and validate on every pull request

Concept

Continuous integration (CI) runs the same checks on every change, before a human reviews it. For a Databricks pipeline repository, run four checks on each pull request:

1. **Lint and format**: `ruff check .` and `ruff format --check .`
2. **Type-check**: `mypy`
3. **Unit tests**: `pytest`, with a local SparkSession (lesson A9-L2). Install Java in the CI runner first.
4. **Bundle validation**: `databricks bundle validate -t dev`. It catches YAML errors and broken references before anything is deployed.

On merge to `main`, a separate job deploys: `databricks bundle deploy -t prod`. Only CI deploys to production, and it does so as a **service principal**, never as a person.

Authenticating CI. Databricks recommends OAuth machine-to-machine authentication for a service principal. The CLI reads the workspace and credentials from environment variables, filled from the CI system's secret store:

```
env:
  DATABRICKS_HOST: ${{ vars.DATABRICKS_HOST }}
  DATABRICKS_CLIENT_ID: ${{ secrets.DATABRICKS_CLIENT_ID }}
  DATABRICKS_CLIENT_SECRET: ${{ secrets.DATABRICKS_CLIENT_SECRET }}
```

Workload identity federation goes a step further: with `DATABRICKS_AUTH_TYPE: github-oidc`, GitHub proves the workflow's identity, and no client secret is stored at all. Prefer it where your workspace supports it.

Two rules keep CI trustworthy:

- **CI must be able to fail.** A test step with `continue-on-error: true`, or a `|| true` after a command, reports success whatever happens.
- **Protect `main`.** Require the CI checks to pass before a pull request can merge. Otherwise CI only reports problems, and nothing stops them.

Install the CLI with the `databricks/setup-cli` action, and pin its version so a CLI release cannot change your pipeline's behaviour overnight.

Why it matters

A team adds CI but marks the test step `continue-on-error: true` "until the flaky tests are fixed". Six weeks later, every pull request shows a green tick. A change that breaks the silver MERGE key merges on a Friday. The failing test had run and failed, but the step's failure no longer failed the job. Production double-counts a wave over the weekend.

With the test step allowed to fail and `main` protected by required checks, the pull request could not have merged.

Worked example: weak code

```
1  on: push
2  jobs:
3    ci:
4      runs-on: ubuntu-latest
5      steps:
6        - uses: actions/checkout@v4
7        - run: pip install -r requirements.txt
8        - run: pytest
❶          continue-on-error: true
❷        - run: databricks bundle deploy -t prod
11         env:
❸            DATABRICKS_TOKEN: dapi-REDACTED-EXAMPLE
```

1. **Line 9.** Failing tests no longer fail the job.
2. **Line 10.** Deploys every push, on every branch, straight to production.
3. **Line 12.** A credential written into the workflow file: committed to Git.

Worked example: strong code

```
1  name: CI
2  on:
3    pull_request:
4    push:
5      branches: [main]
6
7  jobs:
8    checks:
```

```
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
      - run: pip install -e . --group test
      - run: ruff check . && ruff format --check .
      - run: mypy
      - run: pytest
      - uses: databricks/setup-cli@main
        with:
          version: 0.270.0
      - run: databricks bundle validate -t dev
        env:
          DATABRICKS_HOST: ${{ vars.DATABRICKS_HOST }}
          DATABRICKS_CLIENT_ID: ${{ secrets.DATABRICKS_CLIENT_ID }}
          DATABRICKS_CLIENT_SECRET: $
{{ secrets.DATABRICKS_CLIENT_SECRET }}

deploy:
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  needs: checks
  runs-on: ubuntu-24.04
  steps:
    - uses: actions/checkout@v4
    - uses: databricks/setup-cli@main
      with:
        version: 0.270.0
    - run: databricks bundle deploy -t prod
      env:
        DATABRICKS_HOST: ${{ vars.DATABRICKS_HOST }}
        DATABRICKS_CLIENT_ID: ${{ secrets.DATABRICKS_CLIENT_ID }}
        DATABRICKS_CLIENT_SECRET: $
{{ secrets.DATABRICKS_CLIENT_SECRET }}
```

1. **Line 20.** Every check can fail the job. Nothing is marked continue-on-error.
2. **Line 26.** Validates the bundle on every pull request, before review.
3. **Line 33.** Deploys only after a merge to main.
4. **Line 34.** Deployment waits for every check to pass.
5. **Line 41.** Deploys as the service principal, with credentials from the CI secret store.

Every pull request is linted, typed, tested and validated. Only a merge to main deploys, as a service principal, with nothing secret in the file. The CLI version shown is an example: pin the version you tested.

Recap card

- **Four CI checks for a pipeline repository** ruff (lint and format), mypy, pytest with local Spark, databricks bundle validate.
- **Who deploys to production?** CI, after a merge to main, as a service principal.
- **OAuth M2M environment variables** DATABRICKS_HOST, DATABRICKS_CLIENT_ID, DATABRICKS_CLIENT_SECRET from the CI secret store.
- **Two ways CI lies** continue-on-error: true, and command || true. Both turn failures green.

Practice

Exercise 1. Multiple choice (level 1)

Which identity should CI use to run `databricks bundle deploy -t prod`?

- a) The personal access token of the most senior engineer
- b) A service principal, authenticated with OAuth M2M or workload identity federation
- c) Whoever opened the pull request
- d) An anonymous connection

Why?

- r1) A service principal is not tied to a person, and its access can be scoped to exactly what deployment needs.
- r2) Personal tokens cannot deploy bundles.

Exercise 2. Fill the gap (level 2)

Complete the environment for the bundle deploy step, using OAuth M2M with values from the CI secret store.

```
env:
  DATABRICKS_HOST: ${{ vars.DATABRICKS_HOST }}
  DATABRICKS_____(1): ${{ secrets.DATABRICKS_CLIENT_ID }}
  DATABRICKS_____(2): ${{ secrets.DATABRICKS_CLIENT_SECRET }}
```

Exercise 3. Spot the bug (level 3)

Every pull request shows a green tick, including ones that broke the MERGE. Tap the line, then fix the step.

```
steps:
  - uses: actions/checkout@v4
  - run: pip install -e . --group dev
```

```
    - run: ruff check .
    - run: pytest
      continue-on-error: true
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Put in order (level 4)

Order the checks in the pull request job, cheapest and fastest first.

- ruff check . and ruff format --check .
- mypy
- databricks bundle validate -t dev
- pytest with a local SparkSession

Exercise 5. Architecture design (level 5)

Configure the repository and workflow. Choose each setting.

Options: On every pull request, and on push to main, Nightly, When someone remembers.

1. When do lint, types, tests and validation run?
2. When does the prod deployment run?
3. Branch protection on main
4. CI authentication

Sources

- Databricks recommends OAuth M2M for service principals in CI; environment variables DATABRICKS_HOST, DATABRICKS_CLIENT_ID and DATABRICKS_CLIENT_SECRET; github-oidc workload identity federation needs no client secret. (verified: https://docs.databricks.com/aws/en/dev-tools/auth/oauth-m2m and https://docs.databricks.com/aws/en/dev-tools/bundles/authentication (checked 2026-09-25))
- databricks/setup-cli installs the Databricks CLI in GitHub Actions and accepts a version input. (search-only: https://github.com/databricks/setup-cli)
- CLI version 0.270.0 in the example is illustrative, not a verified current release. (unverified: Pin the version you have tested; check https://github.com/databricks/cli/releases.)

Track A best-practice checklist

☐ Every function has type hints, a docstring and one job.

☐ No SparkSession is created at import time. Functions receive spark as a parameter.

☐ Functions are imported as F: `from pyspark.sql import functions as F`.

☐ Table names, column maps and thresholds come from config or registry tables.

☐ Every table states its grain in a comment or table property.

☐ Bronze appends. Silver deduplicates the source, then MERGEs. Gold aggregates by every grain column.

☐ Every run writes one record to the run log table with an explicit schema.

☐ CRITICAL checks raise. WARN checks log and continue.

☐ Every division has a guard for zero and null.

☐ Counts come from one agg pass, not repeated count() calls.

☐ Incremental loads compare source and target on the same keys.

☐ Row filters use a SQL UDF with SET ROW FILTER. Grants go to groups.

☐ Tests run on a local SparkSession in CI before every merge.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

A0-L1 Spark DataFrames, the model and the core methods

A0-L1 exercise 1

Hints. (1) Spark is lazy. (2) Which call returns a result to Python instead of a new DataFrame? (3) count returns a number.

Answer. c) `totals.count()` Because: r1) count() is an action. The other lines are transformations that only describe the plan.

Transformations build a plan. An action such as `count()`, `show()`, `collect()` or a write runs it.

Common wrong answers.

- `a`: filter is a transformation. It returns a new DataFrame and runs nothing yet.
- `b`: groupBy and agg are transformations too. Spark waits for an action.
- `d`: orderBy is a transformation. The plan grows, nothing runs.
- `c|r2`: Right line, wrong reason. Position does not matter. count() is an action.

A0-L1 exercise 2

Hints. (1) Apply the filter first. (2) Which rows have w above 1? (3) Then add up w per region.

Answer.

```
1 [('N', 2.0), ('S', 4.0)]
```

The filter keeps 2.0 for N and 4.0 for S, the groupBy sums them, and orderBy puts N first.

Common wrong answers.

- `[('N', 3.0), ('S', 4.0)]`: The filter runs first: w > 1 drops the row with 1.0, so N totals 2.0.
- `[('S', 4.0), ('N', 2.0)]`: orderBy("region") sorts N before S.

A0-L1 exercise 3

Hints. (1) The method that follows groupBy. (2) The function that adds values. (3) `agg` and `sum`.

Answer.

```
totals = df.filter(F.col("w") > 0).groupBy("region").agg(F.sum("w").alias("total"))
```

`groupBy(...).agg(...)` summarises each group. `F.sum("w")` adds the weights, and `alias` names the result.

Common wrong answers.

- g1 = `agg`; g2 = `count`: F.count counts rows. The task asks for the total weight: F.sum.
- g1 = `select`; g2 = `sum`: After groupBy, summarise with agg.

A0-L1 exercise 4

Hints. (1) Filter with `F.col("in_scope") & (F.col("w") > 0)`, brackets around the comparison. (2) Then `.groupBy("region")`. (3) `.agg(F.count("*").alias("n"), F.sum("w").alias("total_weight"))`.

Answer.

```
from pyspark.sql import DataFrame
from pyspark.sql import functions as F


def region_totals(df: DataFrame) -> DataFrame:
    """Row count and weight total per region, for in-scope rows with a positive weight.

    Args:
        df: Respondents with region, in_scope and w columns.

    Returns:
        One row per region with n and total_weight.
    """
    return (
        df.filter(F.col("in_scope") & (F.col("w") > 0))
        .groupBy("region")
        .agg(F.count("*").alias("n"), F.sum("w").alias("total_weight"))
    )
```

One chain of transformations: filter, group, aggregate. The cluster does the work, and the function returns a DataFrame for the caller to write or show.

A0-L1 exercise 5

Hints. (1) What does withColumn return? (2) DataFrames never change in place. (3) `return df.withColumn(...)`.

Answer. The bug is on line 3.

Fixed version:

```
1 def add_weight_band(df: DataFrame) -> DataFrame:
2     """Add weight_band: high above 3, otherwise normal."""
3     return df.withColumn("weight_band", F.when(F.col("w") > 3,
    "high").otherwise("normal"))
```

`withColumn` returns a new DataFrame. Line 3 discards it, so the function returns the original. Return the new DataFrame, or assign it back to `df`.

Common wrong answers.

- Line 4: Returning df is right once df holds the new DataFrame. Which line throws the new DataFrame away?

A0-L2 The SQL that pipelines use

A0-L2 exercise 1

Hints. (1) What does comparing anything with NULL give? (2) Unknown, never true. (3) SQL has a dedicated test.

Answer. b) `WHERE final_weight IS NULL` Because: r1) A comparison with NULL is unknown, never true. IS NULL is the test made for it.

NULL means unknown, so `= NULL` is never true. `IS NULL` matches missing values.

Common wrong answers.

- `a`: = NULL is never true, so it matches no rows, even missing ones.
- `c`: 0 is a real weight, not a missing one.
- `d`: NOT on a NULL is still unknown, so missing rows are not kept.
- `b|r2`: Right condition, wrong reason. It is about correctness: = NULL never matches.

A0-L2 exercise 2

Hints. (1) What does COUNT(*) count? (2) What does COUNT(column) skip? (3) Rows against non-NULL values.

Answer.

```
1 3 2
```

`COUNT(*)` counts all three rows. `COUNT(final_weight)` counts the two non-NULL weights.

Common wrong answers.

- `3 3`: COUNT(column) counts only non-NULL values, so it skips the missing weight.
- `2 2`: COUNT(*) counts rows, whatever their values.

A0-L2 exercise 3

Hints. (1) A join that keeps left rows with no match. (2) Joins name their condition after one keyword. (3) `ANTI` and `ON`.

Answer.

```
SELECT r.respondent_id, r.region_code
FROM respondents r
LEFT ANTI JOIN regions g
  ON r.region_code = g.region_code;
```

`LEFT ANTI JOIN ... ON` returns the left rows with no match on the right: the orphan codes.

Common wrong answers.

- g1 = `OUTER`; g2 = `ON`: A LEFT OUTER JOIN keeps every respondent. An anti join keeps only those with no match.
- g1 = `SEMI`; g2 = `ON`: A semi join keeps rows that do match. You want the ones that do not.

A0-L2 exercise 4

Hints. (1) Put the numbering in a subquery: `SELECT *, ROW_NUMBER() OVER (...) AS rn FROM bronze_deliveries`. (2) Inside OVER: `PARTITION BY respondent_id, wave_id ORDER BY delivery_version DESC`. (3) Outside, select the four columns and keep `WHERE rn = 1`.

Answer.

```
SELECT respondent_id, wave_id, delivery_version, final_weight
FROM (
  SELECT *,
         ROW_NUMBER() OVER (PARTITION BY respondent_id, wave_id ORDER BY
    delivery_version DESC) AS rn
  FROM bronze_deliveries
)
WHERE rn = 1;
```

`ROW_NUMBER()` numbers each respondent's versions within a wave, newest first, and `WHERE rn = 1` keeps one row per key. Without `DESC` you keep the

oldest version, and without `wave_id` you keep one row per respondent across all waves.

A0-L2 exercise 5

Hints. (1) Which rows does the WHERE clause keep? (2) Is any comparison with NULL ever true? (3) `WHERE final_weight IS NULL`.

Answer. The bug is on line 3.

Fixed version:

```
1 SELECT COUNT(*) AS missing_weights
2 FROM research_prod.research_silver.nat_respondents
3 WHERE final_weight IS NULL;
```

`= NULL` is unknown for every row, so no row passes the filter and the count is always 0. `IS NULL` counts the missing weights.

Common wrong answers.

- Line 1: COUNT(*) counts the rows the WHERE clause keeps. The WHERE clause keeps none.
- Line 2: The table name is a valid three-level name.

A0-L3 The lakehouse map, Databricks, Unity Catalog, Delta and medallion layers

A0-L3 exercise 1

Hints. (1) Three parts, three levels. (2) catalog.schema.table. (3) The middle one.

Answer. b) The schema Because: r1) Unity Catalog names run catalog.schema.table, so the middle part is the schema.

Unity Catalog names are `catalog.schema.table`. Here the catalog is the environment, the schema is the layer and the table is the tracker's silver table.

Common wrong answers.

- `a`: The catalog comes first: research_prod, the production environment.
- `c`: The table comes last: nat_respondents.
- `d`: A workspace is where you work. It is not part of a table name.
- `b|r2`: Right part, wrong reason. Position decides: catalog.schema.table.

A0-L3 exercise 2

Hints. (1) Each statement that changes the table makes one version. (2) What did the table hold at version 1? (3) Only the first insert had run.

Answer.

```
1
```

Version 1 is the table right after the first insert, so it holds the single row with id 1.

Common wrong answers.

- `1 2`: Version 1 is the table after the first insert. The second insert made version 2.
- `2`: VERSION AS OF 1 reads the table as it was after the first insert, which held only 1.

A0-L3 exercise 3

Hints. (1) Bronze holds the deliveries as received. (2) Silver holds one row per respondent per wave. (3) `raw`, `respondents`, `metrics`.

Answer.

```
bronze: research_prod.research_bronze.nat_raw
silver: research_prod.research_silver.nat_respondents
gold:   research_prod.research_gold.nat_metrics
```

The convention is `[tracker]_raw`, `[tracker]_respondents` and `[tracker]_metrics`, so a table's name says what it holds and the schema says its layer.

Common wrong answers.

- g1 = `bronze`; g2 = `silver`; g3 = `gold`: The layer is already in the schema name. The table name says what the table holds: raw, respondents, metrics.

A0-L3 exercise 4

Hints. (1) `parts = name.split(".")`. (2) Check `len(parts) != 3` and whether any part is an empty string. (3) Put the name in the message: `f"expected catalog.schema.table, got {name!r}"`.

Answer.

```
def parse_table_name(name: str) -> tuple[str, str, str]:
    """Split a Unity Catalog table name into catalog, schema and table.

    Args:
```

```
        name: A name such as
    research_prod.research_silver.nat_respondents.

    Returns:
        (catalog, schema, table).

    Raises:
        ValueError: If the name does not have exactly three non-empty
    parts.
    """
    parts = name.split(".")
    if len(parts) != 3 or any(part == "" for part in parts):
        raise ValueError(f"expected catalog.schema.table, got {name!r}")
    catalog, schema, table = parts
    return catalog, schema, table
```

A job that refuses anything but a full, three-part name can never write to whatever catalog happens to be the default.

A0-L3 exercise 5

Hints. (1) A file is not a table. (2) As received, then clean, then aggregated. (3) Volume, bronze, silver, gold.

Answer.

- Files as sent: a volume.
- Rows as received, with audit columns: bronze.
- Clean, deduplicated respondents: silver.
- Business-ready aggregates: gold.

Files sit in a volume, and each table layer has one job: bronze as received, silver clean, gold aggregated.

Common wrong answers.

- raw_rows = `silver`: Silver keeps one row per respondent per wave. Every delivered row, duplicates included, belongs in bronze.
- clean = `gold`: Gold holds aggregates. Clean respondent-level rows are silver.

A1-L1 Single-purpose functions and clear names

A1-L1 exercise 1

Hints. (1) Look at the naming table in the concept. (2) Classes and constants have their own styles. (3) Functions: lower case, words joined by underscores, verb first.

Answer. b) `check_table_health` Because: r1) Functions use snake_case and start with a verb.

Functions use `verb_noun` in snake_case. PascalCase is for classes and UPPER_SNAKE_CASE for constants, so the style alone tells a reader what kind of object they are looking at.

Common wrong answers.

- `a`: PascalCase marks a class, such as `CheckResult`. A function uses snake_case.
- `c`: camelCase is the Java and JavaScript convention. PEP 8 and your standard use snake_case for functions.
- `d`: UPPER_SNAKE_CASE marks a constant, such as `MAX_WEIGHT_CV`.
- `b|r2`: Right name, wrong reason. Name length has no effect on speed. The convention signals what kind of object it is.

A1-L1 exercise 2

Hints. (1) Tables follow `[tracker]_[content]`. (2) Bronze holds raw deliveries, silver holds respondents, gold holds metrics. (3) The job pattern is `[tracker]_[layer]_pipeline`.

Answer.

```
1 BRONZE_TABLE = "research_bronze.nat_raw"
2 SILVER_TABLE = "research_silver.nat_respondents"
3 GOLD_TABLE = "research_gold.nat_metrics"
4 JOB_NAME = "nat_silver_pipeline"
```

`nat_raw`, `nat_respondents` and `nat_metrics` name what each table holds. The schema carries the layer. Jobs use `nat_silver_pipeline`.

Common wrong answers.

- g1 = `nat_bronze`: The layer is already in the schema name, `research_bronze`. The table name says what it holds: `nat_raw`.
- g4 = `silver_nat_pipeline`: Jobs start with the tracker: `[tracker]_[layer]_pipeline`. Notebooks start with the layer.
- g2 = `nat_silver`: Silver tables are named after their grain: one row per respondent, so `nat_respondents`.

A1-L1 exercise 3

Hints. (1) What does the word `get` promise the caller? (2) Which line changes a table? (3) Delete line 6. Deleting data belongs in its own, clearly named function.

Answer. The bug is on line 6.

Fixed version:

```
def get_unprocessed_waves(spark: SparkSession, bronze: str, silver: str,
    tracker_id: str) -> list[str]:
     """Return wave ids in bronze but not yet in silver for one tracker."""
     bronze_waves = spark.table(bronze).filter(F.col("tracker_id") ==
    tracker_id).select("wave_id").distinct()
     silver_waves = spark.table(silver).filter(F.col("tracker_id") ==
    tracker_id).select("wave_id").distinct()
     waves = [row.wave_id for row in
    bronze_waves.subtract(silver_waves).collect()]
     return waves
```

Line 6 deletes rows from silver inside a function named `get_`. A caller who only wants the list of waves loses data. Remove the line. If you need that clean-up, give it its own function, such as `delete_waves()`, and call it from the orchestrator.

Common wrong answers.

- Line 5: `collect()` on a short list of wave ids is fine. Look for a line that changes data.
- Line 4: Filtering silver by tracker is correct. It fixes a known defect in the reference code.

A1-L1 exercise 4

Hints. (1) Each function takes `df: DataFrame` and returns a DataFrame. (2) Move line 3 into `filter_in_scope` and line 4 into `aggregate_base`. (3) Drop `== True`, and leave `spark.table` and `.write` out of both functions.

Answer.

```
from pyspark.sql import DataFrame
from pyspark.sql import functions as F


def filter_in_scope(df: DataFrame) -> DataFrame:
    """Keep respondents flagged as in scope."""
    return df.filter(F.col("in_scope"))


def aggregate_base(df: DataFrame) -> DataFrame:
    """Weighted base at wave x region grain."""
    return df.groupBy("wave_id",
   "region").agg(F.sum("final_weight").alias("weighted_base"))
```

Pure transforms take a DataFrame and return one. Reading and writing stay in the orchestrator, so the transforms run in a unit test with a handful of rows.

A1-L1 exercise 5

Hints. (1) Values that change between environments are config. (2) Anything that touches the catalog is a read or a write. (3) The orchestrator only sequences the other functions.

Answer.

- Table names are configuration. Load them once and pass them in.
- `spark.table(...)` is a read. It takes the session and the table name as parameters.
- Summing weights is a pure transform.
- The orchestrator calls each step and logs the run. It holds no business logic.

Config holds names and thresholds. Reads touch the catalog. Transforms turn one DataFrame into another. The orchestrator calls them in order and logs the outcome.

Common wrong answers.

- table_name = `orchestrator`: A table name typed into the orchestrator is still hardcoded. Put it in config, so dev and prod runs point at different tables.
- weights = `orchestrator`: If the orchestrator holds the aggregation, you cannot test it without running the whole pipeline. Make it a transform.
- spark_table = `transform`: A transform receives a DataFrame. Reading a table needs a session and a catalog, which is the read function's job.

A1-L2 Type hints and docstrings a reviewer can trust

A1-L2 exercise 1

Hints. (1) The default is None. Does the hint allow None? (2) Python 3.10 added the `X | Y` union syntax. (3) `str | None = None`.

Answer. b) `def log_run(error_message: str | None = None) -> None:` Because: r1) The hint must include None, because None is a value the parameter takes.

`str | None = None` says the parameter is a string or None, and defaults to None. Python ignores hints at run time. mypy and readers rely on them.

Common wrong answers.

- `a`: `str = None` claims the value is always a string. mypy strict mode rejects the implicit Optional.

- `c`: No hint at all. mypy --strict rejects unannotated parameters.
- `d`: This makes the type `None` and the default the class `str`. It is backwards.
- `b|r2`: Right signature, wrong reason. Python ignores hints at run time. They exist for tools and readers.

A1-L2 exercise 2

Hints. (1) What does `for k in "id"` loop over? (2) A string is a sequence of characters. (3) Two characters, two conditions, joined by AND.

Answer.

```
1 t.i = s.i AND t.d = s.d
```

A string is iterable, so the loop yields `i` and `d`. The condition names two columns that do not exist. With `merge_keys: list[str]`, mypy flags the call before it runs.

Common wrong answers.

- `t.id = s.id`: That is what the caller intended. Iterating a string yields its characters, `i` then `d`.
- `TypeError`: Python raises no error. A string is iterable, which is why the bug is silent until Spark runs the MERGE.

A1-L2 exercise 3

Hints. (1) The type for a fixed set of values lives in `typing`. (2) Its name means the values are written out literally. (3) `Literal`, and `None` for the return type.

Answer.

```
1 from typing import Literal
2
3 def record_check(name: str, status: Literal["PASS", "WARN", "FAIL"]) ->
    None:
4     ...
```

`Literal["PASS", "WARN", "FAIL"]` restricts the value to those three strings. `-> None` states the function returns nothing.

Common wrong answers.

- g1b = `Enum`: An Enum is a class you define. Inline string choices use `Literal[...]` from `typing`.
- g2 = `void`: Python has no `void`. A function that returns nothing is annotated `-> None`.

- g1b = `str`: `str` accepts any string, so `"pass"` or `"FAILED"` would slip through. `Literal` lists the allowed values.

A1-L2 exercise 4

Hints. (1) Check the type first with `isinstance(keys, str)`, then check for an empty list. (2) Build each clause with an f-string that uses `target` and `source`. (3) `" AND ".join(f"{target}.{k} = {source}.{k}" for k in keys)`.

Answer.

```python
def build_merge_condition(keys: list[str], target: str = "t", source: str = "s") -> str:
    """Build a MERGE ON condition that matches every key column.

    Args:
        keys: Columns that identify one target row.
        target: Alias of the target table.
        source: Alias of the source DataFrame.

    Returns:
        The condition, for example "t.id = s.id AND t.wave = s.wave".

    Raises:
        TypeError: If keys is a string instead of a list.
        ValueError: If keys is empty.
    """
    if isinstance(keys, str):
        raise TypeError("keys must be a list of column names, not a string")
    if not keys:
        raise ValueError("keys must name at least one column")
    return " AND ".join(f"{target}.{k} = {source}.{k}" for k in keys)
```

The hint `list[str]` lets mypy catch a string at the call site. The run-time `TypeError` catches callers that skip mypy, such as a notebook. The `ValueError` stops an empty condition, which would match nothing and insert every row.

A1-L2 exercise 5

Hints. (1) Import `Literal` from `typing` and name the return type. (2) `change` is `float | None`. `tolerance` is `float`. (3) Keep `if change is None:`. Truthiness would turn a change of 0.0 into SKIP.

Answer.

```python
from typing import Literal

ChangeStatus = Literal["SKIP", "WARN", "PASS"]


```

```python
def classify_change(change: float | None, tolerance: float = 0.10) -> ChangeStatus:
    """Classify a wave-over-wave change in a share.

    Args:
        change: Absolute change in the share, or None when a share is missing.
        tolerance: Largest change that still passes.

    Returns:
        SKIP when change is None, WARN above tolerance, otherwise PASS.
    """
    if change is None:
        return "SKIP"
    return "WARN" if change > tolerance else "PASS"
```

The Literal return type means mypy rejects a typo such as `\"PASSED\"` at every caller. `float | None` records that a change can be missing. The `is None` test keeps a real change of 0.0 as PASS.

A1-L3 Config over hardcoded values, and PEP 8 with ruff

A1-L3 exercise 1

Hints. (1) What changes when the same code runs in dev and in prod? (2) Logic stays the same. Names and limits change. (3) The catalog differs between environments.

Answer. b) The catalog name, `research_prod` or `research_dev` Because: r1) It changes between environments while the logic stays the same.

The catalog name varies by environment. The formula, the step order and the write strategy are logic, which belongs in reviewed, tested code.

Common wrong answers.

- `a`: The formula is business logic. It is the same in every environment, so it belongs in a tested function.
- `c`: The step order is the pipeline's design. It lives in the orchestrator and changes only with a code review.
- `d`: MERGE versus append is a design decision about idempotence, not a setting.
- `b|r2`: Right choice, wrong reason. Length is irrelevant. Configuration holds what varies between environments, trackers or contract versions.

A1-L3 exercise 2

Hints. (1) What does `frozen=True` do to attribute assignment? (2) The exception fires before the value changes. (3) Two lines are printed.

Answer.

```
1 blocked
2 research_dev
```

`frozen=True` makes assignment raise `FrozenInstanceError`, so `blocked` prints and the catalog stays `research_dev`. No code can switch a run to production halfway through.

Common wrong answers.

- `research_prod`: A frozen dataclass rejects assignment after creation, so the value stays `research_dev`.
- `blocked research_prod`: The assignment raised before it changed anything. The value stays `research_dev`.

A1-L3 exercise 3

Hints. (1) One command lints, one formats. (2) The lint command's name means inspecting code. (3) `ruff check .` and `ruff format --check .`.

Answer.

```
1 ruff check .
2 ruff format --check .
```

`ruff check .` reports lint errors such as F821 and E712. `ruff format --check .` exits non-zero when any file would be reformatted, which fails CI without touching the code.

Common wrong answers.

- g1 = `lint`: ruff has no `lint` command. Linting runs with `ruff check`.
- g2 = `fmt`: The formatter command is `ruff format`. Add `--check` so CI reports instead of rewriting.
- g1 = `format`; g2 = `check`: Swapped. `ruff check` lints, `ruff format --check` verifies formatting.

A1-L3 exercise 4

Hints. (1) Declare three fields on a frozen dataclass, then add a `@property` for the table name. (2) Index the dict with `raw["key"]`. `.get` with a default hides a missing key. (3) Check `0 <= tolerance <= 1` before you build the object.

Answer.

```python
from dataclasses import dataclass


@dataclass(frozen=True)
class RunConfig:
    """Settings for one pipeline run."""

    catalog: str
    tracker: str
    stability_tolerance: float

    @property
    def silver_table(self) -> str:
        """Three-level name of the silver table."""
        return f"{self.catalog}.research_silver.{self.tracker}
    _respondents"


def load_run_config(raw: dict[str, object]) -> RunConfig:
    """Build a validated RunConfig from parsed JSON.

    Args:
        raw: Parsed configuration.

    Returns:
        The frozen configuration.

    Raises:
        KeyError: If a required key is missing.
        ValueError: If stability_tolerance is outside 0 to 1.
    """
    tolerance = float(str(raw["stability_tolerance"]))
    if not 0 <= tolerance <= 1:
        raise ValueError(f"stability_tolerance
    {tolerance} is outside 0 to 1")
    return RunConfig(
        catalog=str(raw["catalog"]),
        tracker=str(raw["tracker"]),
        stability_tolerance=tolerance,
    )
```

Indexing with `raw[\"key\"]` fails at load with the key's name. Defaults such as `raw.get(\"catalog\", \"research_prod\")` are worse than no config: a missing value silently points the run at production.

A1-L3 exercise 5

Hints. (1) Search the function for a catalog name. (2) Every table name must come from `cfg`. (3) Add `scope_table` to RunConfig and read `cfg.scope_table`.

Answer. The bug is on line 4.

Fixed version:

```
1 def weighted_base_by_wave(spark: SparkSession, cfg: RunConfig) ->
    DataFrame:
2     """Weighted base per wave from the configured silver table."""
3     silver = spark.table(cfg.silver_table)
4     scope = spark.table(cfg.scope_table)
5     in_scope = silver.join(scope, "region").filter(F.col("in_scope"))
6     return
    in_scope.groupBy("wave_id").agg(F.sum("final_weight").alias("weighted_base"))
```

Line 4 hardcodes a production table, so every environment reads production scope rules. Add `scope_table` to the config and read it from `cfg`.

Common wrong answers.

- Line 3: Line 3 reads the silver table from config, which is correct. Look for a name typed into the code.
- Line 5: The join and filter are fine. The problem is which table the join reads.

A2-L1 Bronze: keep every delivery exactly as received

A2-L1 exercise 1

Hints. (1) What is bronze for? (2) Which layer cleans and validates? (3) Bronze keeps everything, labelled with where it came from.

Answer. b) Append them as received, with the audit columns. Because: r1) Bronze records deliveries as received. Silver and the data contract decide what to do with bad rows.

Bronze records every delivery as received. The contract check flags the null weights, and silver decides how to handle them. Nothing is lost.

Common wrong answers.

- `a`: Dropping them in bronze destroys the evidence. You can no longer tell the supplier which rows were wrong.
- `c`: Imputing a weight in bronze invents data. The weighted totals would look plausible and be wrong.
- `d`: Accept or reject is the contract check's decision, after the delivery is recorded. Bronze records first.
- `b|r2`: Right action, wrong reason. Delta columns accept nulls unless you add a NOT NULL constraint.

A2-L1 exercise 2

Hints. (1) What type does inferSchema pick for a column of digits? (2) Integers have no leading zeros. (3) The value prints as an int.

Answer.

```
123
```

`inferSchema` reads the column as an integer, so `00123` becomes `123`. Read bronze without inferSchema and every column stays a string, zeros included.

Common wrong answers.

- `00123`: That is what you get without inferSchema. With it, Spark sees digits, picks an integer type, and drops the leading zeros.
- `'00123'`: The value is no longer a string. inferSchema made the column an integer.

A2-L1 exercise 3

Hints. (1) Bronze never removes what is already there. (2) The timestamp function's name describes the current moment. (3) `current_timestamp` and `append`.

Answer.

```
audited = raw.withColumn("ingestion_timestamp", F.current_timestamp())
audited.write.format("delta").mode("append").saveAsTable(table)
```

`F.current_timestamp()` stamps each load. `mode("append")` adds rows and leaves earlier deliveries untouched.

Common wrong answers.

- g2 = `overwrite`: Overwrite deletes every earlier delivery. Bronze is append-only.
- g1 = `now`: There is no `F.now`. The PySpark function is `F.current_timestamp()`.
- g2 = `merge`: `merge` is not a write mode, and bronze never updates rows. Use `append`.

A2-L1 exercise 4

Hints. (1) `df.withColumns({...})` adds several columns in one call (PySpark 3.3 and later). (2) Constants go through `F.lit()`. The timestamp comes from `F.current_timestamp()`. (3) Add columns only. No filter, no cast.

Answer.

```
from pyspark.sql import DataFrame
from pyspark.sql import functions as F
```

```
3
4
5  def add_audit_columns(df: DataFrame, ingestion_id: str, source_file: str,
       delivery_version: int) -> DataFrame:
6      """Add the four bronze audit columns to a delivery.
7
8      Args:
9          df: The delivery as read, all columns as strings.
10         ingestion_id: One UUID for this load.
11         source_file: Path of the delivery file.
12         delivery_version: Supplier's version number for the delivery.
13
14     Returns:
15         The delivery with ingestion_id, source_file, ingestion_timestamp
       and delivery_version.
16     """
17     return df.withColumns({
18         "ingestion_id": F.lit(ingestion_id),
19         "source_file": F.lit(source_file),
20         "ingestion_timestamp": F.current_timestamp(),
21         "delivery_version": F.lit(delivery_version),
22     })
```

`withColumns` adds all four audit columns in one step. Everything else stays as delivered: no filter, no cast.

A2-L1 exercise 5

Hints. (1) What happens to existing rows when this job writes? (2) One word on line 5 decides it. (3) Use `mode("append")`.

Answer. The bug is on line 5.

Fixed version:

```
1  def ingest_to_bronze(spark: SparkSession, path: str, table: str,
       delivery_version: int) -> None:
2      """Append one delivery file to bronze as received."""
3      raw = spark.read.option("header", True).csv(path)
4      audited = add_audit_columns(raw, str(uuid.uuid4()), path,
       delivery_version)
5      audited.write.format("delta").mode("append").saveAsTable(table)
```

`overwrite` replaces the table, so each delivery erases the one before. Bronze must append.

Common wrong answers.

- Line 3: Reading without inferSchema is correct for bronze. Look at how the rows are written.
- Line 4: A new UUID per load is correct: each delivery gets its own ingestion_id.

A2-L1 exercise 6

Hints. (1) Bronze never changes what the supplier sent. (2) Silver has one clean row per respondent per wave. (3) Gold holds aggregates.

Answer.

- Bronze stores deliveries as received.
- Silver renames, casts and resolves versions, so it holds one row per respondent per wave.
- Gold aggregates for reporting, with the grain stated.

Bronze keeps evidence, silver makes it clean and unique, gold makes it ready to report.

Common wrong answers.

- cast = `bronze`: Casting in bronze loses the original value when the cast fails or guesses wrong. Cast in silver, with checks.
- dedupe = `bronze`: Bronze keeps every delivery, duplicates included. Silver picks the newest version.
- reach = `silver`: Silver holds respondent-level rows. Aggregates for reporting belong in gold.

A2-L2 Silver: canonical names, checked types, one row per respondent per wave

A2-L2 exercise 1

Hints. (1) Does the frame have a column called `wgt`? (2) What does withColumnRenamed do when the source column is missing? (3) It returns the frame unchanged.

Answer.

```
1 ['respondent_id', 'weight']
```

`withColumnRenamed` is a no-op when the source column is missing. No exception, no warning. Check the columns yourself before you rename.

Common wrong answers.

- `['respondent_id', 'final_weight']`: There is no column called `wgt`, so nothing is renamed. `weight` keeps its name.
- `AnalysisException`: No error. withColumnRenamed silently ignores a missing column, which is exactly the danger.

A2-L2 exercise 2

Hints. (1) Which Spark version does Runtime 17.3 LTS use? (2) What changed about casts in Spark 4.0? (3) ANSI mode is on by default.

Answer. b) The job fails with CAST_INVALID_INPUT. Because: r1) Spark 4.0 enables ANSI mode by default, and ANSI casts raise on malformed input.

Spark 4.0 turns ANSI mode on, so a malformed cast raises `CAST_INVALID_INPUT`. `try_cast` returns NULL instead, which you then count.

Common wrong answers.

- `a`: That was the behaviour before ANSI mode became the default. On Spark 4.0 use `try_cast` to get NULL.
- `c`: Casting never drops rows. It either raises or returns a value.
- `d`: Spark never turns malformed text into 0.0. It raises, or with try_cast returns NULL.
- `b|r2`: Right outcome, wrong reason. Delta accepts NULLs unless a NOT NULL constraint says otherwise. The error comes from the ANSI cast.

A2-L2 exercise 3

Hints. (1) The safe variant of cast starts with a word meaning attempt. (2) It exists as a SQL function and, since PySpark 4.0, as a Column method. (3) `try_cast`.

Answer.

```
df = df.withColumns({c: F.col(c).try_cast(t) for c, t in
    type_map.items()})
```

`Column.try_cast` returns NULL where the value cannot be cast. Count those NULLs and let a data-quality check decide what to do.

Common wrong answers.

- g1 = `cast`: Under ANSI mode on Spark 4.0, `cast` raises on the first bad value. You need the variant that returns NULL.
- g1 = `astype`: `astype` is an alias of `cast`, so it raises under ANSI mode too.

A2-L2 exercise 4

Hints. (1) Find the missing columns with a set difference. (2) `missing = sorted(set(column_map) - set(df.columns))`. (3) Raise if `missing` is not empty, then `return df.withColumnsRenamed(column_map)`.

Answer.

```
from pyspark.sql import DataFrame

```

```
3
4  def rename_to_canonical(df: DataFrame, column_map: dict[str, str]) ->
       DataFrame:
5      """Rename supplier columns to registry names.
6
7      Args:
8          df: A bronze delivery.
9          column_map: Supplier name to canonical name.
10
11     Returns:
12         The frame with canonical names.
13
14     Raises:
15         ValueError: If any source column in column_map is missing from df.
16     """
17     missing = sorted(set(column_map) - set(df.columns))
18     if missing:
19         raise ValueError(f"missing source columns: {missing}")
20     return df.withColumnsRenamed(column_map)
```

The set difference lists every missing source column in one go. Only then is the rename safe, because `withColumnsRenamed` would skip missing columns without telling you.

A2-L2 exercise 5

Hints. (1) What is silver's grain? (2) Which columns does the deduplication treat as the key? (3) The key must include wave_id.

Answer. The bug is on line 5.

Fixed version:

```
1  def prepare_silver(df: DataFrame, column_map: dict[str, str], type_map:
       dict[str, str]) -> DataFrame:
2      """Rename, cast and keep the newest version per respondent and
       wave."""
3      renamed = rename_to_canonical(df, column_map)
4      typed = cast_to_registry_types(renamed, type_map)
5      return latest_per_key(typed, ["respondent_id", "wave_id"],
       "delivery_version")
```

Deduplicating on `respondent_id` alone keeps one wave per respondent and throws the others away. The key must match the grain: respondent and wave.

Common wrong answers.

- Line 3: The rename is checked and correct. Look at which keys the deduplication uses.
- Line 4: try_cast is the right choice for silver. The rows vanish in a later step.

A2-L2 exercise 6

Hints. (1) Checks on source names come before anything renames them. (2) A CRITICAL result must stop the run before the write. (3) The MERGE needs a source with one row per key.

Answer.

1. Check that every mapped source column exists
2. Rename to canonical names
3. try_cast to registry types
4. Count cast failures and apply the WARN or CRITICAL threshold
5. Keep the newest version per respondent and wave
6. MERGE into silver

Check, rename, cast, count failures, deduplicate, merge. Each step depends on the one before, and nothing reaches silver until the checks pass.

Common wrong answers.

- After the rename, the source names are gone. Check the source columns first.
- Count cast failures before the MERGE. A CRITICAL failure must stop bad data reaching silver.
- Deduplicating after the MERGE is too late. A repeated key fails the MERGE, or inserts twice when the target has no match.

A2-L3 Gold: state the grain, group by all of it, guard every ratio

A2-L3 exercise 1

Hints. (1) List the grain columns in the docstring. (2) Compare them with the groupBy. (3) tracker_id is missing.

Answer. The bug is on line 3.

Fixed version:

```
1 def base_by_subgroup(df: DataFrame) -> DataFrame:
2     """Weighted base at tracker x wave x region grain."""
3     return df.groupBy("tracker_id", "wave_id",
    "region").agg(F.sum("final_weight").alias("weighted_base"))
```

The grain names three columns, and the `groupBy` has two. Without `tracker_id`, rows from different trackers fold into one.

Common wrong answers.

- Line 2: The docstring is the contract, and it is right. The code has to match it.

A2-L3 exercise 2

Hints. (1) What does `F.when(cond, 1)` return when `cond` is false? (2) What is the sum of a column that holds only NULL? (3) NULL prints as None in Python.

Answer.

```
None
```

`F.when` without `otherwise` yields NULL for the unmatched row, and the sum of only NULLs is NULL. Power BI shows a blank instead of 0. Wrap the sum in `F.coalesce(..., F.lit(0))`.

Common wrong answers.

- `0`: `F.when` without `otherwise` returns NULL for non-matching rows, and `sum` of only NULLs is NULL. Wrap it in `F.coalesce(..., F.lit(0))`.
- `1`: The `when` condition is false for the only row, so there is nothing to add.

A2-L3 exercise 3

Hints. (1) The first function returns its first non-null argument. (2) The second returns a value only when a condition holds. (3) `coalesce` and `when`.

Answer.

```
F.coalesce(F.sum(reached), F.lit(0.0)).alias("weighted_reach")
...
F.when(F.col("weighted_base") > 0, F.col("weighted_reach") /
    F.col("weighted_base"))
```

`F.coalesce(x, 0)` turns a NULL sum into 0. `F.when(base > 0, ratio)` returns NULL when the base is 0, because there is no `otherwise`.

Common wrong answers.

- g1 = `ifnull`: `ifnull` works in SQL, but the standard DataFrame function for this is `F.coalesce`, which takes any number of columns.
- g2 = `where`: `where` filters rows. You want a conditional value per row: `F.when`.
- g1 = `nvl`: `nvl` is a SQL function name. In the DataFrame API use `F.coalesce`.

A2-L3 exercise 4

Hints. (1) Build the reached-weight expression with `F.when`, then aggregate with `groupBy(*grain)`. (2) Wrap each sum in `F.coalesce(..., F.lit(0.0))`. (3) Add the percentage with `F.when(F.col("weighted_base") > 0, reach / base)`.

Answer.

```python
from pyspark.sql import DataFrame
from pyspark.sql import functions as F


def reach_metrics(df: DataFrame, grain: list[str]) -> DataFrame:
    """Weighted reach, one row per grain key.

    Args:
        df: In-scope respondents with reach_flag and final_weight.
        grain: Columns that define one output row.

    Returns:
        unweighted_n, weighted_base, weighted_reach and weighted_reach_pct per grain key.
    """
    reached = F.when(F.col("reach_flag") == 1, F.col("final_weight"))
    metrics = df.groupBy(*grain).agg(
        F.count("*").alias("unweighted_n"),
        F.coalesce(F.sum("final_weight"), F.lit(0.0)).alias("weighted_base"),
        F.coalesce(F.sum(reached), F.lit(0.0)).alias("weighted_reach"),
    )
    return metrics.withColumn(
        "weighted_reach_pct",
        F.when(F.col("weighted_base") > 0, F.col("weighted_reach") / F.col("weighted_base")),
    )
```

`groupBy(*grain)` takes the grain as data, so the same function serves every gold table. Coalesce turns empty sums into 0, and the guarded ratio returns NULL for an empty base.

A2-L3 exercise 5

Hints. (1) Three of these are ordinary custom properties or comments. (2) One property key has a special meaning in Spark. (3) Ownership is set with ALTER TABLE ... OWNER TO.

Answer. c) `ALTER TABLE research_gold.nat_metrics SET TBLPROPERTIES ('owner' = 'data.engineering@example.org')` Because: r1) `owner` is a reserved table property, so Spark rejects setting it this way.

Spark reserves `owner` and rejects it with `UNSUPPORTED_FEATURE.SET_TABLE_PROPERTY`. Your reference `unity_catalog.sql` sets it, so that script fails. Use `ALTER TABLE ... OWNER TO` for ownership, and a comment or custom property for the grain.

Common wrong answers.

- `a`: COMMENT ON TABLE is valid and a good home for the grain.
- `b`: A custom property such as `grain` is valid. Tools can read it with SHOW TBLPROPERTIES.
- `d`: A custom property such as `layer` is valid.
- `c|r2`: Right statement, wrong reason. Property values can be any string. The key `owner` is reserved, and Spark raises UNSUPPORTED_FEATURE.SET_TABLE_PROPERTY.

A3-L1 Idempotent MERGE, and why you deduplicate the source first

A3-L1 exercise 1

Hints. (1) Count how many source rows match the one target row. (2) Ask yourself which of the two source rows Delta should use for the update. (3) Delta raises DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE in this case.

Answer. b) The MERGE fails with an error. Because: r1) Two source rows match one target row, and Delta cannot tell which should win.

Delta requires at most one source row per matched target row. With two, the update is ambiguous, so the `MERGE` fails with `DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE`. Deduplicate the source first.

Common wrong answers.

- `a`: Delta has no notion of 'last' in an unordered DataFrame. Row order in Spark is not guaranteed, so Delta refuses to guess.
- `d`: Spark gives no row order without an explicit sort, so 'first' has no meaning. Delta refuses to guess.
- `c`: Both rows match the existing target row, so neither goes to the insert clause. Duplicate inserts happen only when no target row matches.

- `b|r2`: Right outcome, wrong reason. Delta does not enforce primary keys. The error comes from the ambiguous match, not from a key constraint.
- `b|r3`: Right outcome, wrong reason. Caching has nothing to do with it. The error comes from two source rows matching one target row.

A3-L1 exercise 2

Hints. (1) Two clauses and one action. (2) The clause names start with `whenMatched` and `whenNotMatched`, and both copy every column. (3) The copy-every-column versions end in `All`. The action that runs a MERGE is not `collect`.

Answer.

```
(
    DeltaTable.forName(spark, "research_silver.nat_respondents").alias("t")
  .merge(source.alias("s"), "t.respondent_id = s.respondent_id AND t.wave_id = s.wave_id")
  .whenMatchedUpdateAll()
  .whenNotMatchedInsertAll()
  .execute()
)
```

`whenMatchedUpdateAll()` copies every source column onto the matched target row. `whenNotMatchedInsertAll()` inserts rows with no match. `execute()` runs the MERGE.

Common wrong answers.

- g3 = `collect`: `collect()` is a DataFrame action. A MERGE builder runs with `execute()`.
- g1 = `whenMatchedUpdate`: `whenMatchedUpdate()` needs a `set=` mapping of columns. With no arguments you want `whenMatchedUpdateAll()`.
- g2 = `whenNotMatchedBySourceDelete`: `whenNotMatchedBySourceDelete()` deletes target rows that are missing from the source. With a one-wave source it would delete every other wave.

A3-L1 exercise 3

Hints. (1) Work through the first run: which clause fires for each row? (2) On the second run, does any source row lack a match? (3) Matched rows update in place. Updates do not add rows.

Answer.

```
3
```

Run one inserts 3 rows. Run two matches all 3 and updates them with identical values. The count stays 3. That is idempotence.

Common wrong answers.

- `6`: 6 is what an append would give. On the second run every source row matches an existing target row, so the MERGE updates instead of inserting.
- `0`: The first run inserts all 3 rows, because none of them matches an empty table.

A3-L1 exercise 4

Hints. (1) The grain is respondent per wave. Which columns identify one row? (2) Compare the deduplication keys with the MERGE condition. (3) A `W2` row for respondent R1 matches the `W1` row for R1, and overwrites it.

Answer. The bug is on line 6.

Fixed version:

```
1  def upsert_respondents(spark: SparkSession, source: DataFrame) -> None:
2      """MERGE one wave of respondents into silver. Grain: respondent per
     wave."""
3      latest = latest_per_key(source, ["respondent_id", "wave_id"],
     "delivery_version")
4      (
5          DeltaTable.forName(spark,
     "research_silver.nat_respondents").alias("t")
6          .merge(latest.alias("s"), "t.respondent_id = s.respondent_id AND
     t.wave_id = s.wave_id")
7          .whenMatchedUpdateAll()
8          .whenNotMatchedInsertAll()
9          .execute()
10     )
```

The condition matches on `respondent_id` alone. A wave `W2` row for a respondent matches that respondent's `W1` row and overwrites it, including `wave_id`. Match on the full grain: `t.respondent_id = s.respondent_id AND t.wave_id = s.wave_id`.

Common wrong answers.

- Line 3: Line 3 deduplicates on both key columns, which matches the grain. Look at what the MERGE matches on.
- Line 7: `whenMatchedUpdateAll()` is right for a redelivery. The problem is which rows count as a match.

A3-L1 exercise 5

Hints. (1) You need a window, a rank and a filter. (2) `Window.partitionBy(*keys).orderBy(F.col(order_col).desc())` sorts each key's rows newest first. (3) Add `F.row_number().over(window)` as a column, keep rank 1, then drop the helper column.

Answer.

```python
from pyspark.sql import DataFrame, Window
from pyspark.sql import functions as F


def latest_per_key(df: DataFrame, keys: list[str], order_col: str) -> DataFrame:
    """Keep the row with the highest order_col for each key.

    Args:
        df: Source rows, possibly with repeated keys.
        keys: Columns that identify one target row.
        order_col: Column that ranks versions. Highest wins.

    Returns:
        One row per key.
    """
    window = Window.partitionBy(*keys).orderBy(F.col(order_col).desc())
    return (
        df.withColumn("_rank", F.row_number().over(window))
        .filter(F.col("_rank") == 1)
        .drop("_rank")
    )
```

A window partitioned by the keys and ordered by `order_col` descending puts the newest row first in each key. `row_number()` numbers them 1, 2, 3. Keep 1, drop the helper column.

A3-L1 exercise 6

Hints. (1) Nothing unchecked should reach silver. (2) The MERGE needs a source with one row per key. (3) The run log records the outcome of everything before it.

Answer.

1. Validate the bronze delivery against its data contract
2. Rename and cast columns from the Variable Registry
3. Keep the newest row per respondent and wave
4. MERGE into silver on respondent_id and wave_id
5. Run the table health check on silver
6. Write one record to the pipeline run log

Check the delivery, map it to canonical names, deduplicate, merge, confirm the table is healthy, then log the run with its final status.

Common wrong answers.

- Deduplicating after the MERGE is too late. The MERGE has already failed, or has already inserted the repeated rows.
- The contract describes the delivery as received. Check it before you rename, or the column names in the contract no longer match.
- Log last, so the run record includes the health check result and the final status.

A3-L2 Schema enforcement is on by default. Evolve on purpose.

A3-L2 exercise 1

Hints. (1) Is schema evolution on or off by default? (2) The table has no region column. (3) Enforcement rejects the write.

Answer. c) The append fails with a schema mismatch. Because: r1) Schema enforcement rejects writes with columns the table does not have.

Enforcement is on by default, so a write with an extra column fails. You opt in to evolution per write, or change the table with `ALTER TABLE`.

Common wrong answers.

- `a`: That needs `mergeSchema`. Without it, enforcement rejects the new column.
- `b`: An append does not drop columns. A MERGE with updateAll does, which makes it the dangerous case.
- `d`: A rescue column is an Auto Loader feature (`_rescued_data`), not a Delta write behaviour.
- `c|r2`: Right outcome, wrong reason. Delta schemas can change, with mergeSchema, withSchemaEvolution or ALTER TABLE. The default is to enforce.

A3-L2 exercise 2

Hints. (1) Does a MERGE fail on an extra source column the way an append does? (2) updateAll and insertAll work on the target's columns. (3) The extra column is ignored without an error.

Answer.

```
['id', 'w']
```

Without schema evolution, `updateAll` and `insertAll` use the target's columns and quietly ignore `region`. Your own column check is the only guard here.

Common wrong answers.

- `['id', 'w', 'region']`: Only with `.withSchemaEvolution()` on the builder. Without it, updateAll and insertAll copy the target's columns and ignore region.
- `AnalysisException`: No error. The MERGE succeeds and the extra source column is silently ignored. That is why this case is dangerous.

A3-L2 exercise 3

Hints. (1) The builder method names what it enables. (2) It exists on DeltaMergeBuilder in delta-spark 4.0. (3) `withSchemaEvolution`.

Answer.

```
(
    DeltaTable.forName(spark, table).alias("t")
    .merge(source.alias("s"), condition)
    .withSchemaEvolution()
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute()
)
```

`withSchemaEvolution()` turns evolution on for this MERGE only, which is safer than a session-wide setting.

Common wrong answers.

- g1 = `mergeSchema`: `mergeSchema` is a write option for appends and overwrites. The MERGE builder uses `withSchemaEvolution()`.
- g1 = `option`: The MERGE builder has no `option` method for this. Call `withSchemaEvolution()`.

A3-L2 exercise 4

Hints. (1) Two set differences, one in each direction. (2) Wrap each in `sorted()`. (3) `assert_no_drift` calls `schema_drift` and raises if either list is non-empty.

Answer.

```
def schema_drift(actual: list[str], expected: list[str]) -> dict[str,
    list[str]]:
    """Compare delivered columns with the registry.
```

```
    Args:
        actual: Columns in the delivery.
        expected: Canonical columns from the registry.

    Returns:
        Sorted unexpected and missing column names.
    """
    return {
        "unexpected": sorted(set(actual) - set(expected)),
        "missing": sorted(set(expected) - set(actual)),
    }


def assert_no_drift(actual: list[str], expected: list[str]) -> None:
    """Raise ValueError when the delivery's columns differ from the registry."""
    drift = schema_drift(actual, expected)
    if drift["unexpected"] or drift["missing"]:
        raise ValueError(f"schema drift: {drift}")
```

Checking both directions matters because Delta treats them differently: an extra column fails an append, but a missing column lands as NULL without an error.

A3-L2 exercise 5

Hints. (1) Which line changes a setting? (2) What does that setting do to every MERGE in the session? (3) Remove the conf.set. Evolve deliberately with withSchemaEvolution() when a change is approved.

Answer. The bug is on line 3.

Fixed version:

```
def upsert_silver(spark: SparkSession, source: DataFrame, table: str, keys: list[str]) -> None:
    """MERGE a deduplicated source into silver."""
    condition = " AND ".join(f"t.{k} = s.{k}" for k in keys)
    (
        DeltaTable.forName(spark, table).alias("t")
        .merge(source.alias("s"), condition)
        .whenMatchedUpdateAll()
        .whenNotMatchedInsertAll()
        .execute()
    )
```

Line 3 turns on automatic schema evolution for the whole Spark session, so every MERGE adds any new source column. Remove it. When you approve a new column, use `ALTER TABLE ... ADD COLUMN` or `.withSchemaEvolution()` on that one MERGE.

Common wrong answers.

- Line 8: updateAll is fine. It only adds columns when evolution is switched on, and something switched it on.
- Line 4: The condition is correct. Look for a setting that changes how every MERGE treats new columns.

A3-L3 Time travel, RESTORE, OPTIMIZE and VACUUM

A3-L3 exercise 1

Hints. (1) The old version's files still exist. (2) Which command makes an old version current? (3) RESTORE TABLE ... TO VERSION AS OF n.

Answer. b) `RESTORE TABLE` to the version before the bad write. Because: r1) The earlier version's files are still on storage, and RESTORE makes that version current again.

RESTORE points the table back at the old version's files, as a new version. History is preserved, so you can inspect the bad write later.

Common wrong answers.

- `a`: It works, but it takes hours. RESTORE takes seconds while the old files still exist.
- `c`: VACUUM deletes old files, which are the ones you need to recover. It makes things worse.
- `d`: OPTIMIZE compacts files in the current version. It does not change the data.
- `b|r2`: Right action, wrong reason. RESTORE writes a new version. The bad write stays in history.

A3-L3 exercise 2

Hints. (1) The time travel clause has three words. (2) The same clause appears in the second statement. (3) `VERSION AS OF` and `RESTORE`.

Answer.

```
1 SELECT * FROM research_gold.nat_metrics VERSION AS OF 41;
2 RESTORE TABLE research_gold.nat_metrics TO VERSION AS OF 41;
```

`VERSION AS OF n` reads a past version. `RESTORE TABLE ... TO VERSION AS OF n` makes it current.

Common wrong answers.

- g1 = `AT VERSION`: The Delta syntax is `VERSION AS OF`.

- g2 = `ROLLBACK`: ROLLBACK undoes a transaction. Delta's command to make an old version current is RESTORE.

A3-L3 exercise 3

Hints. (1) Does RESTORE delete history? (2) Every commit adds one version. (3) The next version after 3.

Answer.

```
1 4
```

RESTORE is a commit like any other, so it becomes version 4 with the contents of version 1. Versions 2 and 3 remain in history.

Common wrong answers.

- `1`: RESTORE does not rewind the version counter. It writes a new version whose content matches version 1.
- `3`: Version 3 is the DELETE. Every write, RESTORE included, adds a new version.

A3-L3 exercise 4

Hints. (1) Which command deletes files? (2) What retention does it use? (3) Line 3. Run VACUUM with the default 7-day retention and drop line 2.

Answer. The bug is on line 3.

Fixed version:

```
1 OPTIMIZE research_silver.nat_respondents ZORDER BY (wave_id);
2 VACUUM research_silver.nat_respondents;
3 ANALYZE TABLE research_silver.nat_respondents COMPUTE STATISTICS;
```

`RETAIN 0 HOURS` deletes every file no longer in the current version, so no earlier version is readable. Line 2 disables the check that would have blocked it. Run VACUUM with default retention, or set the retention properties to the window you need.

Common wrong answers.

- Line 1: OPTIMIZE compacts files and keeps history. It does not block time travel.
- Line 2: Line 2 disables the guard, which is bad, but on its own it deletes nothing. The line that removes the files is the problem.

A3-L3 exercise 5

Hints. (1) Which property decides how long VACUUM keeps old files? (2) The log must cover the same window. (3) VACUUM still runs, with default arguments.

Answer.

- Set both retention properties to 30 days. From Databricks Runtime 18.0, the log retention must be at least the deleted-file retention.
- Keep running a plain `VACUUM t`. It respects the 30-day property and stops storage growing without limit.

Both retention properties at 30 days keep every version from the last month readable. A plain VACUUM then cleans up only what is older.

Common wrong answers.

- deleted = `d7`: With 7 days, VACUUM removes files after a week, so versions older than 7 days are unreadable. The policy needs 30.
- vacuum = `none`: Without VACUUM, storage grows with every write. Keep it, and let the retention property protect 30 days.
- vacuum = `zero`: RETAIN 0 HOURS overrides the property and deletes history immediately.
- log = `d7`: The log must keep entries at least as long as the data files, or the versions cannot be reconstructed.

A4-L1 The three-level namespace, one catalog per environment

A4-L1 exercise 1

Hints. (1) Which level can you lock a whole environment behind? (2) Grants on the top level control everything below it. (3) The catalog.

Answer. a) Catalog Because: r1) Permissions and storage attach at catalog level, so dev principals can be kept out of prod entirely.

One catalog per environment gives a hard boundary. A dev job without privileges on `research_prod` cannot read or write it by accident.

Common wrong answers.

- `b`: Schemas are better used for layers. Putting environments there mixes dev and prod under one catalog's permissions.
- `c`: A prefix is a naming habit, not a boundary. Anyone with rights on the schema can write both.

- `d`: A column puts dev and prod rows in the same table. One bad filter mixes them.
- `a|r2`: Right level, wrong reason. The point is isolation: grants on research_prod do not reach research_dev, and the reverse.

A4-L1 exercise 2

Hints. (1) catalog.schema.table. (2) The environment is the catalog. (3) research_prod, then research_gold.

Answer.

```
1 GOLD = "research_prod.research_gold.nat_metrics"
```

`research_prod.research_gold.nat_metrics`: environment, layer, table.

Common wrong answers.

- g1 = `research_gold`; g2 = `research_prod`: Swapped. The catalog comes first and holds the environment. The schema holds the layer.
- g1 = `hive_metastore`: hive_metastore is the legacy store. Production tables live in a Unity Catalog catalog such as research_prod.

A4-L1 exercise 3

Hints. (1) Count the parts in the table name. (2) What decides the catalog for a two-part name? (3) Add `cfg.catalog` as the first part.

Answer. The bug is on line 3.

Fixed version:

```
1 def write_metrics(df: DataFrame, cfg: RunConfig) -> None:
2     """Write gold metrics for the configured environment."""
3     target = f"{cfg.catalog}.research_gold.{cfg.tracker}_metrics"
4     df.write.format("delta").mode("append").saveAsTable(target)
```

A two-part name resolves against the session's current catalog, which here is production. Build a three-part name from `cfg.catalog`.

Common wrong answers.

- Line 4: Append is right for this job. The question is which table it writes to.

A4-L1 exercise 4

Hints. (1) Keep the suffixes in a dict constant. (2) Check for a dot in catalog and for an unknown layer before building the name. (3) `f"{catalog}.research_{layer}.{tracker}_{LAYER_SUFFIX[layer]}"`.

Answer.

```python
LAYER_SUFFIX = {"bronze": "raw", "silver": "respondents", "gold": "metrics"}


def table_name(catalog: str, layer: str, tracker: str) -> str:
    """Three-level Unity Catalog name for a tracker's table in one layer.

    Args:
        catalog: Environment catalog, for example research_dev.
        layer: bronze, silver or gold.
        tracker: Tracker short name, for example nat.

    Returns:
        The fully qualified table name.

    Raises:
        ValueError: If the layer is unknown or the catalog contains a dot.
    """
    if "." in catalog:
        raise ValueError(f"catalog must be a single name, got {catalog!r}")
    if layer not in LAYER_SUFFIX:
        raise ValueError(f"unknown layer {layer!r}")
    return f"{catalog}.research_{layer}.{tracker}_{LAYER_SUFFIX[layer]}"
```

One function owns the naming convention. It rejects inputs that would produce a wrong name instead of silently creating a new table.

A4-L1 exercise 5

Hints. (1) The strongest boundary goes at the top. (2) Layers group tables with the same contract. (3) Tables hold one dataset at one grain.

Answer.

- Catalog per environment: `research_dev`, `research_prod`.
- Schema per layer, plus `research_governance` for the run log, registry and contracts.
- Table per tracker and grain: `nat_respondents`.

Environment, layer, table: each level answers one question, and the full name tells a reader all three.

Common wrong answers.

- layer = `catalog`: A catalog per layer multiplies catalogs by environments, and grants must then be repeated everywhere. Use schemas for layers.
- tracker = `schema`: A schema per tracker scatters the layers. Keep layers as schemas, and trackers as tables within them.

- env = `schema`: Schemas under one catalog share its permissions and storage. Environments need the stronger boundary of separate catalogs.

A4-L2 Grant to groups, grant the least, grant at the right level

A4-L2 exercise 1

Hints. (1) The namespace goes catalog, schema, table. (2) Each USE privilege opens one level. (3) The data privilege comes last.

Answer.

1. GRANT USE CATALOG ON CATALOG research_prod TO `research_analysts`
2. GRANT USE SCHEMA ON SCHEMA research_prod.research_gold TO `research_analysts`
3. GRANT SELECT ON SCHEMA research_prod.research_gold TO `research_analysts`

Reading a table needs USE CATALOG, USE SCHEMA and SELECT. Granting SELECT on the schema makes every current and future table readable.

Common wrong answers.

- The grants all work in any order, but read them outermost first: catalog, then schema, then data. This order is inside out.
- Catalog comes before schema. USE CATALOG opens the catalog, USE SCHEMA opens the schema inside it.

A4-L2 exercise 2

Hints. (1) How many privileges does a read need? (2) Each parent level must be usable. (3) USE CATALOG and USE SCHEMA are missing.

Answer. b) The query fails, because they lack USE CATALOG and USE SCHEMA. Because: r1) Reading needs USE on the parent catalog and schema as well as SELECT on the table.

A read needs USE CATALOG, USE SCHEMA and SELECT. With SELECT alone the query fails with a permission error.

Common wrong answers.

- `a`: SELECT on the table is necessary but not enough. The catalog and schema must be usable too.
- `c`: Missing privileges raise a permission error. They never return an empty result.

- `d`: Masks are unrelated. The query fails before any column is read.
- `b|r2`: Right outcome, wrong reason. SELECT can be granted on a table. The missing pieces are USE CATALOG and USE SCHEMA.

A4-L2 exercise 3

Hints. (1) Which principal is not a group? (2) An email address names one person. (3) Create a group such as research_silver_readers and grant to it.

Answer. The bug is on line 3.

Fixed version:

```
1 GRANT USE CATALOG ON CATALOG research_prod TO `research_analysts`;
2 GRANT USE SCHEMA, SELECT ON SCHEMA research_prod.research_gold TO
    `research_analysts`;
3 GRANT USE CATALOG ON CATALOG research_prod TO `research_silver_readers`;
4 GRANT USE SCHEMA, SELECT ON SCHEMA research_prod.research_silver TO
    `research_silver_readers`;
```

Line 3 grants to one person. Grant to a group instead, and add Sam to the group. When Sam moves teams, one membership change removes the access.

Common wrong answers.

- Line 2: That grant goes to a group at schema level, which is the pattern to follow.
- Line 1: USE CATALOG to a group is correct and needed.

A4-L2 exercise 4

Hints. (1) Two statements are enough. (2) One for the catalog, one for the schema. (3) `GRANT USE SCHEMA, SELECT, MODIFY ON SCHEMA research_prod.research_silver TO ...`.

Answer.

```
1 GRANT USE CATALOG ON CATALOG research_prod TO `research-pipeline-sp`;
2 GRANT USE SCHEMA, SELECT, MODIFY ON SCHEMA research_prod.research_silver
    TO `research-pipeline-sp`;
```

USE CATALOG opens the catalog. USE SCHEMA, SELECT and MODIFY on the schema cover every table in it, now and in future. Nothing else is needed.

A4-L2 exercise 5

Hints. (1) Who writes, and who only reads? (2) Does anyone in the list need raw deliveries? (3) Groups, not users, for every grant.

Answer.

- Analysts read gold: USE CATALOG, USE SCHEMA and SELECT on the gold schema.
- The pipeline writes silver: the same plus MODIFY on the silver schema.
- Analysts have no reason to see raw deliveries, so no bronze access.
- Administration goes through an admin group, never through grants on user accounts.

Least privilege: people read the layer they report from, pipelines write the layers they own, and admin rights live in a group.

Common wrong answers.

- analysts = `modify`: Reports only read. MODIFY would let an analyst change gold outside the pipeline.
- analysts_bronze = `select`: Bronze holds raw deliveries, which can include fields gold never exposes. Give access only when there is a need.
- admin = `user_all`: Grants to user accounts outlive the people's roles. Use a group.

A4-L3 Row filters and column masks in Unity Catalog

A4-L3 exercise 1

Hints. (1) The filter is a SQL function. (2) You attach it with ALTER TABLE. (3) SET ROW FILTER ... ON (column).

Answer. b) `ALTER TABLE t SET ROW FILTER f ON (col)` Because: r1) Databricks row filters are SQL UDFs bound to a table with SET ROW FILTER.

Create a SQL UDF that returns BOOLEAN, then bind it with `ALTER TABLE ... SET ROW FILTER f ON (col)`.

Common wrong answers.

- `a`: That is Snowflake syntax. Databricks raises a parse error on it.
- `c`: GRANT has no WHERE clause. Grants control the whole object.
- `d`: A view filters only readers who use the view. Anyone with SELECT on the table bypasses it. A row filter applies to the table itself.
- `b|r2`: Right statement, wrong reason. You can add a row filter to an existing table with ALTER TABLE.

A4-L3 exercise 2

Hints. (1) The function checks account-level group membership. (2) The column clause has two words. (3) `is_account_group_member` and `SET MASK`.

Answer.

```
1 CREATE FUNCTION research_prod.research_governance.mask_postcode(postcode
    STRING)
2 RETURN CASE WHEN is_account_group_member('pii_readers') THEN postcode
    ELSE '***' END;
3
4 ALTER TABLE research_prod.research_silver.nat_respondents
5 ALTER COLUMN postcode SET MASK
    research_prod.research_governance.mask_postcode;
```

`is_account_group_member('pii_readers')` checks the account group. `ALTER COLUMN ... SET MASK f` binds the mask to the column.

Common wrong answers.

- g1 = `is_member`: `is_member` checks workspace-local groups. Unity Catalog uses account-level groups: `is_account_group_member`.
- g1 = `current_user`: `current_user()` returns a name, not a group check. Masks should test group membership.
- g2 = `SET ROW FILTER`: Row filters apply to the table. A column uses SET MASK.

A4-L3 exercise 3

Hints. (1) Which line names one identity? (2) Access reviews look at groups and grants, not at function bodies. (3) Replace the email check with a group check.

Answer. The bug is on line 3.

Fixed version:

```
1 CREATE OR REPLACE FUNCTION
    research_prod.research_governance.respondent_filter(tracker_id STRING)
2 RETURN is_account_group_member('research_approved_analysts')
3     OR is_account_group_member('research_pipelines');
```

Line 3 grants access to one named identity inside the function body, where no access review looks. Put the service principal in a group and test the group.

Common wrong answers.

- Line 2: Testing an account group is the right pattern. Look for a named person or identity.
- Line 1: A governance schema is the right home for the function.

A4-L3 exercise 4

Hints. (1) The function returns TRUE for rows the reader may see. (2) Group members see everything, so the group check stands alone on one side of OR. (3) Then bind with ALTER TABLE ... SET ROW FILTER ... ON (region).

Answer.

```
CREATE OR REPLACE FUNCTION
   research_prod.research_governance.region_filter(region STRING)
RETURN is_account_group_member('research_all_regions') OR region =
   'NATIONAL';

ALTER TABLE research_prod.research_gold.nat_metrics
SET ROW FILTER research_prod.research_governance.region_filter ON
   (region);
```

`group OR region = 'NATIONAL'` is TRUE for every row when the reader is in the group, and only for national rows otherwise. `AND` would hide everything from people outside the group. The function does nothing until `SET ROW FILTER` binds it.

A4-L3 exercise 5

Hints. (1) Whole table, some rows, one column. (2) Grants are all or nothing on an object. (3) Filters remove rows. Masks change values.

Answer.

- No access at all: do not grant SELECT. Grants decide who reads the table.
- Some rows only: a row filter decides which rows each reader sees.
- All rows, hidden values: a column mask decides what each reader sees in one column.

Grants control access to the object, filters control rows, masks control column values. Combine them: grant SELECT to analysts, filter trackers, mask postcodes.

Common wrong answers.

- team = `filter`: A filter that returns nothing still lets the team query the table and see its structure. Withhold the grant.
- tracker = `mask`: A mask changes values in a column. The requirement removes whole rows.
- postcode = `filter`: Filtering would remove the respondents entirely. The requirement keeps the rows and hides one column.

A5-L1 Lazy evaluation, and why every count() re-runs the plan

A5-L1 exercise 1

Hints. (1) Which call returns a plain Python value instead of a DataFrame? (2) Transformations return DataFrames. (3) count() returns an int, so it must run the plan.

Answer. c) `n = df3.count()` Because: r1) count() is an action. The other lines are transformations that only extend the plan.

`count()` returns a number, so Spark must execute the plan. The other lines return DataFrames and only describe work.

Common wrong answers.

- `a`: filter is a transformation. It returns a new DataFrame without reading any data.
- `b`: withColumn is a transformation. Nothing runs yet.
- `d`: groupBy().agg() is still a transformation. It runs only when an action such as show or write asks for the result.
- `c|r2`: Right line, wrong reason. Every line runs as Python. Only actions make Spark execute the plan.

A5-L1 exercise 2

Hints. (1) How many actions are there? (2) Does Spark keep results between actions without cache? (3) Two full runs over 1,000 rows.

Answer.

```
2000
```

Nothing is kept between actions, so each `count()` runs the UDF on all 1,000 rows. This was measured locally on PySpark 4.0.4.

Common wrong answers.

- `1000`: Each action re-runs the whole plan, UDF included. Two counts mean two full evaluations.
- `500`: The filter comes after the UDF, so the UDF runs on all 1,000 rows each time.
- `0`: Defining df runs nothing, but each count() is an action that executes the plan.

A5-L1 exercise 3

Hints. (1) The DataFrame method that takes several aggregate expressions. (2) The function that counts rows where a condition is true. (3) `agg` and `count_if`.

Answer.

```
stats = df.agg(
    F.count("*").alias("rows"),
    F.count_if(F.col("final_weight").isNull()).alias("null_weights"),
).collect()[0]
```

`df.agg(...)` evaluates every expression in one pass. `F.count_if(cond)` counts rows where `cond` is true.

Common wrong answers.

- g1 = `select`: `select` with aggregate functions also works, but `agg` is the idiomatic call for a single aggregated row.
- g2 = `count`: `F.count` counts non-null values of an expression, and `isNull()` is never null. Every row would count. Use `count_if`.
- g2 = `sum`: `F.sum` of a boolean fails. You would need `.cast("int")` first. `count_if` says what you mean.

A5-L1 exercise 4

Hints. (1) Every value can be an aggregate expression. (2) `F.count("*")`, `F.count_if(...)`, `F.min(...)`, `F.max(...)`. (3) Put all five in one `df.agg(...)`, collect once, and return `row.asDict()`.

Answer.

```
def table_health(df: DataFrame) -> dict[str, float | int | None]:
    """Row count, null counts and weight range in one pass over df."""
    row = df.agg(
        F.count("*").alias("rows"),
        F.count_if(F.col("respondent_id").isNull()).alias("null_ids"),
        F.count_if(F.col("final_weight").isNull()).alias("null_weights"),
        F.min("final_weight").alias("min_weight"),
        F.max("final_weight").alias("max_weight"),
    ).collect()[0]
    return row.asDict()
```

Five aggregate expressions in one `agg` produce one row in one scan. The original ran five separate jobs over the same data.

A5-L1 exercise 5

Hints. (1) Simple aggregates can share one pass. (2) Different shapes need the data stored somewhere reusable. (3) Serverless has no cache APIs.

Answer.

- Combine every count and sum into one `agg`. One scan answers them all.
- For checks that need different shapes, write the expensive result to a Delta table once and read it back.
- Serverless compute does not support cache APIs, so `cache()` is not an option there.

Combine what you can into one action. Store an expensive result in Delta when several differently shaped queries need it. Keep cache() for classic clusters.

Common wrong answers.

- several_counts = `repeat`: Each action re-runs the join. Four checks would run the expensive join four times.
- many_shapes = `cache`: On serverless compute, cache APIs are not supported. Materialise to a Delta table instead.
- cache_serverless = `use`: Databricks documents that DataFrame and SQL cache APIs are not supported on serverless compute.

A5-L2 Built-in functions over Python UDFs

A5-L2 exercise 1

Hints. (1) Which option never leaves the JVM? (2) Are trim and upper available as built-ins? (3) `F.trim` and `F.upper`.

Answer. c) `F.upper(F.trim(F.col("s")))` Because: r1) Built-ins run inside the JVM with no data transfer to Python.

Built-ins run in Spark's generated JVM code on the executors. UDFs of either kind pay to move data to Python and back.

Common wrong answers.

- `a`: A Python UDF moves every value to a Python process and back. It measured about six times slower than a built-in.
- `b`: A pandas UDF beats a Python UDF because it uses Arrow batches, but it still leaves the JVM. A built-in wins.
- `d`: collect() pulls 50 million values onto one machine. It is the slowest option and can crash the driver.
- `c|r2`: Right choice, wrong reason. Built-ins run on the executors, like everything else. They win by staying in the JVM.

A5-L2 exercise 2

Hints. (1) What does `\s+` match? (2) What do built-ins return for NULL input? (3) Whitespace removed, then upper-cased. NULL stays NULL.

Answer.

```
['AB1', None]
```

The regex removes all whitespace, `upper` capitalises, and the NULL row stays NULL. No special handling needed.

Common wrong answers.

- `[' AB 1 ', None]`: `regexp_replace` with `\s+` removes every run of whitespace, including the inner space.
- `PythonException`: Built-ins handle NULL. Only a Python UDF would raise on the None row.
- `['AB1', '']`: NULL input gives NULL output, not an empty string.

A5-L2 exercise 3

Hints. (1) Padding on the left has a function named after that side. (2) Pulling a capture group out of a string uses a regex function. (3) `lpad` and `regexp_extract`.

Answer.

```
df = df.withColumns({
    "respondent_id": F.lpad(F.col("respondent_id"), 8, "0"),
    "region": F.regexp_extract(F.col("area_code"), r"^([A-Z]{2})-", 1),
})
```

`F.lpad(col, 8, "0")` adds leading zeros. `F.regexp_extract(col, pattern, 1)` returns capture group 1, or an empty string when there is no match.

Common wrong answers.

- g1 = `rpad`: `rpad` pads on the right. Leading zeros go on the left: `lpad`.
- g2 = `regexp_replace`: `regexp_replace` substitutes text. To pull out a capture group, use `regexp_extract` with the group index.
- g2 = `substring`: `substring` takes positions, not a pattern, and cannot check the hyphen. `regexp_extract` with group 1 matches the rule exactly.

A5-L2 exercise 4

Hints. (1) The function takes a Column and returns a Column expression. (2) `F.when(age < 35, "16-34").when(age < 55, "35-54")`. (3) Finish with `.when(age.isNotNull(), "55+")`, not `.otherwise`.

Answer.

```python
from pyspark.sql import Column
from pyspark.sql import functions as F


def age_band(age: Column) -> Column:
    """16-34, 35-54 or 55+. NULL age gives NULL."""
    return (
        F.when(age < 35, "16-34")
        .when(age < 55, "35-54")
        .when(age.isNotNull(), "55+")
    )
```

`F.when` chains evaluate in order. Without `otherwise`, an unmatched row is NULL, and `age < 35` is NULL for a NULL age, so the final `isNotNull()` guard keeps NULL ages NULL.

A5-L2 exercise 5

Hints. (1) What does the UDF receive for a NULL postcode? (2) `None.replace` raises AttributeError. (3) Replace the UDF with `F.upper(F.regexp_replace(...))`.

Answer. The bug is on line 6.

Fixed version:

```python
from pyspark.sql import DataFrame
from pyspark.sql import functions as F


def clean_postcodes(df: DataFrame) -> DataFrame:
    """Remove spaces from postcodes and upper-case them. NULL stays
    NULL."""
      return df.withColumn("postcode",
    F.upper(F.regexp_replace(F.col("postcode"), r"\s+", "")))
```

For a NULL postcode the UDF receives `None`, and `None.replace` raises inside the Python worker, failing the job. Built-ins pass NULL through, and run faster.

Common wrong answers.

- Line 4: The decorator makes it a UDF, which is slow, but the crash happens where a value is used.
- Line 11: The call is fine. The failure is inside the function, on a NULL value.

A5-L3 Joins, broadcast, partitions and reading the plan

A5-L3 exercise 1

Hints. (1) Which side is small? (2) What is the most expensive step in a join of two large tables? (3) Copy the small side to every executor.

Answer. b) BroadcastHashJoin, with the lookup sent to every executor Because: r1) Broadcasting the small side avoids shuffling the large side.

With a tiny lookup, broadcast it. The large table stays where it is, and each executor joins locally.

Common wrong answers.

- `a`: A sort-merge join shuffles all 40 million rows across the network to meet 200 lookup rows. Broadcasting the lookup avoids that.
- `c`: That moves the join into a slow Python UDF. A broadcast join does the same job in the JVM.
- `d`: A cross join multiplies 40 million by 200 before filtering: 8 billion rows.
- `b|r2`: Right strategy, wrong reason. Broadcasting costs memory on every executor. It wins by avoiding the shuffle, and only when the broadcast side is small.

A5-L3 exercise 2

Hints. (1) The hint names the strategy. (2) An equality join with a broadcast side uses a hash table. (3) Broadcast + Hash + Join.

Answer.

```
1 BroadcastHashJoin
```

`F.broadcast` makes Spark build a hash table from `regions` on every executor. The plan shows `BroadcastHashJoin` with `LeftOuter`.

Common wrong answers.

- `SortMergeJoin`: The broadcast hint tells Spark to broadcast `regions`, so the plan uses BroadcastHashJoin.
- `BroadcastNestedLoopJoin`: That node appears for joins without an equality condition. This join is on region_code, so it is a hash join.

A5-L3 exercise 3

Hints. (1) The function names the join strategy. (2) Keep all rows from the left side. (3) The explain mode that splits the plan into numbered, detailed steps.

Answer.

```
joined = fact.join(F.broadcast(regions), on="region_code", how="left")
joined.explain(mode="formatted")
```

`F.broadcast(regions)` forces a broadcast hash join, `how="left"` keeps every fact row, and `mode="formatted"` prints a detailed plan with pushed filters.

Common wrong answers.

- g2 = `inner`: An inner join drops fact rows whose region code is missing from the lookup, without any warning. Use a left join.
- g3 = `verbose`: explain modes are simple, extended, codegen, cost and formatted. `formatted` gives the per-step details.
- g1 = `cache`: Caching does not change the join strategy. `F.broadcast` does.

A5-L3 exercise 4

Hints. (1) How many partitions does the write use? (2) One partition means one task. (3) Remove `.repartition(1)`.

Answer. The bug is on line 4.

Fixed version:

```
def write_gold(metrics: DataFrame, table: str) -> None:
    """Write gold metrics for the wave."""
    metrics.write.format("delta").mode("append").saveAsTable(table)
```

`repartition(1)` funnels every row into one task, so 15 of 16 cores sit idle. Remove it. For file sizes, rely on OPTIMIZE, liquid clustering or predictive optimization.

Common wrong answers.

- Line 6: Append is the right mode here. The slowness comes from how many tasks do the writing.
- Line 5: Delta is the right format. Look at the partition count.

A5-L3 exercise 5

Hints. (1) `fact.join(F.broadcast(regions), on=..., how=...)`. (2) A left join keeps unknown codes, with region NULL. (3) `enriched.agg(F.count_if(F.col("region").isNull())).collect()[0][0]`.

Answer.

```
from pyspark.sql import DataFrame
from pyspark.sql import functions as F


def enrich_with_regions(fact: DataFrame, regions: DataFrame) -> DataFrame:
```

```
    """Left-join fact rows to the small regions lookup, broadcasting the lookup."""
    return fact.join(F.broadcast(regions), on="region_code", how="left")


def unmatched_codes(enriched: DataFrame) -> int:
    """Rows whose region_code had no match in the lookup."""
    return int(enriched.agg(F.count_if(F.col("region").isNull())).collect()[0][0])
```

The broadcast left join keeps every fact row and never shuffles the fact table. Counting NULL regions afterwards turns silently unknown codes into a number your QA can check.

A6-L1 Data contracts with severity levels and structured results

A6-L1 exercise 1

Hints. (1) Would the problem corrupt totals or the grain? (2) Or is it something a person should look at, with no immediate harm? (3) Keys and weights are CRITICAL. Labels and small shifts are WARN.

Answer.

- Duplicate keys break the silver grain and the MERGE: CRITICAL.
- A weight total out of range makes every weighted figure wrong: CRITICAL.
- A few unexpected labels are worth investigating but do not corrupt totals: WARN.
- A small distribution shift can be real change: WARN, reviewed by a person.

CRITICAL is for problems that make published numbers wrong or break the table. WARN is for anomalies that need a human look.

Common wrong answers.

- keys = `warn`: Duplicate keys fail the MERGE or break the grain. Silver must not receive them.
- weights = `warn`: Every weighted estimate depends on the weight total. Publishing with wrong weights is worse than publishing late.
- category = `critical`: Twelve odd labels should not stop the whole wave. Log them and follow up with the supplier.

A6-L1 exercise 2

Hints. (1) How many round trips with the supplier do you want? (2) Which failures must stop the run? (3) Store all results, then raise once.

Answer. b) All three results stored, then one exception naming both CRITICAL failures. Because: r1) Evaluate everything, record everything, then act on severity once.

Run every check, store every result, then raise one exception listing all CRITICAL failures. The WARN is stored and reviewed.

Common wrong answers.

- `a`: Stopping at the first failure hides the second, and the supplier needs another round trip.
- `c`: Printed text is lost. Results must be records in a table.
- `d`: Two CRITICAL failures must stop the run before silver.
- `b|r2`: Right outcome, wrong reason. Here you deliberately raise late, after all checks have run, so one run reports everything.

A6-L1 exercise 3

Hints. (1) The typing construct for exact values. (2) The dataclass option that forbids changing fields after creation. (3) `Literal` and `frozen`.

Answer.

```
Status = Literal["PASS", "WARN", "FAIL", "SKIP"]


@dataclass(frozen=True)
class CheckResult:
    check_name: str
    status: Status
```

`Literal[...]` lists the allowed strings. `frozen=True` means no code can change a result after the check records it.

Common wrong answers.

- g1 = `Union`: `Union` combines types, not values. A fixed set of strings is a `Literal`.
- g2 = `slots`: `slots=True` saves memory but still allows changes. `frozen=True` makes a result immutable once recorded.

A6-L1 exercise 4

Hints. (1) Collect all CRITICAL failures into a list first. (2) Raise once, joining their names into the message. (3) Return the WARN results when nothing CRITICAL failed.

Answer.

```
class ContractViolation(Exception):
```

```
    """Raised when one or more CRITICAL checks fail."""


def enforce(results: list[dict[str, str]]) -> list[dict[str, str]]:
    """Act on contract results.

    Args:
        results: Every check result for one delivery.

    Returns:
        The WARN results, for logging.

    Raises:
        ContractViolation: If any CRITICAL check failed. The message names each one.
    """
    critical = [r for r in results if r["status"] == "FAIL" and r["severity"] == "CRITICAL"]
    if critical:
        names = ", ".join(f"{r['check_name']} (expected {r['expected']}, got {r['actual']})" for r in critical)
        raise ContractViolation(f"{len(critical)} critical check(s) failed: {names}")
    return [r for r in results if r["status"] == "WARN"]
```

Collecting before raising means one run reports every CRITICAL failure. Filtering on both status and severity keeps a MEDIUM failure from stopping the pipeline.

A6-L1 exercise 5

Hints. (1) Which `sum` does line 11 call? (2) An import can replace a built-in name for the whole module. (3) Import `functions as F` instead of importing `sum` by name.

Answer. The bug is on line 2.

Fixed version:

```
from dataclasses import dataclass, field
from pyspark.sql import functions as F


@dataclass
class ContractValidationResult:
    checks: list = field(default_factory=list)

    @property
    def pass_rate(self) -> float:
        passed = sum(1 for c in self.checks if c.status == "PASS")
        return passed / len(self.checks) if self.checks else 0
```

Line 2 imports Spark's `sum`, which replaces Python's built-in for the whole module. Line 11 then passes a generator to a Spark column function, which fails. Import the module as `F` and use `F.sum` where you need Spark's version. This is defect 2 in your reference README.

Common wrong answers.

- Line 11: The generator and the built-in sum are correct. But which `sum` does this line call?
- Line 12: The empty-list guard is fine. The failure happens one line earlier, caused by an import.

A6-L1 exercise 6

Hints. (1) Three behaviours: keep, drop, fail. (2) The requirement says never reach the table, but keep going. (3) Drop.

Answer. b) `@dp.expect_or_drop("valid_id", "respondent_id IS NOT NULL")` Because: r1) expect_or_drop removes violating rows and records how many were dropped.

`expect` keeps and records, `expect_or_drop` drops and records, `expect_or_fail` stops. Choose the one that matches the severity of the rule.

Common wrong answers.

- `a`: `expect` keeps violating rows and only records metrics. NULL keys would reach the table.
- `c`: `expect_or_fail` stops the whole update on the first violation. The requirement keeps valid rows flowing.
- `d`: A later filter lets bad rows land first, and loses the violation metrics.
- `b|r2`: Right decorator, wrong reason. Failing the update is expect_or_fail. expect_or_drop drops the rows and keeps going.

A6-L2 Statistical checks that survive empty, single-row and zero data

A6-L2 exercise 1

Hints. (1) `F.stddev` is an alias for `stddev_samp`. (2) The sample formula divides by n - 1. (3) With n = 1 the result is NULL, printed as None.

Answer.

```
1 None
```

`F.stddev` computes the sample standard deviation, dividing by n - 1. For one row that is undefined, so the result is NULL. Your checks must expect it.

Common wrong answers.

- `0.0`: `F.stddev` is the sample standard deviation, which divides by n - 1. With one row that is undefined, so Spark returns NULL.
- `1.2`: That is the mean. The spread of a single value is undefined for the sample formula.

A6-L2 exercise 2

Hints. (1) Which mode is on by default in Spark 4.0? (2) Does that mode tolerate division by zero? (3) It raises, and the message suggests try_divide.

Answer. c) The query fails with DIVIDE_BY_ZERO. Because: r1) ANSI mode is on by default in Spark 4.0, and it raises on division by zero, DOUBLE included.

Under ANSI mode, division by zero raises `DIVIDE_BY_ZERO`, even for doubles. Use `F.try_divide(a, b)` or `F.when(b > 0, a / b)`.

Common wrong answers.

- `a`: Without ANSI mode some engines return Infinity for doubles. Spark 4.0 with ANSI on raises DIVIDE_BY_ZERO.
- `b`: NULL is what `F.try_divide` returns. The plain `/` raises under ANSI mode.
- `d`: No engine returns 0.0 for a division by zero. Spark 4.0 raises.
- `c|r2`: Right outcome, wrong reason. The error comes from the ANSI division, before anything is written.

A6-L2 exercise 3

Hints. (1) The safe division function starts with try. (2) CV = standard deviation / mean. (3) `try_divide` and `mean`.

Answer.

```
cv = F.try_divide(F.stddev("final_weight"), F.mean("final_weight"))
```

`F.try_divide(F.stddev(w), F.mean(w))` gives the CV, or NULL when the mean is 0 or NULL.

Common wrong answers.

- g1 = `divide`: There is no `F.divide`. The safe version is `F.try_divide`, which returns NULL for a zero divisor.
- g2 = `stddev`: The CV divides the standard deviation by the mean.

A6-L2 exercise 4

Hints. (1) Check n first, then cv with `is None`. (2) Only compare cv with max_cv once you know it is a number. (3) Return the status and a short reason in a tuple.

Answer.

```
def check_weight_spread(stats: dict[str, float | int | None], max_cv: float = 1.5) -> tuple[str, str]:
    """Classify the weight coefficient of variation.

    Args:
        stats: n (row count) and cv (None when undefined).
        max_cv: Largest CV that passes.

    Returns:
        (status, reason), where status is SKIP, WARN or PASS.
    """
    n = stats["n"]
    cv = stats["cv"]
    if n is None or n < 2:
        return ("SKIP", "fewer than 2 rows")
    if cv is None:
        return ("SKIP", "cv undefined")
    if cv > max_cv:
        return ("WARN", f"cv {cv:.4f} above {max_cv}")
    return ("PASS", f"cv {cv:.4f}")
```

Ordering the checks means the comparison only ever sees a real number. `is None` keeps a CV of 0.0 as a pass.

A6-L2 exercise 5

Hints. (1) What is `0.0 and 0.12` in Python? (2) The block is skipped for a share of exactly zero. (3) Use `is not None` on both shares.

Answer. The bug is on line 3.

Fixed version:

```
def compare_share(metric: str, current_pct: float | None, prior_pct: float | None, tolerance: float = 0.10) -> dict[str, object] | None:
    """Return a stability result, or None when a share is missing."""
    if current_pct is not None and prior_pct is not None:
        change = abs(current_pct - prior_pct)
        return {"metric": metric, "change_pp": round(change, 4), "status": "WARN" if change > tolerance else "PASS"}
    return None
```

`0.0` is falsy, so the largest possible drop skipped the comparison. This is defect 8 in your reference README. Test for missing values with `is not None`.

Common wrong answers.

- Line 5: The rounding and status logic are fine once reached. The share of 0.0 never reaches them.
- Line 4: The subtraction is correct. The problem is the condition guarding it.

A6-L3 Reconcile counts and totals between layers

A6-L3 exercise 1

Hints. (1) What does silver keep when a key appears twice? (2) What does bronze keep? (3) Count distinct keys in bronze.

Answer. b) The number of distinct (respondent_id, wave_id) keys in bronze for W14 Because: r1) Silver keeps one row per key, the newest version, so it matches the number of distinct keys across all deliveries.

Silver holds one row per `(respondent_id, wave_id)`. The matching bronze figure is the number of distinct keys across all deliveries of the wave.

Common wrong answers.

- `a`: Bronze holds both deliveries, so its row count is roughly double. Silver keeps one row per key.
- `c`: Only if both deliveries hold exactly the same keys. Distinct keys is correct in every case.
- `d`: A redelivery can add or drop respondents. Silver keeps every key ever delivered for the wave, at its newest version.
- `b|r2`: Right figure, wrong reason. Bronze is append-only and keeps every delivery.

A6-L3 exercise 2

Hints. (1) Can 0.1 be stored exactly as a binary float? (2) What is 0.1 + 0.2 printed in full? (3) isclose allows a tiny relative difference.

Answer.

```
1 False True
```

`0.1 + 0.2` is `0.30000000000000004`, so `==` is False. `math.isclose` allows a relative difference of 1e-9, so it is True. Weight totals need the same treatment.

Common wrong answers.

- `True True`: 0.1 and 0.2 have no exact binary representation, so their sum is 0.30000000000000004. Exact equality fails.

- `False False`: `math.isclose` compares with a relative tolerance of 1e-9 by default, so it treats the two values as equal.

A6-L3 exercise 3

Hints. (1) `math.isclose(expected, actual, rel_tol=1e-9, abs_tol=1e-6)`. (2) Format numbers with an f-string: `f"{x:.2f}"`. (3) `difference` is `actual - expected`, as a float.

Answer.

```python
import math


def reconciliation_result(name: str, expected: float, actual: float) -> dict[str, object]:
    """Compare two layer totals and return a CRITICAL check result.

    Args:
        name: Check name, for example weight_total_silver_gold.
        expected: Total from the upstream layer.
        actual: Total from the downstream layer.

    Returns:
        A check result with status PASS or FAIL and the difference.
    """
    ok = math.isclose(expected, actual, rel_tol=1e-9, abs_tol=1e-6)
    return {
        "check_name": name,
        "status": "PASS" if ok else "FAIL",
        "severity": "CRITICAL",
        "expected": f"{expected:.2f}",
        "actual": f"{actual:.2f}",
        "difference": actual - expected,
    }
```

`rel_tol` absorbs rounding in large totals. `abs_tol` covers totals near zero, where any relative tolerance is meaningless.

A6-L3 exercise 4

Hints. (1) What does bronze hold after a redelivery? (2) Silver has one row per key. Which bronze figure matches that? (3) Count distinct respondent IDs within the wave.

Answer. The bug is on line 3.

Fixed version:

```python
def bronze_silver_keys(bronze: DataFrame, silver: DataFrame, wave_id: str) -> tuple[int, int]:
    """Keys in bronze and rows in silver for one wave."""
```

```
3     b = bronze.filter(F.col("wave_id") ==
    wave_id).agg(F.count_distinct("respondent_id")).collect()[0][0]
4     s = silver.filter(F.col("wave_id") == wave_id).count()
5     return b, s
```

Bronze keeps every delivery, so its raw count grows with each redelivery. Count distinct keys for the wave. Within one wave, distinct `respondent_id` is the same as distinct `(respondent_id, wave_id)`.

Common wrong answers.

- Line 4: Silver has one row per key, so its row count is the right figure. The bronze side is wrong.

A6-L3 exercise 5

Hints. (1) Each layer is checked before the next reads it. (2) Nothing is published before every check passes. (3) Record the evidence, then publish.

Answer.

1. MERGE the wave into silver
2. Reconcile bronze distinct keys with silver rows
3. Build gold metrics for the wave
4. Reconcile silver in-scope totals with gold totals
5. Store reconciliation results with the run log
6. Refresh the Power BI semantic model

Load, check, load, check, record, publish. A failed reconciliation stops the sequence before clients see anything.

Common wrong answers.

- Reconcile silver before building gold, so a bad silver load never feeds gold.
- Store the results before publishing, so the published wave has a record of passing.
- Publishing before the silver-to-gold reconciliation lets unreconciled numbers reach clients.

A7-L1 Structured logging with the logging module, not print

A7-L1 exercise 1

Hints. (1) What is the default level threshold? (2) Is INFO above or below WARNING? (3) Only the warning appears, as the bare message.

Answer.

```
3 unexpected labels
```

With no configuration, the effective level is WARNING and Python's last-resort handler prints just the message. Configure a level and format in the entry point to see INFO lines with context.

Common wrong answers.

- `loaded 3 unexpected labels`: Without configuration, loggers pass WARNING and above. The INFO line is dropped.
- `WARNING:silver:3 unexpected labels`: That format comes from `logging.basicConfig()`. Without it, Python's last-resort handler prints the message only.

A7-L1 exercise 2

Hints. (1) The module's own name is a built-in variable. (2) One logger method adds the traceback automatically. (3) A bare statement re-raises the current exception.

Answer.

```python
logger = logging.getLogger(__name__)

try:
    rows = do_merge(df)
except Exception:
    logger.exception("merge failed")
    raise
```

`getLogger(__name__)` names the logger after the module. `logger.exception` logs at ERROR with the traceback. A bare `raise` keeps the original exception and fails the job.

Common wrong answers.

- g1 = `"root"`: A hardcoded name loses the module path. `__name__` names the logger after the module automatically.
- g2 = `error`: `error` logs at ERROR but without the traceback, unless you add exc_info=True. `exception` includes it.
- g3 = `pass`: `pass` swallows the failure, so the job reports success. Re-raise with a bare `raise`.

A7-L1 exercise 3

Hints. (1) What does the scheduler see when the function returns normally? (2) Returning 0 looks like an empty but successful merge. (3) Replace `return 0` with `raise`.

Answer. The bug is on line 8.

Fixed version:

```
1 def merge_wave(df, run_id: str, wave_id: str) -> int:
2     """MERGE one wave. Logs failures."""
3     log = logging.LoggerAdapter(logger, {"run_id": run_id, "wave_id":
    wave_id})
4     try:
5         return do_merge(df)
6     except Exception:
7         log.exception("merge failed")
8         raise
```

Returning 0 converts a failure into an apparently successful empty run. Log it, then `raise`, so the job fails and failure notifications fire.

Common wrong answers.

- Line 7: Logging the exception with its traceback is right. The problem is what happens next.
- Line 5: The MERGE call is fine. Look at how its failure is handled.

A7-L1 exercise 4

Hints. (1) `logging.LoggerAdapter(logger, {...})` attaches context. (2) Branch on `rows == 0`. (3) `log.warning("merge wrote 0 rows")` and `log.info("merge finished rows=%d", rows)`.

Answer.

```
1  import logging
2
3
4  def wave_logger(run_id: str, wave_id: str) -> logging.LoggerAdapter:
5      """Module logger with run and wave context on every line."""
6      return logging.LoggerAdapter(logging.getLogger("pipelines.silver"),
     {"run_id": run_id, "wave_id": wave_id})
7
8
9  def log_merge_result(log: logging.LoggerAdapter, rows: int) -> None:
10     """INFO for a normal merge, WARNING for an empty one."""
11     if rows == 0:
12         log.warning("merge wrote 0 rows")
13     else:
14         log.info("merge finished rows=%d", rows)
```

The adapter puts run and wave IDs on every line without repeating them in each call. A separate WARNING for an empty merge makes the suspicious case stand out.

A7-L1 exercise 5

Hints. (1) What would you want to see on a normal night? (2) What should stand out without stopping the run? (3) Bulky detail only when debugging.

Answer.

- Normal milestones: INFO.
- A result that is possible but suspicious: WARNING.
- A failure, logged with log.exception before re-raising: ERROR.
- Bulky diagnostic detail: DEBUG, switched on only when needed.

Levels let one log serve two audiences: INFO and above for the nightly check, DEBUG for the engineer troubleshooting.

Common wrong answers.

- zero = `info`: Zero rows usually means an upstream problem. At INFO it disappears among normal lines.
- fail = `warning`: A failed MERGE stops the run. That is ERROR, with the traceback.
- plan = `info`: A full plan on every run floods the log. Keep it at DEBUG.

A7-L2 A pipeline run log table with an explicit schema

A7-L2 exercise 1

Hints. (1) Where does Spark get the column types here? (2) What type can it infer from None alone? (3) It raises.

Answer. b) It fails with CANNOT_DETERMINE_TYPE. Because: r1) Spark infers each column's type from the data, and a column holding only None gives it nothing to infer from.

With no schema, Spark infers types from values. A column of only `None` has no type to infer, so it raises `CANNOT_DETERMINE_TYPE`. Declare a `StructType`.

Common wrong answers.

- `a`: Only with an explicit schema. Inference sees only None and cannot choose STRING.
- `c`: Spark does not drop columns during inference. It raises.
- `d`: Inference fails before any table is involved.

- `b|r2`: Right outcome, wrong reason. None is fine when the schema is declared. The problem is inferring a type from nothing but None.

A7-L2 exercise 2

Hints. (1) The module that generates unique ids. (2) The class that provides utc. (3) `datetime.now(timezone.utc)`.

Answer.

```
1 import uuid
2 from datetime import datetime, timezone
3
4 run_id = str(uuid.uuid4())
5 started_at = datetime.now(timezone.utc)
```

`import uuid` and `from datetime import datetime, timezone` supply every name. `datetime.now(timezone.utc)` gives a timezone-aware UTC timestamp.

Common wrong answers.

- g3 = `utcnow`: `datetime.utcnow()` is deprecated from Python 3.12 and returns a naive datetime. Use `datetime.now(timezone.utc)`.
- g2 = `date`: You need `timezone`, for `timezone.utc`.

A7-L2 exercise 3

Hints. (1) What does Spark infer from a column that holds only None? (2) Where can you tell Spark the types instead? (3) Build a StructType and pass it to createDataFrame with a tuple row.

Answer. The bug is on line 3.

Fixed version:

```
1 def run_record(spark: SparkSession, run_id: str, rows: int, status: str,
    error_message: str | None) -> DataFrame:
2     """One run log row."""
3     schema = T.StructType([
4         T.StructField("run_id", T.StringType(), False),
5         T.StructField("rows_processed", T.LongType(), False),
6         T.StructField("status", T.StringType(), False),
7         T.StructField("error_message", T.StringType(), True),
8     ])
9     return spark.createDataFrame([(run_id, rows, status, error_message)],
    schema)
```

Without a schema, the successful case has `error_message=None` only, and inference fails. A `StructType` declares the type, so None is simply NULL.

Common wrong answers.

- Line 7: Passing None is correct for a success. The problem is how Spark decides the column's type.

A7-L2 exercise 4

Hints. (1) Declare seven StructFields in order, with error_message nullable. (2) Build the row as a tuple in the same order. (3) `spark.createDataFrame([row], RUN_LOG_SCHEMA)`.

Answer.

```
from datetime import datetime, timezone

from pyspark.sql import DataFrame, SparkSession
from pyspark.sql import types as T

RUN_LOG_SCHEMA = T.StructType([
    T.StructField("run_id", T.StringType(), False),
    T.StructField("pipeline_name", T.StringType(), False),
    T.StructField("rows_processed", T.LongType(), False),
    T.StructField("status", T.StringType(), False),
    T.StructField("error_message", T.StringType(), True),
    T.StructField("started_at", T.TimestampType(), False),
    T.StructField("finished_at", T.TimestampType(), False),
])


def run_record(spark: SparkSession, run_id: str, pipeline_name: str, rows_processed: int, status: str, started_at: datetime, error_message: str | None = None) -> DataFrame:
    """One run log row typed by RUN_LOG_SCHEMA.

    Args:
        spark: Active session.
        run_id: Unique id of the run.
        pipeline_name: Job name.
        rows_processed: Rows written by the run.
        status: SUCCESS or FAILED.
        started_at: UTC start time.
        error_message: Error text for a failed run.

    Returns:
        A one-row DataFrame ready to append to the run log table.
    """
    finished_at = datetime.now(timezone.utc)
    row = (run_id, pipeline_name, rows_processed, status, error_message, started_at, finished_at)
    return spark.createDataFrame([row], RUN_LOG_SCHEMA)
```

The module-level schema makes every record the same shape, whatever its values. Tuples in schema order are the simplest row format.

A7-L2 exercise 5

Hints. (1) The id must exist before any work is logged. (2) try, except, finally is Python's fixed order. (3) finally runs whether the work succeeded or raised.

Answer.

1. Create run_id and started_at
2. try: run the layer and read rows_processed from the commit metrics
3. Set status SUCCESS
4. except: set status FAILED and error_message, log.exception, then raise
5. finally: append the run record to the run log table

`finally` runs after both paths, so every run is recorded with its true status, and the re-raise in `except` still fails the job.

Common wrong answers.

- The except block belongs before finally in the try statement, and the record must see the FAILED status the except block sets.
- Create the run id first, so log lines during the work carry it.

A7-L3 Alert on failures, and on the silence that hides them

A7-L3 exercise 1

Hints. (1) Does the run fail? (2) When does on_failure send? (3) No failure, no email.

Answer. b) Nothing. Because: r1) The task returns normally, so the run succeeds and on_failure never fires.

Catching and not re-raising turns a failure into a success, so the failure notification never fires. Log, record FAILED, then re-raise.

Common wrong answers.

- `a`: on_failure fires when the run fails. Swallowing the exception made the run succeed.
- `c`: There is no warning notification for a caught exception. The run simply succeeds.
- `d`: Only if on_success were configured. It is not.

- `b|r2`: Right outcome, wrong reason. Notifications work for every task type. The run succeeded because the exception was swallowed.

A7-L3 exercise 2

Hints. (1) The event key names what happened to the run. (2) Send to the on-call group address. (3) `on_failure` and `data-oncall@example.org`.

Answer.

```
email_notifications:
  on_failure:
    - data-oncall@example.org
```

`email_notifications.on_failure` lists the addresses to notify when a run fails. Use a group address so the alert survives holidays and team changes.

Common wrong answers.

- g1 = `on_error`: The key is `on_failure`. Other keys include on_start and on_success.
- g2 = `sam.jones@example.org`: Send alerts to a group address. One person's inbox goes quiet when they are away.

A7-L3 exercise 3

Hints. (1) Which values does the table hold? (2) Does 'FAIL' equal 'FAILED'? (3) Use status = 'FAILED'.

Answer. The bug is on line 3.

Fixed version:

```
SELECT pipeline_name, run_id, error_message
FROM research_prod.research_governance.pipeline_run_log
WHERE status = 'FAILED'
  AND finished_at >= current_timestamp() - INTERVAL 1 DAY;
```

String comparison is exact, so `'FAIL'` never matches `'FAILED'`. Typed statuses in the writer (Literal) and a test query against real rows catch this.

Common wrong answers.

- Line 4: The one-day window is fine. Compare the status literal with what the table stores.

A7-L3 exercise 4

Hints. (1) Track every pipeline name, starting with no success. (2) Keep the latest SUCCESS finished_at per pipeline. (3) Stale when there is no success, or the latest is before now - max_age_hours.

Answer.

```python
from datetime import datetime, timedelta


def stale_pipelines(runs: list[dict[str, object]], now: datetime, max_age_hours: int = 26) -> list[str]:
    """Pipelines whose latest SUCCESS is older than max_age_hours, or that never succeeded.

    Args:
        runs: Run log rows with pipeline_name, status and finished_at.
        now: Current time, timezone-aware.
        max_age_hours: Largest acceptable age of the latest success.

    Returns:
        Sorted pipeline names.
    """
    cutoff = now - timedelta(hours=max_age_hours)
    latest: dict[str, datetime | None] = {}
    for r in runs:
        name = str(r["pipeline_name"])
        latest.setdefault(name, None)
        finished = r["finished_at"]
        if r["status"] == "SUCCESS" and isinstance(finished, datetime):
            current = latest[name]
            if current is None or finished > current:
                latest[name] = finished
    return sorted(name for name, last in latest.items() if last is None or last < cutoff)
```

Recording every pipeline first means one that never succeeded still appears. Comparing only the latest success stops a single later failure from raising a false alarm.

A7-L3 exercise 5

Hints. (1) Which problems make the job fail? (2) Which problems leave the job successful, or never started? (3) Is every change worth waking someone?

Answer.

- Exceptions fail the job: the failure notification handles them, provided nothing swallows them.
- A job that never runs cannot fail: detect the missing SUCCESS with a query.
- A success that did nothing: query for rows_processed = 0.
- Normal variation is not an alert. Alerts that fire on noise get ignored.

Failures that raise are covered by job notifications. Silence and empty successes need scheduled queries. Noise needs no alert at all.

Common wrong answers.

- paused = `job`: A paused job never runs, so it never fails, and the failure notification never fires.
- zero_rows = `job`: The job succeeds, so no failure notification fires. Only a query on the run log sees the zero.
- slow = `query`: A one-minute change is noise. Alerting on it trains people to ignore alerts.

A8-L1 Process only the waves you have not processed, correctly

A8-L1 exercise 1

Hints. (1) Which silver waves does the function compare against? (2) Is W14 in silver, for any tracker? (3) Both of A's waves appear in silver.

Answer. b) None Because: r1) Tracker B's W14 in silver makes tracker A's W14 look done.

Without a tracker filter on silver, any tracker's wave counts as done. A's W14 never loads, and nothing fails.

Common wrong answers.

- `a`: That is the correct answer to the question the function should ask. Because silver is not filtered by tracker, B's W14 hides A's.
- `c`: W13 is in silver for tracker A, so it is correctly excluded. W14 is wrongly excluded too.
- `d`: W13 is already loaded for tracker A.
- `b|r2`: Right outcome, wrong reason. subtract removes only matching values. Here every value matches, because silver mixes trackers.

A8-L1 exercise 2

Hints. (1) The Catalog method that checks whether a table exists. (2) The join type that keeps left rows without a match. (3) `tableExists` and `left_anti`.

Answer.

```
1 if not spark.catalog.tableExists(silver):
2     return sorted(r.wave_id for r in source.collect())
3 new = source.join(done, "wave_id", "left_anti")
```

`spark.catalog.tableExists` covers the first run. A `left_anti` join keeps bronze waves with no matching silver wave.

Common wrong answers.

- g2 = `left`: A left join keeps every bronze wave, matched or not. You want the ones with no match: left_anti.
- g2 = `left_semi`: left_semi keeps the waves that do match: the ones already done. You want the opposite.
- g1 = `exists`: The Catalog method is `tableExists`.

A8-L1 exercise 3

Hints. (1) Compare lines 3 and 4. (2) Which side lacks the tracker filter? (3) Add `.filter(F.col("tracker_id") == tracker_id)` to silver.

Answer. The bug is on line 4.

Fixed version:

```
1 def unprocessed_waves(spark: SparkSession, bronze: str, silver: str,
    tracker_id: str) -> list[str]:
2
    """Waves in bronze for this tracker that silver does not hold for this
    tracker."""
3     source = spark.table(bronze).filter(F.col("tracker_id") ==
    tracker_id).select("wave_id").distinct()
4     done = spark.table(silver).filter(F.col("tracker_id") ==
    tracker_id).select("wave_id").distinct()
5     return sorted(r.wave_id for r in source.join(done, "wave_id",
    "left_anti").collect())
```

Silver is read for all trackers, so another tracker's wave hides this tracker's wave. Filter both sides by the same keys.

Common wrong answers.

- Line 3: Bronze is filtered by tracker, which is right. Compare it with how silver is read.
- Line 5: The left anti join is correct once both sides cover the same tracker.

A8-L1 exercise 4

Hints. (1) Build the bronze waves for the tracker first. (2) Return them all if silver does not exist. (3) Otherwise filter silver the same way and left-anti-join.

Answer.

```
1 from pyspark.sql import SparkSession
2 from pyspark.sql import functions as F
3
4
5 def unprocessed_waves(spark: SparkSession, bronze: str, silver: str,
    tracker_id: str) -> list[str]:
```

```
    """Wave ids in bronze for this tracker that silver does not hold for this tracker.

    Args:
        spark: Active session.
        bronze: Bronze table name.
        silver: Silver table name. May not exist yet.
        tracker_id: Tracker to scope both sides to.

    Returns:
        Sorted wave ids to process.
    """
    source = spark.table(bronze).filter(F.col("tracker_id") == tracker_id).select("wave_id").distinct()
    if not spark.catalog.tableExists(silver):
        return sorted(r.wave_id for r in source.collect())
    done = spark.table(silver).filter(F.col("tracker_id") == tracker_id).select("wave_id").distinct()
    return sorted(r.wave_id for r in source.join(done, "wave_id", "left_anti").collect())
```

Scoping both sides by tracker, handling the missing table and anti-joining gives the exact set of new waves. Sorting keeps runs deterministic.

A8-L1 exercise 5

Hints. (1) What does the detection compare? (2) Is W14 on both sides? (3) Redeliveries need a signal beyond the wave ID.

Answer. b) It is skipped, because W14 is already in silver. Because: r1) The detection compares wave IDs only, and W14 exists on both sides.

Wave-ID detection cannot see redeliveries. Compare the latest ingestion timestamp per wave, or read bronze incrementally with change data feed or Auto Loader.

Common wrong answers.

- `a`: The detection never looks at ingestion_id. It compares wave IDs, and W14 is in silver.
- `c`: Nothing runs for W14, so nothing can fail.
- `d`: W14 is not selected at all.
- `b|r2`: Right outcome, wrong reason. Bronze appends every redelivery. The detection is what skips it.

A8-L2 Auto Loader for file ingestion into bronze

A8-L2 exercise 1

Hints. (1) Where does a stream keep its progress? (2) It is a separate path, given in the writeStream options. (3) checkpointLocation.

Answer. b) The checkpoint location Because: r1) Auto Loader records discovered files in a key-value store in the checkpoint and resumes from it.

The checkpoint holds a RocksDB store of discovered files and the stream's progress. On restart, Auto Loader continues from it, so files are neither skipped nor repeated.

Common wrong answers.

- `a`: The schema location stores the inferred schema, not the list of processed files.
- `c`: _rescued_data holds values that did not fit the schema. It says nothing about which files were loaded.
- `d`: Auto Loader never reads the target table to decide what to load. The checkpoint does that.
- `b|r2`: Right answer, wrong reason. Auto Loader leaves source files in place. It records them in the checkpoint.

A8-L2 exercise 2

Hints. (1) The source format has the same prefix as its options. (2) Where the inferred schema is stored. (3) The trigger that processes everything available, then stops.

Answer.

```
raw = (
    spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("cloudFiles.schemaLocation", schema_path)
    .load(landing)
)
raw.writeStream.option("checkpointLocation",
   checkpoint).trigger(availableNow=True).toTable(table)
```

`format("cloudFiles")` selects Auto Loader. `cloudFiles.schemaLocation` stores the schema. `trigger(availableNow=True)` runs the stream as a batch.

Common wrong answers.

- g1 = `autoloader`: The source format is named `cloudFiles`.

- g3 = `once`: `once=True` processes everything in a single micro-batch. `availableNow=True` also stops when done, but can split a large backlog into several batches, which Databricks recommends for Auto Loader.
- g2 = `checkpointLocation`: The checkpoint goes on the writer. The reader option for the schema store is `cloudFiles.schemaLocation`.

A8-L2 exercise 3

Hints. (1) Which mode applies when you do not provide a schema? (2) How does that mode pick up a new column? (3) Fail once, restart with the new schema.

Answer. b) The stream stops with UnknownFieldException after recording the new schema. The next start includes region. Because: r1) The default mode without a schema is addNewColumns, which evolves by failing once and restarting.

Without a schema, the default is `addNewColumns`: the stream stops with `UnknownFieldException`, the schema location gains the column, and a restart continues. Jobs with retries make that restart automatic.

Common wrong answers.

- `a`: Nothing is dropped silently. In addNewColumns mode the stream stops so the new column is picked up.
- `c`: That describes the `rescue` mode. The default without a schema is `addNewColumns`.
- `d`: Auto Loader never rebuilds the target. It evolves the schema for new files.
- `b|r2`: Right outcome, wrong reason. It does change the schema: it records the evolved schema, then needs a restart.

A8-L2 exercise 4

Hints. (1) How many streams share the same checkpoint? (2) A checkpoint records one stream's progress. (3) Give each stream its own checkpoint path.

Answer. The bug is on line 1.

Fixed version:

```
1 def start_ingest(spark: SparkSession, landing: str, table: str,
    schema_path: str, checkpoint: str) -> StreamingQuery:
2     """Load new files from landing into table. checkpoint must be unique
    to this stream."""
3     raw =
    spark.readStream.format("cloudFiles").option("cloudFiles.format",
    "csv").option("cloudFiles.schemaLocation", schema_path).load(landing)
```

```
4     return raw.writeStream.option("checkpointLocation",
    checkpoint).trigger(availableNow=True).toTable(table)
```

Both streams write their progress to the same checkpoint, so each reads the other's state and skips files. One checkpoint per stream, passed in from config.

Common wrong answers.

- Line 6: The read is fine, with its own schema location. The problem is on the write side.

A8-L2 exercise 5

Hints. (1) Latency or cost? (2) The checkpoint must be durable and private to the stream. (3) Bronze keeps evidence.

Answer.

- availableNow on a schedule: incremental processing at batch cost.
- A dedicated checkpoint per stream in a Unity Catalog volume, so it survives and is governed.
- Keep _rescued_data in bronze and count it in the contract checks: it is evidence of schema drift.

Scheduled availableNow, a durable per-stream checkpoint in a volume, and rescued data counted as a quality signal.

Common wrong answers.

- trigger = `continuous`: A continuous stream keeps compute running all day for a few files. Scheduled availableNow costs far less.
- checkpoint = `tmp`: Temporary storage can be cleared. Losing the checkpoint reloads every file.
- checkpoint = `shared`: Streams must not share a checkpoint. They would skip each other's files.
- rescued = `drop`: Dropping _rescued_data in bronze destroys evidence. Bronze keeps everything.

A8-L3 Change data feed, so downstream layers read only what changed

A8-L3 exercise 1

Hints. (1) An update records two versions of the row. (2) Which one comes after the change? (3) post = after.

Answer. c) update_postimage Because: r1) An update records the row before (pre-image) and after (post-image). The post-image holds the new values.

An update writes `update_preimage` (old values) and `update_postimage` (new values). Use the post-image, and drop the pre-image.

Common wrong answers.

- `a`: insert marks a new row. An update of an existing row produces a pre-image and a post-image.
- `b`: The pre-image is the row before the update, with the old weight.
- `d`: delete marks a removed row.
- `c|r2`: Right row type, wrong reason. Updates appear as update_preimage and update_postimage, not delete plus insert.

A8-L3 exercise 2

Hints. (1) Which versions does table_changes('c1', 2) read? (2) How many rows does one updated row produce? (3) Two for the update, one for the delete.

Answer.

```
3
```

From version 2: `update_preimage` and `update_postimage` for id 1, and `delete` for id 2. Three rows. This was run on delta-spark 4.0.1.

Common wrong answers.

- `2`: The update produces two rows, pre-image and post-image, plus one delete row: three.
- `5`: Version 1's inserts come before the starting version 2, so they are not included.

A8-L3 exercise 3

Hints. (1) The read option names what you are reading. (2) startingVersion is inclusive. (3) `readChangeFeed` and `last_processed_version + 1`.

Answer.

```
changes = (
    spark.read.option("readChangeFeed", "true")
    .option("startingVersion", last_processed_version + 1)
    .table(silver)
)
```

`readChangeFeed` switches the read to the change feed. Starting at `last_processed_version + 1` reads each change exactly once.

Common wrong answers.

- g2 = `last_processed_version`: startingVersion is inclusive. Starting at the last processed version reads its changes twice.
- g1 = `enableChangeDataFeed`: `delta.enableChangeDataFeed` is the table property. The read option is `readChangeFeed`.
- g2 = `0`: Version 0 rereads all history every run, and fails if CDF was enabled later.

A8-L3 exercise 4

Hints. (1) Filter out update_preimage first. (2) Window: partition by keys, order by _commit_version descending. (3) Keep row_number 1, drop the helper column.

Answer.

```python
from pyspark.sql import DataFrame, Window
from pyspark.sql import functions as F


def latest_state(changes: DataFrame, keys: list[str]) -> DataFrame:
    """One row per key: its newest change, excluding update pre-images.

    Args:
        changes: Rows read with readChangeFeed.
        keys: Columns identifying one row of the source table.

    Returns:
        The newest insert, update_postimage or delete per key.
    """
    window = Window.partitionBy(*keys).orderBy(F.col("_commit_version").desc())
    return (
        changes.filter(F.col("_change_type") != "update_preimage")
        .withColumn("_rank", F.row_number().over(window))
        .filter(F.col("_rank") == 1)
        .drop("_rank")
    )
```

Pre-images share a commit version with their post-images, so without the filter a pre-image can win the ranking and bring back old values. Ranking by version keeps the newest change per key.

A8-L3 exercise 5

Hints. (1) Which change types are not deletes? (2) One of them holds the old values. (3) Keep insert and update_postimage only.

Answer. The bug is on line 4.

Fixed version:

```
1 def current_rows(spark: SparkSession, table: str, last_processed_version:
    int) -> DataFrame:
2     """Inserted and updated rows since the last run, with their new
    values."""
3     changes = spark.read.option("readChangeFeed",
    "true").option("startingVersion", last_processed_version +
    1).table(table)
4     return changes.filter(F.col("_change_type").isin(["insert",
    "update_postimage"]))
```

Excluding only deletes still keeps `update_preimage`, the old version of each updated row. Name the change types you want: `insert` and `update_postimage`.

Common wrong answers.

- Line 3: The read starts after the last processed version, which is correct.

A9-L1 pytest basics, and a SparkSession fixture shared by every test

A9-L1 exercise 1

Hints. (1) How long does a SparkSession take to start? (2) How often should it start in one test run? (3) Once.

Answer. b) session Because: r1) Spark takes seconds to start, and tests can safely share one local session.

`scope="session"` starts Spark once for the whole run. Tests stay independent as long as each creates its own data and tables.

Common wrong answers.

- `a`: Function scope rebuilds the fixture for every test: 40 Spark start-ups.
- `c`: Same cost as function scope, plus duplicated set-up code in every test.
- `d`: Class scope rebuilds per test class. Session scope builds once.
- `b|r2`: Right scope, wrong reason. Fixtures of any scope can yield. Session scope wins on start-up cost.

A9-L1 exercise 2

Hints. (1) How many values does parametrize receive? (2) Each value becomes its own test. (3) 3 + 1.

Answer.

```
4
```

`parametrize` generates one test per value, so pytest reports `test_wave_loads[W13]`, `[W14]`, `[W15]` and `test_empty_wave_list`: four.

Common wrong answers.

- `2`: Each parametrize value is a separate test: three for test_wave_loads, plus one.
- `3`: Three from the parametrized test, plus test_empty_wave_list: four.

A9-L1 exercise 3

Hints. (1) The decorator that declares a fixture. (2) The widest scope. (3) The keyword that pauses a function and resumes it later.

Answer.

```python
@pytest.fixture(scope="session")
def spark() -> Iterator[SparkSession]:
    session = SparkSession.builder.master("local[2]").getOrCreate()
    yield session
    session.stop()
```

`@pytest.fixture(scope="session")` builds the fixture once. `yield` gives the session to the tests, and the line after it runs at the end.

Common wrong answers.

- g3 = `return`: `return` ends the function, so `session.stop()` never runs. `yield` hands over the session and resumes for teardown.
- g2 = `module`: Module scope starts Spark once per test file. Session scope starts it once for the run.

A9-L1 exercise 4

Hints. (1) Pick inputs where the weighted and unweighted answers differ. (2) [1, 0] with weights [3.0, 1.0]: weighted 0.75, unweighted 0.5. (3) Add `assert weighted_share([1, 0], [0.0, 0.0]) is None`.

Answer.

```python
def test_weighted_share() -> None:
    assert weighted_share([1, 0], [3.0, 1.0]) == 0.75
    assert weighted_share([1, 0], [0.0, 0.0]) is None
```

A test proves something only if a plausible bug makes it fail. Known answers chosen to separate the right rule from the wrong one, plus the edge case, catch both bugs.

A9-L1 exercise 5

Hints. (1) What is the default fixture scope? (2) How many times does Spark start in 45 tests? (3) `@pytest.fixture(scope="session")`.

Answer. The bug is on line 5.

Fixed version:

```
1 import pytest
2 from pyspark.sql import SparkSession
3
4
5 @pytest.fixture(scope="session")
6 def spark():
7     session = SparkSession.builder.master("local[2]").getOrCreate()
8     yield session
9     session.stop()
```

The default function scope starts and stops Spark for every test, at several seconds each. Session scope starts it once.

Common wrong answers.

- Line 9: Stopping the session in teardown is right. The problem is how often the fixture runs.
- Line 7: The builder is fine. How many times is it called in the run?

A9-L2 A fast local SparkSession and readable test data

A9-L2 exercise 1

Hints. (1) The default suits clusters, not tests. (2) It is a round number in the hundreds. (3) 200.

Answer.

```
1 200
```

The default is 200, which suits large data on a cluster. On test data it creates hundreds of empty tasks per shuffle. Set it to 2.

Common wrong answers.

- `2`: Two is what you should set for tests. The default is 200.
- `auto`: Databricks can manage shuffle partitions automatically, but open-source Spark's default setting is 200.

A9-L2 exercise 2

Hints. (1) The local master takes the thread count in brackets. (2) Match the shuffle partitions to the threads. (3) The helper from the delta package ends with `_pip`.

Answer.

```
builder = (
    SparkSession.builder.master("local[2]")
    .config("spark.sql.shuffle.partitions", "2")
    .config("spark.sql.extensions",
    "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog",
    "org.apache.spark.sql.delta.catalog.DeltaCatalog")
)
spark = configure_spark_with_delta_pip(builder).getOrCreate()
```

`local[2]` runs two threads in-process, two shuffle partitions avoid empty tasks, and `configure_spark_with_delta_pip` adds the Delta JAR that matches your delta-spark version.

Common wrong answers.

- g1 = `local[1]`: One thread hides bugs that only appear with several partitions. Use two.
- g1 = `yarn`: yarn needs a cluster. Unit tests run in-process with `local[...]`.
- g3 = `configure_delta`: The delta-spark helper is `configure_spark_with_delta_pip`.

A9-L2 exercise 3

Hints. (1) Build the defaults dict inside the function. (2) Compare the override keys with the default keys. (3) Raise KeyError for unknown keys, then update and return.

Answer.

```
def respondent(**overrides: object) -> dict[str, object]:
    """A valid silver respondent row. Override only the fields a test is
    about.

    Raises:
        KeyError: If an override names an unknown field.
    """
    row: dict[str, object] = {"respondent_id": "R1", "wave_id": "W1",
    "final_weight": 1.0, "in_scope": True}
    unknown = set(overrides) - set(row)
    if unknown:
        raise KeyError(f"unknown fields: {sorted(unknown)}")
    row.update(overrides)
    return row
```

A fresh dict per call keeps tests independent. Rejecting unknown fields turns a typo in a test into an immediate error instead of a test that checks nothing.

A9-L2 exercise 4

Hints. (1) Column names alone leave Spark to infer types. (2) What can it infer from [None, None]? (3) Pass a DDL string with the types.

Answer. The bug is on line 4.

Fixed version:

```
1 def weights_df(spark: SparkSession, weights: list[float | None]) ->
    DataFrame:
2     """One respondent per weight."""
3     rows = [(f"R{i}", w) for i, w in enumerate(weights)]
4     return spark.createDataFrame(rows,
    "respondent_id STRING, final_weight DOUBLE")
```

A list of names gives Spark no types, so it infers them. With only `None` weights there is nothing to infer from. A DDL schema string fixes the types for every test.

Common wrong answers.

- Line 3: Building tuples is fine. The problem is that Spark has to guess the types.

A9-L2 exercise 5

Hints. (1) Spark runs on the JVM. (2) Tests depend on the fixture and the builders. (3) Infrastructure first, then data helpers, then tests.

Answer.

1. Install Java 17 or 21
2. Create a virtual environment and install pyspark, delta-spark and pytest
3. Write a session-scoped spark fixture in conftest.py
4. Write test data builders with explicit schemas
5. Write tests with known answers against the builders

Java, then the Python environment, then the shared fixture, then data builders, then the tests that use them.

Common wrong answers.

- Either order installs fine, but Spark needs Java before anything runs: check it first so the first test run does not fail.
- Tests need the spark fixture to create DataFrames. Write it before the tests.

A9-L3 Compare DataFrames properly, and test that MERGE is idempotent

A9-L3 exercise 1

Hints. (1) What is the default for checkRowOrder? (2) How are floats compared? (3) Both differences are tolerated.

Answer. a) Passes Because: r1) It ignores row order by default and compares floats with a relative tolerance.

`assertDataFrameEqual` ignores row order unless you set `checkRowOrder=True`, and compares floats within `rtol=1e-5`. Tested on PySpark 4.0.4.

Common wrong answers.

- `b`: checkRowOrder is False by default, so order is ignored.
- `c`: Floats compare with rtol=1e-5 by default, so 0.30000000000000004 equals 0.3.
- `d`: checkRowOrder=True would make it stricter. The default ignores order.
- `a|r2`: Right outcome, wrong reason. It compares rows too. assertSchemaEqual compares only schemas.

A9-L3 exercise 2

Hints. (1) Sorting handles the order. (2) What is 0.1 + 0.2 exactly? (3) Exact equality on floats.

Answer.

```
False
```

Sorting fixes the order but not floating-point rounding. That is why a helper with tolerances beats hand-written comparisons.

Common wrong answers.

- `True`: Sorting fixes the order, but 0.1 + 0.2 is 0.30000000000000004, so the tuples differ.

A9-L3 exercise 3

Hints. (1) The module name says what it is for. (2) The parameter name says what it checks. (3) `testing` and `checkRowOrder`.

Answer.

```
from pyspark.testing import assertDataFrameEqual, assertSchemaEqual

assertDataFrameEqual(top_regions, expected, checkRowOrder=True)
```

`pyspark.testing` provides the helpers. `checkRowOrder=True` makes order part of the comparison, which matters for ranked output.

Common wrong answers.

- g1 = `sql`: The test helpers live in `pyspark.testing`.
- g2 = `ordered`: The parameter is `checkRowOrder`.

A9-L3 exercise 4

Hints. (1) load(), snapshot, load() again. (2) The snapshot must be materialised with collect(), or it re-reads the changed table. (3) `assertDataFrameEqual(spark.table(table), first)`.

Answer.

```
1  from collections.abc import Callable
2
3  from pyspark.sql import SparkSession
4  from pyspark.testing import assertDataFrameEqual
5
6
7  def check_idempotent(spark: SparkSession, load: Callable[[], None],
     table: str) -> None:
8      """Assert that running load twice leaves table as one run did.
9
10     Raises:
11         AssertionError: If the second run changed the table.
12     """
13     load()
14     first = spark.table(table).collect()
15     load()
16     assertDataFrameEqual(spark.table(table), first)
```

`collect()` freezes the first state. A DataFrame would be lazy and re-read the table after the second run, so the comparison would always pass. `assertDataFrameEqual` then compares without caring about order.

A9-L3 exercise 5

Hints. (1) Does a groupBy return rows in a fixed order? (2) collect() lists keep that order. (3) Use assertDataFrameEqual, which ignores order.

Answer. The bug is on line 5.

Fixed version:

```
1  def test_gold_metrics(spark: SparkSession) -> None:
2      silver = spark.createDataFrame([("R1", "W1", "N", 1.0), ("R2", "W1",
     "S", 1.0)], "respondent_id STRING, wave_id STRING, region STRING,
     final_weight DOUBLE")
3      expected = spark.createDataFrame([("N", 1.0), ("S", 1.0)], "region
     STRING, weighted_base DOUBLE")
```

```
4     actual = aggregate_base(silver).select("region", "weighted_base")
5     assertDataFrameEqual(actual, expected)
```

Aggregation output has no guaranteed order, so the two lists match only when the partitions happen to line up. `assertDataFrameEqual` compares the rows as a multiset.

Common wrong answers.

- Line 4: Selecting the columns to compare is fine. The flakiness comes from how they are compared.

A10-L1 Git folders, one branch per change, and no secrets in the repository

A10-L1 exercise 1

Hints. (1) Which path includes review and tests? (2) What should be the only way into production? (3) A merge, then automation.

Answer. b) Automation, after a pull request merges into the deployment branch Because: r1) Production changes must pass review and CI, and automation makes that the only path.

Production Git folders hold the deployment branch and are updated only by automation after a merge, so every production change was reviewed and tested.

Common wrong answers.

- `a`: A developer's own testing skips review and CI. Production should only change through a merge.
- `c`: Direct edits in production drift from Git and bypass every check.
- `d`: User folders hold work in progress on personal branches. Production needs a stable deployment branch.
- `b|r2`: Right answer, wrong reason. People can edit Git folders. The rule is that production ones change only through automation.

A10-L1 exercise 2

Hints. (1) The dbutils module named after what it stores. (2) The key names the secret inside the scope. (3) `secrets` and `supplier_api_key`.

Answer.

```
1  token = dbutils.secrets.get(scope="research", key="supplier_api_key")
```

`dbutils.secrets.get(scope=..., key=...)` reads the secret at run time, and Databricks redacts it in notebook output.

Common wrong answers.

- g1 = `widgets`: Widgets hold job parameters, which appear in the job UI and run history. Secrets come from `dbutils.secrets`.
- g1 = `fs`: `dbutils.fs` reads files. Reading a key from a file puts it back on storage in plain text.

A10-L1 exercise 3

Hints. (1) Which line holds a credential? (2) Where should the credential come from instead? (3) Delete it and take the token as a parameter.

Answer. The bug is on line 4.

Fixed version:

```
1 import requests
2
3 SUPPLIER_URL = "https://supplier.example.org/api/deliveries"
4
5
6 def fetch_deliveries(token: str) -> requests.Response:
7     """Fetch the delivery list. token comes from dbutils.secrets.get at
    run time."""
8     return requests.get(SUPPLIER_URL, headers={"Authorization": f"Bearer
    {token}"}, timeout=30)
```

Line 4 commits a credential. Make the function take `token: str` and have the notebook read it with `dbutils.secrets.get`. Rotate the leaked token, because Git history keeps it.

Common wrong answers.

- Line 3: A public API URL is not a secret. It could even move to config, but it is not what blocks the review.
- Line 9: Line 9 uses the token. The leak is where the token is written.

A10-L1 exercise 4

Hints. (1) Nothing reaches main without review. (2) CI runs on the pull request. (3) Deployment is the last step, done by automation.

Answer.

1. Create a feature branch from main in your Git folder
2. Change the code and add or update tests
3. Push and open a pull request
4. CI runs lint, type checks and tests

5. A colleague reviews and approves
6. Merge to main; automation deploys to production

Branch, change, pull request, CI, review, merge and automatic deployment. Every production change passes the same gates.

Common wrong answers.

- Deploying before review and CI means production runs unchecked code.
- Create the branch first, so your changes never touch main directly.

A10-L1 exercise 5

Hints. (1) What must stay out of logs and history? (2) What changes per environment? (3) What is specific to one machine?

Answer.

- Credentials: a secret scope, read with dbutils.secrets.get.
- Code: Git, reviewed and tested.
- Environment choice: a job parameter or target setting, so the same code deploys to dev and prod.
- Local environments: ignored by Git. Pin dependencies in pyproject.toml instead.

Secrets in a secret scope, code in Git, environment choice as a parameter, local environments ignored.

Common wrong answers.

- token = `param`: Job parameters show in the job definition and run history. Credentials belong in a secret scope.
- token = `repo`: Git history keeps every committed secret for good.
- catalog = `repo`: Hardcoding the catalog in code means dev runs write to prod. Pass it in per environment.
- env = `commit`: A virtual environment is machine-specific and large. Commit the pinned dependency list instead.

A10-L2 Declarative Automation Bundles and parameterised jobs

A10-L2 exercise 1

Hints. (1) Was the rename breaking? (2) Did you need to change your files? (3) Only the name changed.

Answer. b) The name. The `databricks bundle` CLI and existing configuration keep working unchanged. Because: r1) Databricks describes the rename as non-breaking.

The rename is non-breaking. The CLI, `databricks.yml` and all existing configuration work as before.

Common wrong answers.

- `a`: The CLI command group is still `databricks bundle`.
- `c`: The configuration file is still `databricks.yml`.
- `d`: Targets are unchanged and remain the way to describe environments.
- `b|r2`: Right answer, wrong reason. It is the same product under a new name.

A10-L2 exercise 2

Hints. (1) The mode values are full words. (2) The prod target sets the same variable as dev. (3) `development`, `production` and `catalog`.

Answer.

```yaml
targets:
  dev:
    mode: development
    default: true
    variables:
      catalog: research_dev
  prod:
    mode: production
    variables:
      catalog: research_prod
```

`mode: development` prefixes names and pauses schedules. `mode: production` adds production checks. Each target sets the same variable to its own value.

Common wrong answers.

- g1 = `dev`: The mode values are `development` and `production`. `dev` is only the target's name.
- g3 = `catalog_name`: The variable is declared as `catalog`. A different name here sets nothing, and prod falls back to the default.

A10-L2 exercise 3

Hints. (1) Which value is the same in every target? (2) Where should environment-specific values come from? (3) Use `${var.catalog}`.

Answer. The bug is on line 7.

Fixed version:

```yaml
resources:
  jobs:
    nat_silver_pipeline:
      name: nat_silver_pipeline
      parameters:
        - name: catalog
          default: ${var.catalog}
      tasks:
        - task_key: silver
          notebook_task:
            notebook_path: ./notebooks/silver_nat_transform.py
```

A literal default of `research_prod` applies to every target, dev included. `${var.catalog}` takes the value each target sets.

Common wrong answers.

- Line 11: A relative notebook path inside the bundle is correct.
- Line 6: Declaring a catalog parameter is right. Its default value is the problem.

A10-L2 exercise 4

Hints. (1) Call `widgets.get("catalog")` and `widgets.get("wave_id")`. (2) Check the catalog against an allowed set. (3) Raise ValueError for a bad catalog or an empty wave_id. Never fill in a default.

Answer.

```python
ALLOWED_CATALOGS = {"research_dev", "research_prod"}


def read_job_parameters(widgets: object) -> dict[str, str]:
    """Read and validate the job parameters passed to this notebook.

    Args:
        widgets: dbutils.widgets, or any object with get(name).

    Returns:
        catalog and wave_id.

    Raises:
        ValueError: If the catalog is not allowed or wave_id is empty.
    """
    get = getattr(widgets, "get")
    catalog = str(get("catalog"))
    wave_id = str(get("wave_id")).strip()
    if catalog not in ALLOWED_CATALOGS:
        raise ValueError(f"catalog {catalog!r} is not one of
{sorted(ALLOWED_CATALOGS)}")
    if not wave_id:
        raise ValueError("wave_id is empty")
```

```
23      return {"catalog": catalog, "wave_id": wave_id}
```

Parameters arrive as strings from outside the code. Validating them at the start turns a mistyped catalog into a clear failure, instead of a run against the wrong environment.

A10-L2 exercise 5

Hints. (1) Catch configuration errors first. (2) You can only run what is deployed. (3) Then look at the results.

Answer.

1. databricks bundle validate -t dev
2. databricks bundle deploy -t dev
3. databricks bundle run -t dev nat_silver_pipeline
4. Check the run log and QA results for the dev run

Validate, deploy, run, then check the evidence. The same sequence runs in CI for prod, without the manual run.

Common wrong answers.

- Validate first. It catches YAML and reference errors before anything is deployed.
- run starts the deployed job. Deploy the change first, or you run the old version.

A10-L3 CI: lint, type-check, test and validate on every pull request

A10-L3 exercise 1

Hints. (1) Who should own production deployments? (2) What identity is not a person? (3) A service principal.

Answer. b) A service principal, authenticated with OAuth M2M or workload identity federation Because: r1) A service principal is not tied to a person, and its access can be scoped to exactly what deployment needs.

A service principal gives automation its own identity with only the rights it needs. Databricks recommends OAuth M2M for it, or workload identity federation with no stored secret.

Common wrong answers.

- `a`: A personal token stops working when that person leaves, and it carries all their access.

- `c`: Pull request authors should not need production rights. The deploy runs after merge, as automation.
- `d`: Databricks requires authentication for every API call.
- `b|r2`: Right identity, wrong reason. Personal tokens can technically deploy. The problem is who owns the access.

A10-L3 exercise 2

Hints. (1) OAuth client credentials have two parts. (2) They match the secret names on the right. (3) `CLIENT_ID` and `CLIENT_SECRET`.

Answer.

```yaml
env:
  DATABRICKS_HOST: ${{ vars.DATABRICKS_HOST }}
  DATABRICKS_CLIENT_ID: ${{ secrets.DATABRICKS_CLIENT_ID }}
  DATABRICKS_CLIENT_SECRET: ${{ secrets.DATABRICKS_CLIENT_SECRET }}
```

`DATABRICKS_CLIENT_ID` and `DATABRICKS_CLIENT_SECRET`, with `DATABRICKS_HOST`, let the CLI authenticate as the service principal.

Common wrong answers.

- g1 = `TOKEN`: DATABRICKS_TOKEN is for personal access tokens. OAuth M2M uses the client ID and secret.
- g2 = `PASSWORD`: Service principals authenticate with a client secret, not a password.

A10-L3 exercise 3

Hints. (1) Which setting stops a failing step from failing the job? (2) It sits under the pytest step. (3) Remove `continue-on-error: true`.

Answer. The bug is on line 6.

Fixed version:

```yaml
steps:
  - uses: actions/checkout@v4
  - run: pip install -e . --group dev
  - run: ruff check .
  - run: pytest
```

`continue-on-error: true` makes the step pass whatever pytest returns. Remove it. If tests are flaky, fix or quarantine them in code, and keep CI honest.

Common wrong answers.

- Line 5: Running pytest is right. The line after it changes what a failure means.

- Line 4: The lint step is fine and can fail the job.

A10-L3 exercise 4

Hints. (1) Which checks take seconds? (2) Which starts a JVM? (3) Which talks to the workspace?

Answer.

1. ruff check . and ruff format --check .
2. mypy
3. pytest with a local SparkSession
4. databricks bundle validate -t dev

Lint and types take seconds, tests take a minute or two with Spark, and validation needs the workspace. Failing fast saves CI time and gives quicker feedback.

Common wrong answers.

- Spark tests take the longest. Run the seconds-long checks first, so a typo fails the job quickly.
- Validation needs workspace credentials and a network call. Run the local checks first.

A10-L3 exercise 5

Hints. (1) Checks belong before review and merge. (2) Production changes only after a merge. (3) Credentials never live in files.

Answer.

- Checks on every pull request and on main.
- Deploy to prod only after a merge, needing the checks.
- Protect main with required checks.
- Authenticate as a service principal: federation first, OAuth M2M second, secrets never in files.

Checks gate every pull request, protection makes them binding, deployment follows a merge, and CI authenticates as a service principal without secrets in the repository.

Common wrong answers.

- trigger_deploy = `every_push`: Deploying every branch push puts unreviewed code in production.
- branch_protection = `none`: Without required checks, CI reports problems but cannot stop a merge.

- auth = `pat`: A token in a workflow file is committed to Git, and ties deployment to one person.
- trigger_checks = `nightly`: Nightly checks find problems after they merge. Run them on the pull request.